INSTITUTO SUPERIOR MANUEL TEIXEIRA GOMES



Sistema IoT de Rastreamento e Geofencing

Relatório de Trabalho Final de Curso

Daniel António Balicevic

Trabalho orientado por:
Prof. Tiago Candeias

2025/2026

INSTITUTO SUPERIOR MANUEL TEIXEIRA GOMES



Sistema IoT de Rastreamento e Geofencing

Relatório de Trabalho Final de Curso

Daniel António Balicevic

Trabalho orientado por:
Prof. Tiago Candeias

2025/2026

Sistema IoT de Rastreamento e Geofencing Declaração de autoria

Declaro ser o (a) autor(a) do trabalho apresentado neste relatório, sendo original e inédito. Autores e trabalhos consultados estão devidamente citados no texto e constam da listagem de referências incluída.

_____

(Assinatura)

O Instituto Superior Manuel Teixeira Gomes tem o direito, perpétuo e sem limites geográficos, de arquivar e publicitar este trabalho através de quaisquer meios, de o divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos não comerciais, desde que seja dado crédito ao autor.

_____

(Assinatura)

Agradecimentos

Ao Prof. Tiago Candeias, orientador deste trabalho, pela disponibilidade, pelo rigor das observações e pela orientação ao longo de todas as fases do projeto.

Ao Instituto Superior Manuel Teixeira Gomes e ao corpo docente do curso de Engenharia Informática, pela formação que tornou possível a realização deste trabalho.

À minha família e aos meus amigos, pelo apoio constante e pela paciência durante os meses de desenvolvimento deste projeto.

Resumo

Este relatório descreve a conceção, o desenvolvimento e a validação do SafeTrack, um sistema IoT de rastreamento e geofencing para monitorização de pessoas vulneráveis, em particular idosos com défice cognitivo e crianças pequenas. Foi construído um produto mínimo viável composto por três camadas integradas: um dispositivo portátil baseado no módulo LilyGO T-SIM7000G, que reúne um microcontrolador ESP32, um recetor GNSS e um modem celular de baixo consumo; uma infraestrutura serverless em Google Firebase, com funções executadas na região europe-west1, responsável pela ingestão de telemetria, pelo processamento espacial e pela notificação; e uma aplicação móvel desenvolvida em Flutter para o cuidador.

O sistema implementado utiliza um intervalo de repouso configurado de 20 segundos (`INTERVAL_S`), distinto do intervalo entre posições: no ensaio de campo foram observados ciclos de 83 segundos em condições de aquisição degradada. Suporta múltiplas zonas seguras circulares e poligonais com semântica de segurança por união, deteta saídas de perímetro através da fórmula de Haversine com supressão de leituras isoladas, e trata a saúde do dispositivo — silêncio prolongado e bateria fraca — como evento de alerta de primeira classe. Quando a aquisição GNSS não é possível em ambiente interior, o dispositivo recorre a um posicionamento alternativo por varrimento Wi-Fi resolvido do lado do servidor. A persistência em MicroSD permite guardar leituras não entregues e tentar o seu reenvio; a recuperação integral sem perdas não foi demonstrada experimentalmente.

Durante o desenvolvimento foram diagnosticados dois problemas de integração não previsíveis na fase de especificação: o conflito de coexistência de radiofrequência entre o recetor GNSS e o transmissor celular, mitigado por uma estratégia de aquisição adaptativa em duas fases, que permite configurar um repouso de 20 segundos, sem garantir essa periodicidade de reporte em campo, e um defeito de validação de certificado na firmware

do módulo utilizado, documentado como limitação assumida com modelo de ameaça explícito.

A validação reuniu 43 testes unitários sobre a lógica crítica do backend, testes funcionais selecionados e observações de campo. Subsistem medições em falta e verificações parciais, explicitadas na matriz de rastreabilidade. A correção de um defeito de autorização revelado por um caso negativo constitui também um resultado do processo de verificação.

Palavras-chave: IoT, rastreamento, geofencing, GNSS, sistemas embebidos.

Abstract

This report describes the design, development and validation of SafeTrack, an IoT tracking and geofencing system for monitoring vulnerable people, namely elderly individuals with cognitive impairment and young children. A minimum viable product was built, comprising three integrated layers: a portable device based on the LilyGO T-SIM7000G module, which combines an ESP32 microcontroller, a GNSS receiver and a low-power cellular modem; a serverless infrastructure on Google Firebase, with functions running in the europe-west1 region, responsible for telemetry ingestion, spatial processing and notification; and a mobile application developed in Flutter for the caregiver.

The implemented system uses a configured sleep interval of 20 seconds (`INTERVAL_S`), which differs from the interval between positions: the field trial recorded 83-second cycles under degraded acquisition conditions. It supports multiple circular and polygonal safe zones with union semantics, detects perimeter breaches using the Haversine formula with suppression of isolated readings, and treats device health — extended silence and low battery — as a first-class alert event. When GNSS acquisition is not possible indoors, the device falls back to server-side Wi-Fi scan positioning. MicroSD persistence supports queuing and retransmission of undelivered readings; complete lossless recovery has not been demonstrated experimentally.

Two integration problems that were not foreseeable at specification time were diagnosed during development: the radio-frequency coexistence conflict between the GNSS receiver and the cellular transmitter, mitigated through a two-stage adaptive acquisition strategy that allows a configured sleep interval of 20 seconds without guaranteeing that reporting frequency in the field, and a certificate validation defect in the module firmware, documented as an accepted limitation with an explicit threat model.

Validation included 43 unit tests of critical backend logic, selected functional tests and field observations. Missing measurements and partial validations are explicitly identified in the traceability matrix. Correcting an authorization defect exposed by a negative test is also an outcome of verification.

Keywords: IoT, tracking, geofencing, GNSS, embedded systems.

Conteúdo

1 Introdução **1**

1.1 Escolha do Tema e Motivação 1
1.2 Formulação do Problema 1
1.3 Determinação dos Objetivos 2
1.4 Estrutura do Documento 3

2 Estado da Arte **5**

2.1 Rastreamento de Populações Vulneráveis com IoT 5
2.2 Geofencing e a Fórmula de Haversine 5
2.3 Conectividade Celular — NB-IoT e LTE-M 6
2.4 Arquitetura Serverless com Firebase 7
2.5 Desenvolvimento Mobile com Flutter 7
2.6 Soluções Existentes e Limitações 8

3 Metodologia **11**

3.1 Descrição do Problema e Abordagem Metodológica 11
3.2 Arquitetura Geral do Sistema 12
3.2.1 Visão Geral das Camadas 12
3.2.2 Camada de Hardware 13
3.2.3 Camada de Cloud / Backend 16
3.2.4 Camada de Aplicação Móvel 17
3.3 Requisitos funcionais 18
3.4 Requisitos não funcionais 19
3.5 Restrições e Limites do Sistema 21
3.6 Atores e Níveis de Segurança 22
3.7 Modelação Funcional 24
3.7.1 Diagrama de Componentes UML 24
3.7.2 Diagrama de Classes UML 25
3.7.3 UC-01 — Registar e efetuar Login 26

3.7.4 UC-02 — Emparelhar Dispositivo 27
3.7.5 UC-03 — Visualizar Mapa em Tempo Real 28
3.7.6 UC-04 — Configurar Cerca Virtual 29
3.7.7 UC-05 — Receber Alerta de Emergência 30
3.7.8 UC-06 — Consultar Nível de Bateria 31
3.7.9 Diagrama de Sequência — Cenário de Evasão 32
3.7.10 Diagrama de Atividades — Lógica Haversine 34
3.7.11 Diagrama de Estados do Dispositivo 36
3.7.12 Diagrama de Atividades UML — Firmware ESP32 38
3.7.13 Diagrama de Atividades — Registo e Login 40
3.7.14 Diagrama de Atividades — Emparelhamento de Dispositivo . . . 42
3.7.15 Diagrama de Atividades — Visualização do Mapa 43
3.7.16 Diagrama de Atividades — Configuração de Geofence 45
3.7.17 Diagrama de Atividades — Receção de Alerta 46
3.7.18 Diagrama de Atividades — Consulta de Bateria 48
3.8 Arquitetura de Navegação da Aplicação 49
3.8.1 Estrutura de Ecrãs 49
3.8.2 Esquema de Navegação 50
3.8.3 Protótipo da Aplicação 51
3.9 Cronograma de Desenvolvimento 57

4 Desenvolvimento e Implementação **59**
4.1 Ambiente de Desenvolvimento e Ferramentas 59
4.1.1 Cadeia de Ferramentas e Versões 59
4.1.2 Organização do Repositório e Controlo de Versões 62
4.1.3 Gestão de Configuração e Segredos 63
4.2 Implementação da Camada de Dispositivo 64
4.2.1 Arquitetura do Firmware e Ciclo Principal 64
4.2.2 Inicialização e Gestão do Modem SIM7000G 65
4.2.3 Aquisição de Posição GNSS 65
4.2.4 Estratégia de Posicionamento em Interior 66
4.2.5 Registo Local em MicroSD e Reenvio Diferido 67
4.2.6 Construção e Transmissão do Payload de Telemetria 68
4.2.7 Gestão de Energia 69
4.2.8 Leitura do Estado da Bateria 70
4.3 Implementação da Camada de Cloud 70

4.3.1 Modelo de Dados Documental 71
4.3.2 Função de Ingestão HTTP 71
4.3.3 Processamento Espacial e Deteção de Evasão 72
4.3.4 Histerese e Supressão de Alertas Duplicados 73
4.3.5 Monitorização de Saúde do Dispositivo 73
4.3.6 Notificações Push e Canal Alternativo 75
4.3.7 Autenticação e Regras de Segurança 75
4.3.8 Implantação, Cold Starts e Custos Operacionais 77
4.4 Implementação da Aplicação Móvel 77
4.4.1 Arquitetura da Aplicação e Gestão de Estado 77
4.4.2 Autenticação: Login, Registo e Recuperação 78
4.4.3 Emparelhamento por Código 80
4.4.4 Mapa em Tempo Real 81
4.4.5 Editor de Cercas Virtuais 83
4.4.6 Histórico de Localizações 85
4.4.7 Alertas: Notificação, Lista e Detalhe 86
4.4.8 Estado do Dispositivo e Bateria 88
4.4.9 Permissões, Estados de Erro e Degradação 89
4.5 Integração e Montagem do Protótipo 89
4.6 Desvios Face à Especificação 91

5 Testes e Validação **93**
5.1 Estratégia de Testes 93
5.2 Testes Unitários Automatizados 94
5.3 Testes de Integração Ponta-a-Ponta 95
5.4 Testes Funcionais por Caso de Uso 96
5.4.1 UC-01 — Registar e Efetuar Login 96
5.4.2 UC-02 — Emparelhar Dispositivo 99
5.4.3 UC-03 — Visualizar Mapa em Tempo Real 99
5.4.4 UC-04 — Configurar Cerca Virtual 100
5.4.5 UC-05 — Receber Alerta de Emergência 103
5.4.6 UC-06 — Consultar Nível de Bateria 103
5.5 Validação dos Requisitos Não Funcionais Quantificados 103
5.5.1 Protocolo Experimental Comum 103
5.5.2 RNF01 — Autonomia Energética 105
5.5.3 RNF02 — Precisão de Localização 107

5.5.4 RNF03 — Latência de Alerta . . . 107
5.5.5 RNF04 — Disponibilidade de Rede e Fallback . . . 109
5.5.6 RNF05 e RNF08 — Usabilidade e Desempenho da Interface . . . 109
5.5.7 RNF06 — Segurança dos Dados . . . 110
5.5.8 RNF07 — Consumo de Dados . . . 111
5.5.9 RNF09 — Robustez Ambiental . . . 112
5.5.10 RNF10 — Escalabilidade . . . 112
5.6 Testes de Campo em Mobilidade Urbana Real . . . 113
5.7 Testes de Robustez e Cenários de Falha . . . 114
5.8 Matriz Consolidada de Rastreabilidade . . . 115
5.9 Ameaças à Validade . . . 115

6 Discussão **119**
6.1 Grau de Cumprimento dos Objetivos . . . 119
6.2 Comparação com Soluções Existentes . . . 120
6.3 Requisitos Não Cumpridos e Sua Justificação Técnica . . . 121
6.4 Limitações Assumidas . . . 124
6.5 Privacidade, Proteção de Dados e Considerações Éticas . . . 126
6.6 Viabilidade Económica . . . 127
6.7 Trabalho Futuro . . . 127

7 Conclusão **131**
7.1 Síntese do Trabalho Realizado . . . 131
7.2 Contribuições . . . 132
7.3 Reflexão Final . . . 133

Bibliografia **135**

A Proposta de Ligações com Componentes Discretos **139**

B Manual do Utilizador **141**
B.1 Âmbito . . . 141
B.2 Antes de começar . . . 141
B.3 Criar conta e iniciar sessão . . . 142
B.4 Emparelhar o dispositivo . . . 142
B.5 Ecrã do mapa . . . 143
B.6 Zonas seguras . . . 144

B.7 Alertas 145
B.8 Histórico 146
B.9 Estado do dispositivo 146
B.10 Cuidados com o dispositivo 147
B.11 Resolução de problemas 147
B.12 Privacidade 148

C Manual Técnico **149**
C.1 Âmbito 149
C.2 Estrutura do repositório 149
C.3 Pré-requisitos e versões 150
C.4 Configuração do projeto Firebase 150
C.5 Segredos e configuração 151
C.6 Implantação do backend 153
C.7 Provisionar um dispositivo 153
C.8 Compilar e gravar o firmware 154
C.9 Simulador de dispositivo 155
C.10 Testes 155
C.11 Modelo de dados 156
C.12 Regras de segurança 157
C.13 Limitações conhecidas 157
C.14 Diagnóstico de problemas correntes 158

Lista de Figuras

3.1 Arquitetura geral do sistema e interação entre as suas camadas. 13
3.2 Diagrama de componentes UML do sistema IoT de rastreamento e geofencing. 24
3.3 Diagrama de classes UML do sistema IoT de rastreamento e geofencing. . . 25
3.4 Diagrama de caso de uso UC-01 — Registar e efetuar Login. 26
3.5 Diagrama de caso de uso UC-02 — Emparelhar Dispositivo. 27
3.6 Diagrama de caso de uso UC-03 — Visualizar Mapa em Tempo Real. . . . 28
3.7 Diagrama de caso de uso UC-04 — Configurar Cerca Virtual. 29
3.8 Diagrama de caso de uso UC-05 — Receber Alerta de Emergência. 30
3.9 Diagrama de caso de uso UC-06 — Consultar Nível de Bateria. 31
3.10 Diagrama de sequência UML do cenário de evasão e geração de alerta. . . . 33
3.11 Diagrama de atividades UML da lógica de geofencing baseada na fórmula de Haversine. 35
3.12 Diagrama de estados UML do dispositivo IoT baseado no ESP32 e no módulo SIM7000G. 37
3.13 Diagrama de atividades UML do ciclo de funcionamento do firmware ESP32. 39
3.14 Diagrama de atividades UML dos processos de registo e login. 41
3.15 Diagrama de atividades UML do emparelhamento de um dispositivo IoT. . . 42
3.16 Diagrama de atividades UML da visualização da posição do dispositivo no mapa. 44
3.17 Diagrama de atividades UML da configuração da geofence. 45
3.18 Diagrama de atividades UML da receção e consulta de um alerta. 47
3.19 Diagrama de atividades UML da consulta do nível de bateria. 48
3.20 Estrutura hierárquica dos ecrãs da aplicação móvel SafeTrack. 49
3.21 Esquema de navegação entre os ecrãs da aplicação móvel SafeTrack. 50
3.22 Logótipo da aplicação SafeTrack. 51
3.23 Ecrã de autenticação (Login / Registo). 52
3.24 Ecrã de configuração da zona segura. 53
3.25 Ecrã principal de rastreamento. 54
3.26 Ecrã de histórico de alertas. 55

3.27 Ecrã de estado do dispositivo. 56

4.1 Ecrã de autenticação da aplicação SafeTrack. 79
4.2 Ecrã de emparelhamento de um dispositivo através de identificador e código. 80
4.3 Ecrã principal com a localização do dispositivo, a zona segura e o estado da bateria. 82
4.4 Biblioteca de zonas seguras e editor de uma cerca virtual circular. 84
4.5 Ecrã de histórico com o percurso, as posições e as estatísticas do período selecionado. 85
4.6 Lista de alertas gerados pelo sistema e respetivo estado de resolução. 87
4.7 Ecrãs de perfil do cuidador e de gestão do dispositivo emparelhado. 88
4.8 Montagem do protótipo em caixa transparente: vista da placa e das antenas à esquerda e da célula 18650 à direita. 90

5.1 Evidências do fluxo de recuperação de palavra-passe (TF-01.5), verificado em 6 de setembro de 2026. A ligação de reposição foi ocultada. 98

A.1 Proposta conceptual de ligações para uma topologia discreta de referência. . 140

Lista de Tabelas

2.1 Comparação entre as soluções existentes e o SafeTrack. 9

3.1 Requisitos funcionais do sistema. 18
3.2 Requisitos não funcionais do sistema. 20
3.3 Restrições e limites do sistema IoT de rastreamento e geofencing. 21
3.4 Atores humanos. 23
3.5 Atores de sistema. 23
3.6 Níveis de segurança e privilégios de acesso. 23
3.7 Fases de desenvolvimento do SafeTrack. 58

4.1 Cadeia de ferramentas por camada do sistema. 59
4.2 Estrutura do repositório e dimensão de cada componente. 62
4.3 Parâmetros temporais da aquisição de posição. 66
4.4 Modelo de dados documental implementado no Firestore. 71

5.1 Distribuição dos testes unitários do *backend* por ficheiro. 94
5.2 Distribuição dos testes unitários da aplicação por ficheiro. 95
5.3 Protocolo de integração ponta a ponta e estado dos registos. 96
5.4 Testes funcionais do caso de uso UC-01. 97
5.5 Testes funcionais do caso de uso UC-02. 100
5.6 Testes funcionais do caso de uso UC-03. 101
5.7 Testes funcionais do caso de uso UC-04. 102
5.8 Testes funcionais do caso de uso UC-05. 104
5.9 Testes funcionais do caso de uso UC-06. 105
5.10 Resultados da validação do requisito RNF01 — autonomia energética. . . . 106
5.11 Dispersão observada e limites da validação do RNF02. 107
5.12 Cenários de previsão da latência de alerta (RNF03). 108
5.13 Resultados da validação do requisito RNF04. 109
5.14 Resultados da avaliação exploratória de usabilidade (RNF05). 110
5.15 Resultados da validação do requisito RNF06 — segurança dos dados. 111

5.16 Projeção do consumo mensal de dados (RNF07). 112
5.17 Resultados da validação do requisito RNF10 — escalabilidade. 112
5.18 Resultados do ensaio de campo na Praia da Galé. 114
5.19 Cenários de falha induzida e comportamento observado. 115
5.20 Matriz de rastreabilidade entre requisitos, implementação e validação. . . . 116

6.1 Custo de materiais do protótipo. 127

B.1 Resolução de problemas correntes. 147

C.1 Versões do ambiente de desenvolvimento. 150
C.2 Parâmetros de configuração do firmware. 152
C.3 Diagnóstico técnico. 158

Siglas

ADC Analog-to-Digital Converter. 73

API Application Programming Interface. 63, 67

APN Access Point Name. 63

APNs Apple Push Notification service. 75, 91, 128

AT conjunto de comandos Attention do modem. 68

BLE Bluetooth Low Energy. 8, 120

BSSID Basic Service Set Identifier. 67

CA Certificate Authority. 68

EDGE Enhanced Data rates for GSM Evolution. 14

eDRX extended Discontinuous Reception. 69, 128

FCM Firebase Cloud Messaging. 58, 71, 75, 91, 115

GNSS Global Navigation Satellite System. 15, 20, 60, 61, 64, 65, 66, 67, 68, 69, 71, 72, 89, 91, 104, 105, 106, 107, 111, 119, 121, 122, 124, 125, 127, 128, 131

GPRS General Packet Radio Service. 14

GPS Global Positioning System. 1, 2, 5, 6, 7, 11, 12, 13, 16

GSM Global System for Mobile Communications. 13

HMAC Hash-based Message Authentication Code. 123

HTTP HyperText Transfer Protocol. 61, 67, 68, 75

HTTPS HyperText Transfer Protocol Secure. 58, 61, 64, 68, 91, 96

IDE Integrated Development Environment. 60

IoT Internet of Things. 1, 2, 3, 8, 11, 13, 131

IP Internet Protocol. 111

JSON JavaScript Object Notation. 7, 14, 64, 68

JWT JSON Web Token. 23

LPWAN Low Power Wide Area Network. 3

MAC Media Access Control. 68, 94

MVP Produto Mínimo Viável. 1, 3, 14, 16, 21

NMEA National Marine Electronics Association. 36

NTP Network Time Protocol. 66, 68

OAuth Open Authorization. 23

PCB Printed Circuit Board. 14

PDP Packet Data Protocol. 60, 64, 65

PSM Power Saving Mode. 69, 128

PSRAM Pseudo-Static Random-Access Memory. 14

RF Radiofrequência. 65

RGPD Regulamento Geral sobre a Proteção de Dados. 126

RSSI Received Signal Strength Indicator. 67, 68

RTC Real-Time Clock. 36

SDK Software Development Kit. 60, 75, 76, 122

SIM Subscriber Identity Module. 93, 127

SNI Server Name Indication. 68

SPI Serial Peripheral Interface. 67

SRAM Static Random-Access Memory. 14

TCP Transmission Control Protocol. 111

TLS Transport Layer Security. 65, 68, 111, 112, 122, 123

UART Universal Asynchronous Receiver-Transmitter. 65

UML Unified Modeling Language. 58

USB Universal Serial Bus. 61, 106

UTC Coordinated Universal Time. 66

UX User Experience. 58

Wi-Fi Wireless Fidelity. 13, 14

1 Introdução

1.1 Escolha do Tema e Motivação

A segurança de pessoas vulneráveis, em particular idosos com défice cognitivo e crianças pequenas, é uma preocupação crescente nas sociedades modernas. Segundo a Alzheimer's Association, 60% das pessoas diagnosticadas com demência irão perder-se em algum momento, sendo frequente que isso aconteça de forma repetida [1]. Este dado ilustra a dimensão real do problema que o presente projeto procura endereçar. A estes casos acrescem situações envolvendo crianças em espaços públicos, nas quais uma distração momentânea por parte do cuidador pode originar consequências graves.

A motivação para o desenvolvimento deste projeto nasce precisamente desta lacuna: a inexistência de uma solução acessível, autónoma e configurável que permita a um cuidador monitorizar a localização do seu dependente em tempo real e ser alertado automaticamente quando este sai de uma zona de segurança previamente definida. O mercado global de rastreadores Global Positioning System (GPS) para cuidados de saúde e monitorização pessoal encontra-se em expansão acelerada, com previsões de crescimento significativo até 2032 [2], o que evidencia a relevância e a oportunidade do tema.

Do ponto de vista pessoal, a escolha deste tema reflete o interesse do autor nas áreas de sistemas embebidos, Internet of Things (IoT) e desenvolvimento de software, combinando numa única solução hardware, backend cloud e aplicação móvel, ou seja, as três vertentes técnicas que este Produto Mínimo Viável (MVP) visa desenvolver e integrar.

1.2 Formulação do Problema

As soluções de rastreamento pessoal atualmente disponíveis no mercado apresentam falhas estruturais que limitam a sua utilidade em contextos de cuidado continuado.

Os rastreadores passivos baseados em Bluetooth, como as Apple AirTags, não possuem GPS autónomo e dependem da proximidade de outros dispositivos da mesma rede para reportar localização. Em ambientes com baixa densidade de utilizadores, ou em zonas rurais e periféricas, este modelo falha com frequência. Por outro lado, os dispositivos dedicados com GPS celular autónomo existentes no mercado implicam frequentemente subscrições mensais com custos elevados, carecem de geofencing configurável pelo utilizador final e não oferecem integração com uma aplicação móvel personalizada adaptada às necessidades específicas do cuidador [3].

Acresce que, como demonstrado por Doyle et al., os rastreadores GPS podem reduzir o sofrimento emocional e o risco de dano físico ao ajudarem familiares a encontrar os seus parentes mais rapidamente, diminuindo a sobrecarga sobre familiares, autoridades e serviços de saúde e apoio social [4]. Esta constatação reforça a necessidade de soluções funcionais, acessíveis e de fácil utilização neste domínio.

1.3 Determinação dos Objetivos

O objetivo principal deste trabalho consiste na prototipagem de um sistema IoT autónomo e bidirecional que dote o cuidador das seguintes capacidades:

- monitorizar a localização exata do portador do dispositivo em tempo real, através de um mapa interativo numa aplicação móvel;
- definir perímetros de segurança virtuais configuráveis, ou *geofencing*, em torno de uma área específica, como a residência ou a escola;
- receber alertas automáticos de emergência, via notificações *push*, sempre que o portador ultrapasse o limite geográfico estabelecido;
- consultar o estado de carga da bateria do dispositivo em tempo real.

Para atingir estes objetivos, o projeto engloba o desenvolvimento integrado de três componentes: um dispositivo portátil baseado no módulo LilyGO T-SIM7000G, com ESP32, GPS e modem celular integrados; um backend *serverless* em Google Firebase

com processamento espacial através da Fórmula de Haversine; e uma aplicação móvel *cross-platform* desenvolvida em Flutter.

O produto final posiciona-se como um MVP de baixo custo, sem dependência de subscrições externas, procurando responder diretamente às limitações identificadas nas soluções comerciais existentes.

1.4 Estrutura do Documento

O presente documento encontra-se organizado da seguinte forma.

O Capítulo 2 apresenta o Estado da Arte, contextualizando as tecnologias utilizadas no projeto, nomeadamente rastreamento IoT, geofencing, conectividade celular Low Power Wide Area Network (LPWAN), arquitetura *serverless* e desenvolvimento mobile, com base em literatura científica e técnica relevante.

O Capítulo 3 descreve a Metodologia adotada, incluindo a especificação de requisitos funcionais e não funcionais, a modelação funcional do sistema com diagramas UML e o cronograma detalhado de desenvolvimento.

O Capítulo 4 documenta o Desenvolvimento e Implementação do sistema, abrangendo o firmware do dispositivo, a configuração do backend Firebase e o desenvolvimento da aplicação móvel em Flutter.

O Capítulo 5 apresenta os Testes e a Validação realizados, incluindo testes funcionais por caso de uso e testes de campo em mobilidade urbana real.

O Capítulo 6 discute os resultados obtidos, compara a solução com alternativas existentes e identifica limitações e possibilidades de trabalho futuro.

O Capítulo 7 apresenta as conclusões do projeto.

2 Estado da Arte

2.1 Rastreamento de Populações Vulneráveis com IoT

A segurança de idosos com défice cognitivo e de crianças constitui uma preocupação crescente nas sociedades modernas. Stavropoulos et al. referem que existe uma necessidade de monitorização, assistência, intervenção e suporte constantes, o que representa uma carga humana e financeira considerável para os próprios indivíduos e para os seus cuidadores [5]. Esta constatação reforça a necessidade de soluções tecnológicas acessíveis que reduzam esse esforço e permitam um acompanhamento mais eficaz.

Os dispositivos de rastreamento com GPS e conectividade cloud surgem como resposta direta a este problema. Segundo a u-blox, estes sistemas permitem aos responsáveis acompanhar a localização em tempo real através do smartphone, quer a criança esteja a caminho da escola, quer se encontre fora do campo de visão [3]. Esta abordagem representa uma evolução clara face aos rastreadores passivos baseados em Bluetooth, que dependem da presença de infraestrutura de terceiros nas proximidades para fornecer dados de localização úteis.

Mahapatra et al. desenvolveram uma pulseira IoT com GPS e geofencing para rastreamento de pessoas em contexto de pandemia, concluindo que a integração de perímetros virtuais com hardware embebido e base de dados em tempo real constitui uma abordagem eficaz para monitorizar populações em risco [6]. O seu trabalho demonstra a viabilidade técnica de sistemas com características próximas das previstas neste projeto.

2.2 Geofencing e a Fórmula de Haversine

O geofencing consiste na definição de um perímetro virtual geográfico que, quando cruzado por um dispositivo, desencadeia automaticamente uma ação, como um alerta ou

o registo de um evento [7]. No caso de perímetros circulares, o processo de avaliação resume-se ao cálculo da distância entre a posição atual do dispositivo e o ponto central da zona definida, comparando-se depois esse valor com o raio configurado.

O algoritmo matemático mais utilizado para este cálculo é a Fórmula de Haversine, que permite determinar a distância do grande círculo entre dois pontos numa esfera a partir das respetivas coordenadas geográficas [8]. A sua utilização é preferível à distância euclidiana simples porque, como referem Raikar et al., a distância euclidiana não é adequada para calcular distâncias entre localizações geográficas, uma vez que esses pontos pertencem à superfície esférica da Terra e não a um plano [9].

A fórmula pode ser expressa da seguinte forma:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cdot\cos(\phi_2)\cdot\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.1}$$

$$d = 2R\cdot\arcsin\left(\sqrt{a}\right) \tag{2.2}$$

em que ϕ representa a latitude, λ a longitude e R o raio médio da Terra. A aplicação prática desta abordagem em sistemas baseados em Flutter e Firebase foi validada num estudo publicado em 2023, no qual se concluiu que a precisão da deteção da posição depende essencialmente do dispositivo e da rede GPS utilizada [10].

2.3 Conectividade Celular — NB-IoT e LTE-M

Para dispositivos IoT portáteis que transmitem pequenos volumes de dados de forma periódica, as tecnologias *Low Power Wide Area Network* revelam-se particularmente adequadas. Entre estas, o NB-IoT e o LTE-M foram desenvolvidos especificamente para cenários de comunicação com baixo consumo energético e cobertura alargada [11].

A distinção entre ambos os protocolos é relevante para este projeto. O LTE-M oferece maior largura de banda e melhor suporte à mobilidade, mas apresenta um consumo energé-

tico superior ao do NB-IoT [12]. Assim, para o envio de pacotes JavaScript Object Notation (JSON) de pequena dimensão, o NB-IoT surge como uma opção particularmente eficiente. Ainda assim, o módulo SIM7000G suporta ambas as tecnologias, conferindo flexibilidade à solução em função da cobertura disponível na região de operação [13].

2.4 Arquitetura Serverless com Firebase

A adoção de uma arquitetura *serverless* elimina a necessidade de gerir servidores dedicados, transferindo a lógica de backend para funções executadas na cloud em resposta a eventos. Como refere Lim Chee Keen, qualquer código responsável por operações de base de dados, chamadas de rede ou tarefas tradicionalmente executadas no servidor pode ser alojado neste modelo, beneficiando de maior eficiência de custos [14].

No contexto deste projeto, esta abordagem é particularmente vantajosa, uma vez que a função de geofencing poderá ser acionada automaticamente sempre que um novo registo GPS for guardado no Firestore, sem necessidade de intervenção manual. O ecossistema Google Firebase reúne ainda, numa única plataforma, os serviços necessários ao sistema: autenticação, base de dados em tempo real, processamento *serverless* e notificações *push*. Esta integração reduz a complexidade de desenvolvimento e facilita a interoperabilidade entre os diferentes componentes da solução.

2.5 Desenvolvimento Mobile com Flutter

O Flutter é um *UI toolkit open-source* da Google que permite compilar aplicações nativas para iOS e Android a partir de uma única base de código [15]. Para um projeto desenvolvido individualmente, esta característica assume especial relevância, pois elimina a necessidade de manter duas implementações distintas e contribui para reduzir o tempo total de desenvolvimento.

A integração do Flutter com o Firebase, através das bibliotecas FlutterFire, e com o Google Maps SDK para visualização cartográfica, torna esta framework tecnicamente coerente com a restante stack do projeto. Além disso, em 2024, foi destacado o potencial

do Flutter para aplicações ligadas ao ecossistema IoT, incluindo cenários com dispositivos portáteis e outros dispositivos inteligentes [16].

2.6 Soluções Existentes e Limitações

A análise das soluções existentes considera três categorias distintas: *trackers* comerciais dedicados, plataformas de rastreamento *open-source* genéricas e sistemas académicos de monitorização IoT para populações com défice cognitivo.

O **AngelSense** é a solução comercial mais próxima em público-alvo, desenhada especificamente para crianças com perturbações do neurodesenvolvimento e adultos com deficiência cognitiva. Distingue-se pela voz bidirecional, pela fixação anti-remoção e, tecnicamente, pela deteção de anomalias por aprendizagem da rotina diária, que alerta o cuidador quando o utilizador se desvia do percurso habitual sem que seja necessário desenhar manualmente uma cerca para cada padrão [17]. O **Jiobit**, integrado na plataforma Life360, privilegia a discrição física e a autonomia energética, com geofences configuradas manualmente e sem aprendizagem de padrões [18]. Ambas as soluções assentam, contudo, em subscrições mensais recorrentes e mantêm os dados de localização na infraestrutura do fabricante, sem possibilidade de auditoria do código.

O **Traccar** é uma plataforma *open-source* de rastreamento com suporte a diversos protocolos, dispositivos e cercas circulares e poligonais [19]. Inclui eventos de inatividade e perda de comunicação e alarmes de bateria fraca reportados por equipamentos compatíveis [20]. A disponibilidade dos alarmes depende do equipamento e do protocolo. O SafeTrack não introduz estas funções de forma inédita: concretiza-as numa integração própria entre firmware, servidor e aplicação do cuidador.

A literatura académica sobre monitorização de doentes com demência propõe tipicamente arquiteturas mais ambiciosas em âmbito, com localização em interior ao nível da divisão através de Bluetooth Low Energy (BLE) e *fingerprinting* [21], mas fica mais distante de hardware real e exige infraestrutura instalada no espaço a monitorizar.

A Tabela 2.1 sistematiza esta comparação.

Tabela 2.1. – Comparação entre as soluções existentes e o SafeTrack.

Critério	**AngelSense**	**Jiobit / Life360**	**Traccar**	**SafeTrack**
Custo recorrente	Subscrição mensal	Subscrição mensal	Nenhum (*self-hosted*)	Dados móveis e nuvem conforme utilização
Propriedade dos dados	Fabricante	Fabricante	Do utilizador	Do cuidador (projeto Firebase próprio)
Código aberto	Não	Não	Sim	Não (repositório privado)
Geofencing	Zonas + rotina aprendida	Zonas manuais	Zonas + regras compostas	Multizona (círculo e polígono) com *debounce*
Localização em interior	Sim (proprietária)	Limitada	Depende do hardware	Wi-Fi (Google Geolocation)
Saúde do dispositivo	Implícita na aplicação	Implícita na aplicação	Inatividade e alarmes de bateria, conforme o dispositivo	Alerta dedicado (silêncio e bateria) com histerese
Voz bidirecional / anti-remoção	Sim	Não	Não aplicável	Não

O posicionamento do SafeTrack reside na integração de um protótipo configurável, com firmware próprio, aplicação do cuidador e projeto administrado pelo utilizador. Os alertas de saúde e os canais de notificação não constituem funcionalidades exclusivas face ao Traccar. O contributo é a implementação e documentação desta combinação no hardware escolhido, incluindo os compromissos de latência, autonomia e segurança. A ausência de subscrição comercial de rastreamento não elimina custos de dados móveis ou de nuvem.

3 Metodologia

3.1 Descrição do Problema e Abordagem Metodológica

O presente projeto surge da necessidade de desenvolver um sistema de monitorização autónomo, acessível e configurável para populações vulneráveis, colmatando as lacunas identificadas nas soluções comerciais atualmente disponíveis. O problema central reside na inexistência de uma solução de baixo custo que combine rastreamento GPS celular autónomo, geofencing configurável pelo utilizador e notificações *push* em tempo real, sem dependência de subscrições externas.

Para responder a este problema, foi adotada uma abordagem de desenvolvimento assente numa metodologia híbrida, que combina princípios da metodologia ágil Scrum com o Modelo em V, característico do desenvolvimento de sistemas embebidos. Esta escolha não foi arbitrária, refletindo a natureza dual do projeto, que integra simultaneamente uma componente de hardware físico e uma componente de software.

A justificação para esta escolha encontra suporte em literatura académica recente. Guerrero-Ulloa et al. concluíram que uma parte significativa dos projetos IoT analisados recorreu ao Scrum como metodologia principal, sendo também frequente a sua combinação com outras metodologias para melhor responder às especificidades destes sistemas [22]. Este dado reforça a ideia de que a utilização isolada de uma única metodologia raramente se adequa à complexidade de projetos que envolvem hardware e software em simultâneo.

Por outro lado, estudos na área da gestão híbrida de projetos referem que gestores e equipas recorrem cada vez mais a abordagens que combinam metodologias ágeis com metodologias tradicionais faseadas, de modo a capitalizar as vantagens de ambas e a reduzir o tempo de desenvolvimento, melhorando simultaneamente a eficiência [23]. Esta tendência é particularmente relevante no contexto de um projeto individual, em que a gestão eficiente do tempo e das dependências técnicas assume um papel crítico.

Neste enquadramento, considerou-se que o Scrum seria mais adequado para a componente de software, nomeadamente para o desenvolvimento da aplicação Flutter e para a configuração do Firebase, por permitir desenvolvimento iterativo com entregas faseadas e ajustes progressivos. Já para a componente de hardware e firmware, incluindo a programação do ESP32, a integração do módulo GPS e a gestão de energia, optou-se por seguir uma abordagem mais estruturada inspirada no Modelo em V, em que cada fase de desenvolvimento possui uma fase de validação correspondente.

Em termos práticos, o desenvolvimento foi organizado nas fases sequenciais descritas na Secção 3.9, cada uma com um critério de conclusão verificável.

Esta estrutura garante que as dependências entre camadas são respeitadas, uma vez que o firmware tem de estar funcional antes de a aplicação poder ser testada com dados reais, permitindo ao mesmo tempo iterações dentro de cada fase sempre que tal se revele necessário.

3.2 Arquitetura Geral do Sistema

3.2.1 Visão Geral das Camadas

O sistema desenvolvido neste projeto assenta numa arquitetura de três camadas distintas e interdependentes, cada uma com responsabilidades bem definidas e comunicando entre si através de interfaces normalizadas. Esta separação em camadas foi uma decisão de design deliberada, permitindo desenvolver, testar e validar cada componente de forma independente, reduzindo a complexidade de integração e facilitando a identificação de erros ao longo do processo de desenvolvimento.

Como ilustrado na Figura 3.1, o sistema é composto pela camada de hardware, responsável pela aquisição e transmissão de dados de localização; pela camada de cloud, responsável pelo armazenamento, processamento espacial e envio de notificações; e pela camada de aplicação móvel, que funciona como interface principal entre o sistema e o cuidador.

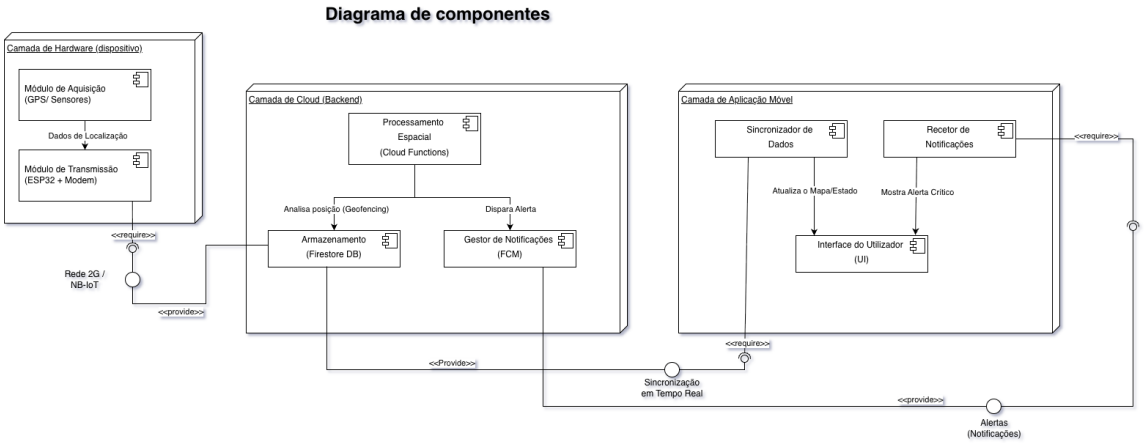


Figura 3.1. – Arquitetura geral do sistema e interação entre as suas camadas.

A comunicação entre a camada de hardware e a camada de cloud é feita exclusivamente através de rede celular, utilizando os protocolos 2G ou NB-IoT disponíveis na região de operação. Por sua vez, a comunicação entre a camada de cloud e a aplicação móvel é realizada através de sincronização em tempo real com o Firestore e do serviço de notificações *push* FCM. Esta arquitetura garante que o dispositivo portátil funciona de forma completamente autónoma, sem depender de Wireless Fidelity (Wi-Fi) ou de Bluetooth, sendo a única condição de funcionamento a existência de cobertura de rede móvel.

3.2.2 Camada de Hardware

A camada de hardware constitui o ponto de entrada do sistema, sendo o componente físico que o utilizador transporta consigo e o responsável por toda a aquisição e transmissão dos dados de localização.

Antes de apresentar o módulo escolhido, importa enquadrar a decisão de arquitetura tomada. No desenvolvimento de sistemas IoT com rastreamento GPS e conectividade móvel, existem essencialmente duas abordagens possíveis. A primeira consiste na utilização de um microcontrolador discreto, como um Arduino ou um ESP32 *standalone*, ao qual são ligados externamente todos os periféricos necessários: um módulo GPS separado, um módulo Global System for Mobile Communications (GSM)/SIM separado, um controlador de carga da bateria, a própria bateria e os respetivos circuitos de proteção e regulação de tensão. Esta abordagem oferece maior flexibilidade de customização, mas implica um esquema elétrico

considerável, múltiplos pontos de falha, maior área de Printed Circuit Board (PCB) e uma complexidade de integração significativa. O Anexo A apresenta uma proposta conceptual de ligações com componentes discretos, destinada a ilustrar a sua interligação e a organização da alimentação.

A segunda abordagem, e a escolhida para este projeto, consiste na utilização de um módulo de desenvolvimento altamente integrado, que reúne todos estes componentes numa única PCB. Esta decisão não foi tomada por ser tecnicamente mais simples, mas por ser a opção de engenharia mais adequada aos objetivos do MVP, uma vez que permite reduzir a área física do dispositivo, minimizar os pontos de falha de hardware e garantir que o tempo de desenvolvimento se concentra na lógica do sistema e não na integração de baixo nível [24, 25].

O módulo selecionado foi o LilyGO T-SIM7000G, uma plataforma de elevada integração concebida especificamente para aplicações IoT remotas, que unifica numa única PCB um microcontrolador de alto desempenho, conectividade móvel multi-banda LPWAN, geolocalização GNSS e um sistema robusto de gestão de energia [24].

O núcleo computacional do sistema é o módulo ESP32-WROVER-B da Espressif Systems, baseado no processador Tensilica Xtensa Dual-Core de 32 bits, capaz de operar até 240 MHz. A arquitetura dual-core é particularmente relevante para este projeto, pois permite dedicar um núcleo exclusivamente à gestão da comunicação com o modem, incluindo comandos AT e pilha TCP/IP, em tempo real, enquanto o segundo núcleo processa a lógica da aplicação e a leitura de sensores. Em termos de memória, o módulo dispõe de 4 MB de Flash para armazenamento do firmware, 520 KB de Static Random-Access Memory (SRAM) interna e 8 MB de Pseudo-Static Random-Access Memory (PSRAM) externa, essenciais para processar *handshakes* TLS 1.2/1.3 durante a comunicação segura com o Firebase e para o *parsing* de *payloads* JSON [24].

As capacidades de conectividade e rastreamento são asseguradas pelo coprocessador SIMCom SIM7000G. Este componente suporta redes LTE Cat-M1, NB-IoT Cat-NB1 e faz *fallback* automático para redes 2G General Packet Radio Service (GPRS)/Enhanced Data rates for GSM Evolution (EDGE), o que garante que o dispositivo opere de forma autónoma em qualquer região com cobertura móvel, sem depender de Wi-Fi ou de infraestrutura local. O módulo GNSS integrado é compatível com as constelações GPS, GLONASS e BeiDou, permitindo obter coordenadas precisas de latitude, longitude e altitude. A interface entre o

ESP32 e o SIM7000G é realizada através do barramento UART, utilizando os pinos GPIO 27 (TX) e GPIO 26 (RX), com controlo do estado de energia do modem através do GPIO 4 [24, 26].

A autonomia energética é um dos requisitos críticos do sistema. A placa integra suporte nativo para bateria Li-ion 18650, gerida pelo IC de proteção DW01A, e um conector para painel solar de 4,4V a 6,8V, gerido pelo IC CN3065, configurado para uma corrente máxima de carregamento de 500 mA. O desenho da alimentação está dimensionado para suportar os picos de corrente das transmissões móveis, até 2,0 A em GPRS/EDGE e cerca de 0,6 A em LTE, mantendo a tensão VBAT estável entre 3,0 V e 4,3 V [25, 26].

Na fase de especificação estimou-se uma autonomia mínima de 24 horas, assumindo um ciclo de transmissão de 60 segundos e a utilização dos modos de baixo consumo disponíveis no módulo [26, 27]. Durante a implementação, a decisão de manter o modem e o recetor Global Navigation Satellite System (GNSS) ligados entre ciclos para reduzir a latência alterou esse pressuposto; a célula efetivamente utilizada na validação tem 3000 mAh e o RNF01 foi formalmente revisto para 6 horas.

A placa integra ainda uma ranhura MicroSD com comunicação SPI, recorrendo aos GPIO 15, 14, 13 e 2, que funcionará como sistema de registo local redundante em cenários de falha de conectividade móvel, permitindo armazenar os dados localmente e enviá-los em bloco quando a ligação for restabelecida. Dispõe também de dois conectores u.FL dedicados para antena LTE de banda completa e antena GPS ativa, bem como de uma ranhura para cartão Nano SIM [24, 25].

Nota. O intervalo de 60 segundos aqui especificado corresponde ao valor definido na fase de especificação. Durante a implementação, a mitigação do conflito de coexistência entre o GNSS e o rádio celular permitiu configurar 20 segundos de repouso (`INTERVAL_S`), sem garantir um ciclo completo com essa duração: observaram-se intervalos de 83 segundos em campo; essa alteração e as suas consequências em autonomia e consumo de dados são analisadas nas Secções 4.2.7 e 6.3.

3.2.3 Camada de Cloud / Backend

A camada de cloud / backend funciona como núcleo lógico do sistema, sendo responsável por receber os dados enviados pelo dispositivo portátil, persistir a informação relevante, aplicar a lógica de geofencing e distribuir eventos para a aplicação móvel. Esta camada foi concebida segundo um paradigma serverless, reduzindo a necessidade de manutenção de servidores dedicados e permitindo concentrar o desenvolvimento na lógica funcional do sistema [28].

Do ponto de vista arquitetural, prevê-se a utilização do ecossistema Firebase como base da infraestrutura cloud, em particular o Firestore para armazenamento estruturado dos dados, o Firebase Authentication para gestão de utilizadores e o Firebase Cloud Messaging para envio de notificações *push*. Esta escolha oferece integração nativa com Flutter, sincronização quase em tempo real e mecanismos de segurança configuráveis por regras declarativas, características particularmente adequadas a um MVP com requisitos de implementação rápida e elevada disponibilidade [28].

Ao nível do fluxo operacional, o dispositivo recolhe periodicamente coordenadas GPS, estado de bateria e metadados de telemetria, transmitindo estes elementos através da rede celular para a cloud. Uma vez recebidos, os dados são armazenados por dispositivo e por instante temporal, permitindo manter o estado atual do sistema e, em simultâneo, conservar um histórico mínimo para consulta posterior. Esta organização facilita tanto a visualização em tempo real na aplicação móvel como a análise de eventos de evasão ou perda de conectividade [28].

Uma das funções centrais desta camada é o processamento espacial associado ao geofencing. Sempre que uma nova localização é recebida, o backend deverá comparar a posição atual com o perímetro virtual definido para o utilizador monitorizado. Para essa verificação, a lógica do sistema recorre ao cálculo da distância geodésica entre coordenadas, podendo apoiar-se na fórmula de Haversine para determinar se o ponto recebido permanece dentro do raio de segurança configurado. Quando é detetada uma violação do perímetro, o backend gera um evento de alerta e desencadeia o envio imediato de uma notificação *push* para o cuidador [8, 28].

A camada backend assume igualmente responsabilidades de segurança e controlo de acesso. A autenticação dos cuidadores deverá ser realizada por credenciais seguras, sendo o acesso aos dados limitado às entidades autorizadas por regras de segurança do Firestore. Desta forma, garante-se que a informação sensível de localização apenas pode ser consultada ou modificada por utilizadores autenticados com permissões explícitas, mitigando riscos de exposição indevida dos dados pessoais do utilizador monitorizado [28].

Em complemento, esta camada desempenha um papel relevante na robustez global do sistema. Para além de centralizar o estado mais recente do dispositivo, deverá registar eventos importantes, como entradas e saídas de perímetro, níveis críticos de bateria e falhas de comunicação. Este registo permite suportar auditoria funcional, diagnóstico de falhas e eventual expansão futura para análise estatística de padrões de mobilidade. Assim, a camada de cloud / backend não se limita ao armazenamento de dados, constituindo antes o elemento de coordenação entre hardware, lógica espacial, segurança e interação com a aplicação móvel [28].

3.2.4 Camada de Aplicação Móvel

A camada de aplicação móvel constitui a interface principal entre o sistema e o cuidador, permitindo visualizar em tempo real a localização do utilizador monitorizado, consultar o estado do dispositivo e interagir com as funcionalidades associadas ao geofencing. Esta aplicação será responsável por apresentar a informação recolhida pela camada de cloud/backend de forma clara e acessível, assegurando que o utilizador consiga interpretar rapidamente o estado do sistema e reagir quando necessário [28].

Para o desenvolvimento desta camada, prevê-se a utilização da framework Flutter, escolhida pela sua capacidade de gerar aplicações *cross-platform* a partir de uma única base de código, o que reduz a complexidade de desenvolvimento e manutenção. A integração com o ecossistema Firebase permitirá reutilizar os mesmos serviços de autenticação, sincronização em tempo real e notificações *push*, mantendo coerência entre a aplicação móvel e a camada backend [28].

Ao nível funcional, a aplicação deverá disponibilizar um mapa interativo com a posição atual do dispositivo, informação sobre o perímetro de segurança definido e alertas visuais sempre que exista uma situação de risco. Em complemento, deverá permitir ao

cuidador consultar histórico recente de localizações, verificar o estado da bateria e receber notificações *push* em caso de ultrapassagem dos limites geográficos configurados [28, 29].

A interface deverá ser simples, intuitiva e orientada para utilização rápida, dado que o contexto de uso pode envolver cuidadores que necessitam de identificar informação crítica sem perda de tempo. Por esse motivo, a aplicação deverá privilegiar uma navegação direta, uma leitura clara dos dados principais e uma hierarquia visual centrada na localização, nos alertas e no estado do dispositivo [29].

Do ponto de vista técnico, a aplicação móvel poderá recorrer ao Google Maps ou a uma solução cartográfica equivalente para representação geográfica, sendo também necessário tratar permissões de localização, notificações e autenticação do utilizador. Esta camada deverá ainda refletir, em tempo útil, quaisquer alterações recebidas da cloud, de forma a manter a informação sincronizada com o estado atual do dispositivo de rastreamento [28].

Em termos de limitações, esta camada depende da disponibilidade de ligação à internet e da correta integração com os serviços cloud, pelo que a experiência do utilizador poderá ser afetada em cenários de fraca conectividade ou falhas externas ao sistema. Ainda assim, a adoção de Flutter e Firebase oferece uma base técnica robusta para implementar uma aplicação móvel leve, responsiva e alinhada com os objetivos do projeto [28].

3.3 Requisitos funcionais

Os requisitos funcionais definem o conjunto de capacidades que o sistema deverá disponibilizar para cumprir os objetivos propostos no projeto. No caso deste sistema IoT de rastreamento e geofencing, estes requisitos traduzem-se nas funcionalidades essenciais que permitem a monitorização remota do utilizador, a gestão dos perímetros de segurança e a geração de alertas em tempo útil.

Tabela 3.1. – Requisitos funcionais do sistema.

Código	Requisito funcional	Descrição
RF01	Aquisição de localização	O sistema deverá adquirir periodicamente a localização geográfica do dispositivo portátil.
RF02	Transmissão de telemetria	O sistema deverá transmitir os dados recolhidos pelo dispositivo para a camada cloud/backend.

Código	Requisito funcional	Descrição
RF03	Autenticação do utilizador	O sistema deverá permitir o acesso seguro do cuidador à aplicação móvel através de autenticação.
RF04	Visualização em mapa	O sistema deverá apresentar a posição atual do dispositivo num mapa interativo na aplicação móvel.
RF05	Configuração de geofencing	O sistema deverá permitir definir, alterar e remover perímetros de segurança virtuais.
RF06	Verificação de perímetro	O backend deverá comparar a posição atual do dispositivo com o perímetro de segurança definido.
RF07	Geração de alertas	O sistema deverá gerar alertas sempre que for detetada a ultrapassagem do perímetro configurado.
RF08	Notificações push	O sistema deverá enviar notificações push para a aplicação móvel quando ocorrer uma situação de alerta.
RF09	Consulta do estado da bateria	O sistema deverá permitir consultar o nível de bateria do dispositivo portátil na aplicação móvel.
RF10	Registo de eventos	O sistema deverá registar eventos relevantes, como leituras de localização e alertas gerados.
RF11	Emparelhamento do dispositivo	O sistema deverá permitir associar um dispositivo à conta do cuidador através de um código de emparelhamento válido.
RF12	Histórico de localizações	O sistema deverá permitir consultar um histórico recente das localizações registadas pelo dispositivo.
RF13	Posicionamento alternativo	O sistema deverá identificar e apresentar uma posição estimada por Wi-Fi quando não for possível obter uma leitura GNSS válida, assinalando a origem e a precisão estimada.

A definição destes requisitos permite estabelecer uma base clara para a modelação funcional do sistema, servindo de suporte direto à elaboração dos casos de uso, do modelo de dados e dos diagramas de sequência e atividade. Desta forma, garante-se a coerência entre os objetivos do projeto, a arquitetura proposta e a validação futura do protótipo.

3.4 Requisitos não funcionais

Os requisitos não funcionais especificam as qualidades e restrições que o sistema deverá satisfazer, abrangendo critérios de desempenho, usabilidade, segurança, fiabilidade e compatibilidade. Estes requisitos complementam os funcionais e são fundamentais para garantir a adequação prática da solução ao contexto de utilização proposto.

Tabela 3.2. – Requisitos não funcionais do sistema.

Código	Requisito não funcional	Descrição
RNF01	Autonomia energética	O dispositivo portátil deverá funcionar continuamente durante pelo menos 6 horas com uma bateria de 3000 mAh e intervalo de repouso configurado de 20 segundos.
RNF02	Precisão de localização	A precisão da localização GPS deverá ser inferior a 10 metros em condições normais de operação.
RNF03	Latência de alertas	O tempo entre a deteção da violação de perímetro e o envio da notificação push não deverá exceder 30 segundos.
RNF04	Disponibilidade da rede	O sistema deverá funcionar com redes 2G/GPRS, NB-IoT ou LTE-M, com fallback automático entre tecnologias.
RNF05	Usabilidade da app	A aplicação móvel deverá ser instalável em dispositivos Android 8+ e iOS 15+, com tempo de aprendizagem inferior a 5 minutos.
RNF06	Segurança dos dados	Os dados de localização deverão ser transmitidos com cifragem TLS 1.2+ e armazenados com regras de acesso restritas.
RNF07	Consumo de dados	O dispositivo não deverá consumir mais de 100 MB por mês em condições normais de utilização.
RNF08	Interface responsiva	A aplicação móvel deverá manter uma taxa de fotogramas superior a 30 fps durante a visualização do mapa.
RNF09	Robustez do dispositivo	O dispositivo deverá suportar temperaturas entre -10°C e +50°C e vibrações normais de utilização quotidiana.
RNF10	Escalabilidade	O sistema deverá suportar pelo menos 10 dispositivos simultâneos na mesma conta de Firebase sem degradação de desempenho.

Nota. O RNF01 foi revisto de 24 para 6 horas na sequência da decisão de manter o modem celular e o recetor GNSS ligados entre ciclos, privilegiando a latência de reporte. A fundamentação técnica desta revisão é apresentada na Secção 6.4.

Estes requisitos não funcionais estabelecem as métricas objetivas de qualidade que o sistema deverá cumprir, permitindo avaliar a viabilidade prática da solução desenvolvida e a sua adequação aos objetivos definidos para o produto mínimo viável.

3.5 Restrições e Limites do Sistema

O presente projeto foi desenvolvido no âmbito de um MVP, pelo que existem restrições técnicas e operacionais que delimitam o seu âmbito de funcionamento. A Tabela 3.3 sintetiza as principais restrições identificadas, organizadas por categoria.

Tabela 3.3. – Restrições e limites do sistema IoT de rastreamento e geofencing.

Código	Categoria	Restrição	Impacto
R-01	Conectividade	O sistema requer cobertura de rede móvel 2G, NB-IoT ou LTE-M para funcionar. Em zonas sem cobertura, os dados são armazenados localmente no cartão MicroSD e enviados quando a ligação for restabelecida.	Médio
R-02	GPS	Em ambientes fechados (*indoor*), o sinal de satélite pode ser insuficiente para obter *fix*. Nestes casos, o sistema reporta a última posição válida registada.	Médio
R-03	Energia	A autonomia do dispositivo está limitada à capacidade da bateria 18650 (2600 mAh). Com um ciclo de transmissão de 60 segundos, estima-se uma autonomia mínima de 24 horas.	Alto
R-04	Latência	O ciclo de atualização de posição é de aproximadamente 60 segundos, não sendo adequado para cenários que exijam rastreamento em tempo real contínuo.	Médio
R-05	Cloud	O sistema depende da disponibilidade dos serviços Google Firebase. Falhas na infraestrutura da Google afetam diretamente o funcionamento do backend.	Alto
R-06	Cloud Functions	Os arranques a frio (*cold starts*) das Cloud Functions podem introduzir latência adicional no processamento de alertas em períodos de inatividade.	Baixo
R-07	Âmbito MVP	A versão atual suporta apenas um dispositivo por conta de cuidador. O suporte multi-dispositivo é identificado como trabalho futuro.	Baixo

Código	Categoria	Restrição	Impacto
R-08	Geofencing	O sistema suporta múltiplas zonas seguras por dispositivo, circulares ou poligonais, sendo o portador considerado seguro enquanto se encontrar dentro de, pelo menos, uma zona ativa. Não são suportadas regras temporais (zonas ativas por horário) nem deteção de anomalias por aprendizagem de rotina.	Baixo
R-09	Histórico	O histórico de posições é apresentado sob a forma de trilho no mapa e de lista cronológica, com filtros temporais. Não inclui reprodução animada do percurso nem exportação para formatos externos.	Baixo
R-10	Dados	O consumo de dados móveis, embora reduzido para pacotes JSON de pequena dimensão, deve ser monitorizado durante os testes de campo para validar a estimativa de 100 MB/mês.	Baixo

Nota. Os valores apresentados em R-03, R-04 e R-10 correspondem à fase de especificação, incluindo a capacidade inicialmente considerada de 2600 mAh. A célula efetivamente utilizada na validação tem 3000 mAh; os restantes parâmetros implementados e os resultados medidos são apresentados na Secção 4.6 e no Capítulo 5.

A classificação de impacto segue a seguinte escala: Alto — afeta diretamente a funcionalidade core do sistema; Médio — afeta a experiência de utilização em cenários específicos; Baixo — afeta funcionalidades secundárias ou tem mitigação disponível.

3.6 Atores e Níveis de Segurança

O sistema envolve dois tipos de atores — humanos e de sistema — com responsabilidades e privilégios de acesso distintos. A Tabela 3.4 descreve os atores humanos, a Tabela 3.5 os atores de sistema e a Tabela 3.6 sintetiza os níveis de segurança e os respetivos privilégios de acesso.

Tabela 3.4. – Atores humanos.

Ator	Tipo	Descrição	Interage com
Cuidador	Humano ativo	Responsável pelo utilizador monitorizado. Utiliza a aplicação móvel para visualizar localização, configurar geofencing e receber alertas.	App Flutter
Utilizador / Portador	Humano passivo	Pessoa monitorizada. Transporta o dispositivo físico. Não interage com qualquer interface de software.	Dispositivo IoT

Tabela 3.5. – Atores de sistema.

Ator	Tipo	Descrição	Protocolo
Dispositivo IoT	Sistema embebido	Adquire coordenadas GPS, lê bateria e transmite telemetria JSON para o Firebase via rede celular.	HTTP POST · 2G/NB-IoT
Firebase Auth	Sistema externo	Valida as credenciais do Cuidador e emite tokens JSON Web Token (JWT) para controlo de acesso.	HTTPS · Open Authorization (OAuth) 2.0
Cloud Function	Sistema interno	Acionada por *trigger* Firestore. Executa a lógica de geofencing (Haversine) e determina o disparo de alertas.	Trigger interno Firebase
FCM	Sistema externo	Entrega notificações *push* ao smartphone do Cuidador mediante instrução da Cloud Function.	HTTP v1 API

Tabela 3.6. – Níveis de segurança e privilégios de acesso.

Nível	Designação	Ator	Privilégios
0	Sem autenticação	Qualquer utilizador	Acesso apenas aos ecrãs de Login e Registo. Sem acesso a dados.
1	Cuidador autenticado	Cuidador	Acesso total aos dados do seu dispositivo — localização, geofence, alertas e bateria. Sem acesso a dados de outros cuidadores.
2	Serviço interno	Cloud Function / Firebase Auth	Acesso total à base de dados via *service account*. Operações de leitura/escrita em todas as coleções. Nunca exposto ao utilizador final.

3.7 Modelação Funcional

Esta secção reúne os diagramas e descrições funcionais do sistema.

3.7.1 Diagrama de Componentes UML

O diagrama de componentes UML apresenta a organização funcional do sistema em três camadas principais: hardware, cloud/backend e aplicação móvel. A camada de hardware é responsável pela aquisição da localização e pela transmissão dos dados através da rede móvel; a camada de cloud/backend centraliza o armazenamento, o processamento espacial e a gestão de notificações; e a camada de aplicação móvel assegura a sincronização dos dados, a receção de alertas e a interação com o cuidador.

Como ilustrado na Figura 3.2, a comunicação entre as camadas ocorre através de interfaces bem definidas. O dispositivo IoT envia telemetria para o backend, o Firestore fornece sincronização em tempo real para a aplicação móvel e o serviço FCM disponibiliza notificações de alerta. Esta separação permite reduzir o acoplamento entre componentes, facilitar a manutenção do sistema e tornar mais clara a responsabilidade de cada módulo no funcionamento global da solução.

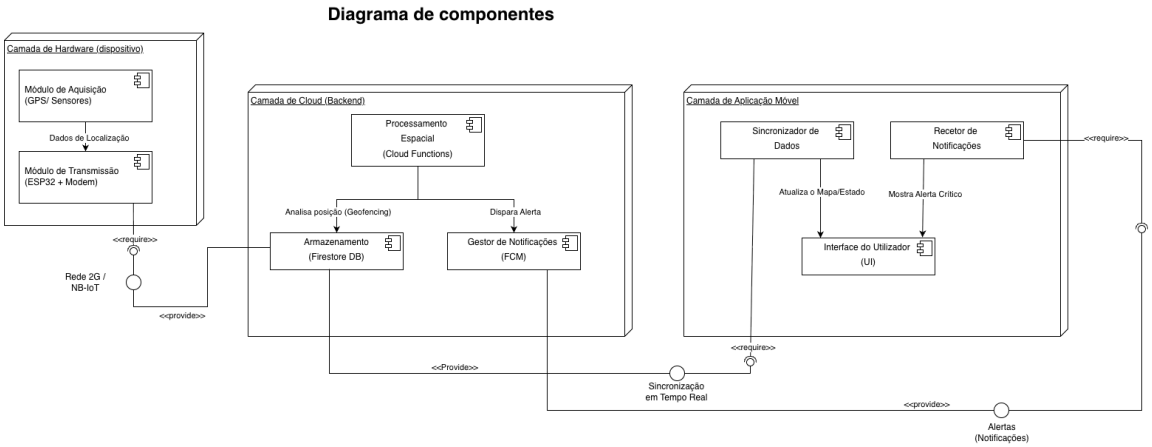


Figura 3.2. – Diagrama de componentes UML do sistema IoT de rastreamento e geofencing.

3.7.2 Diagrama de Classes UML

O diagrama de classes UML representa a estrutura lógica das principais entidades do sistema e as relações existentes entre os módulos de autenticação, monitorização, geofencing, telemetria e notificações. A modelação distingue classes de domínio, como Cuidador, Dispositivo, TelemetriaGPS, GeofenceConfig e Alerta, de classes de controlo e serviço, como AuthController, MapaController, GeofenceService, TelemetriaService e NotificacaoService.

Como ilustrado na Figura 3.3, as entidades centrais herdam comportamentos comuns de persistência e validação através de classes ou interfaces abstratas, enquanto os serviços encapsulam a lógica operacional do sistema. Esta organização facilita a separação de responsabilidades, permite validar os dados antes de serem persistidos e torna explícitas as dependências entre a aplicação móvel, o backend e os mecanismos de geração de alertas.

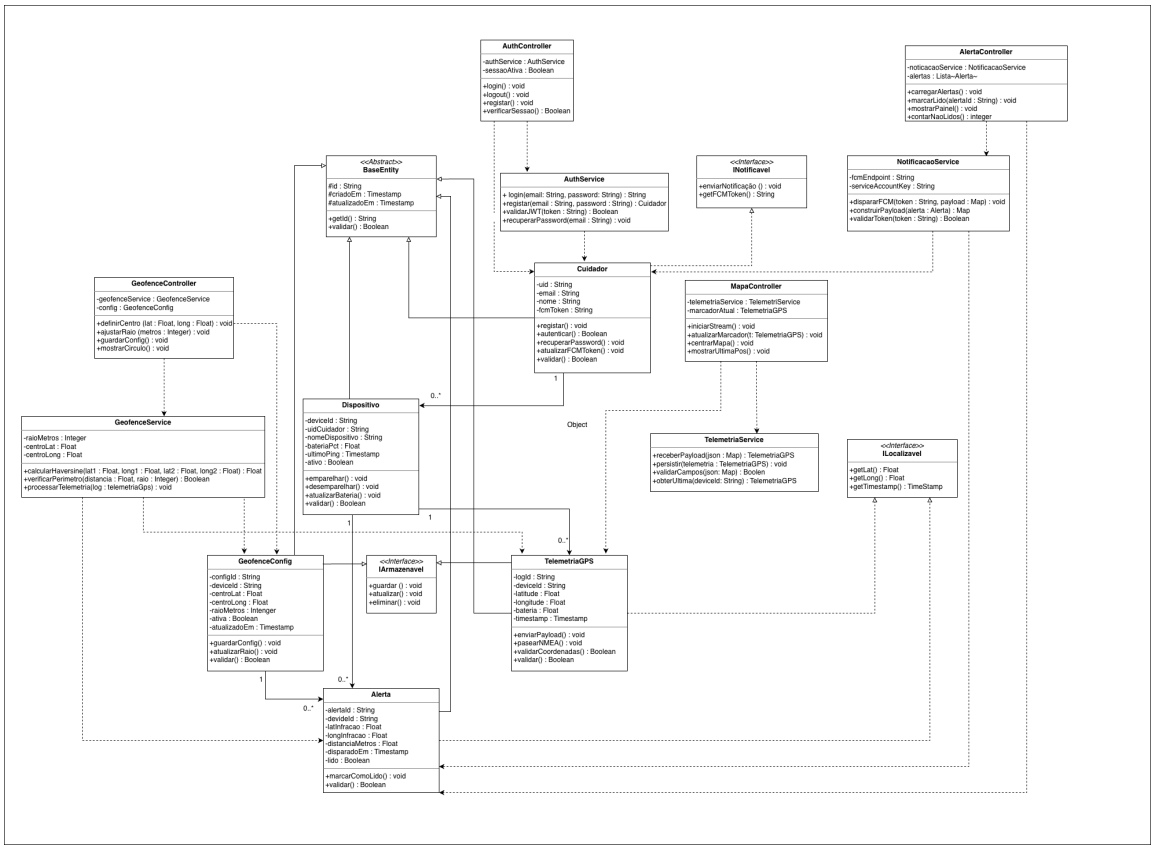


Figura 3.3. – Diagrama de classes UML do sistema IoT de rastreamento e geofencing.

3.7.3 UC-01 — Registar e efetuar Login

O caso de uso UC-01 descreve o processo de criação de conta, autenticação e recuperação de palavra-passe do cuidador na aplicação móvel. Este fluxo envolve a interação direta do cuidador com a aplicação Flutter e a validação das credenciais através do Firebase Auth, permitindo controlar o acesso aos dados do dispositivo monitorizado.

Como apresentado na Figura 3.4, o utilizador pode criar uma conta com email e palavra-passe, efetuar login, recuperar a palavra-passe por email e, em caso de autenticação válida, guardar a sessão através de token JWT. Quando as credenciais são inválidas, o sistema devolve uma mensagem de erro, impedindo o acesso às funcionalidades protegidas da aplicação.

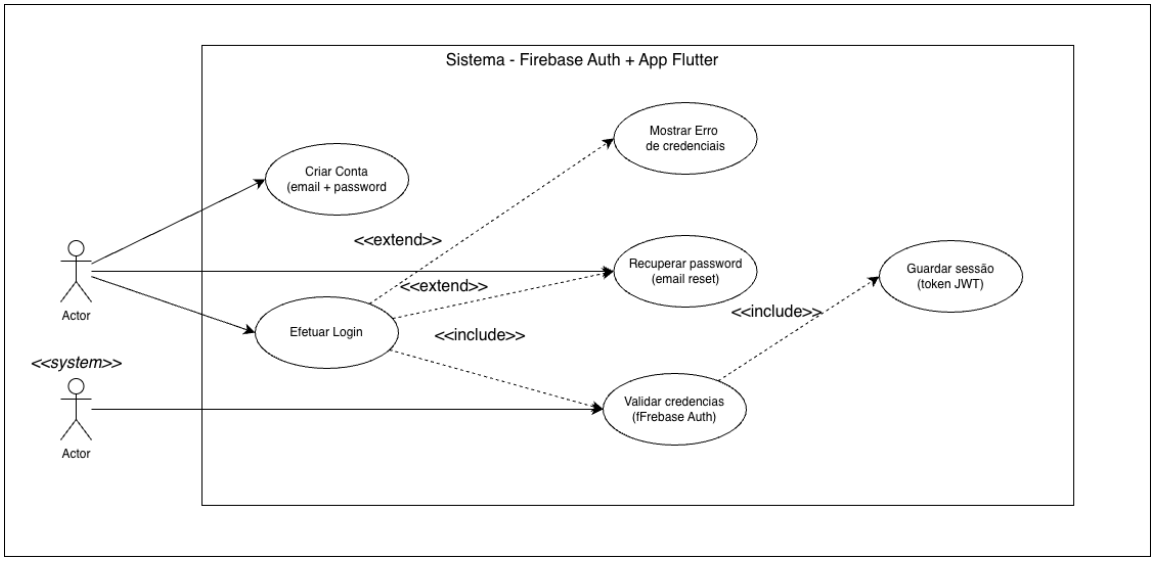


Figura 3.4. – Diagrama de caso de uso UC-01 — Registar e efetuar Login.

3.7.4 UC-02 — Emparelhar Dispositivo

O caso de uso UC-02 descreve o processo de associação de um dispositivo IoT à conta do cuidador. Neste fluxo, o cuidador introduz o identificador único do dispositivo na aplicação móvel, sendo esse identificador validado na base de dados Firestore antes de a associação ser concluída.

Como representado na Figura 3.5, a aplicação envia o identificador para validação no Firestore. Caso o ID seja reconhecido, o dispositivo pode ser associado à conta do cuidador; caso contrário, o sistema apresenta uma mensagem de erro indicando que o identificador não foi reconhecido. Este mecanismo impede a associação de dispositivos inexistentes ou não registados no backend.

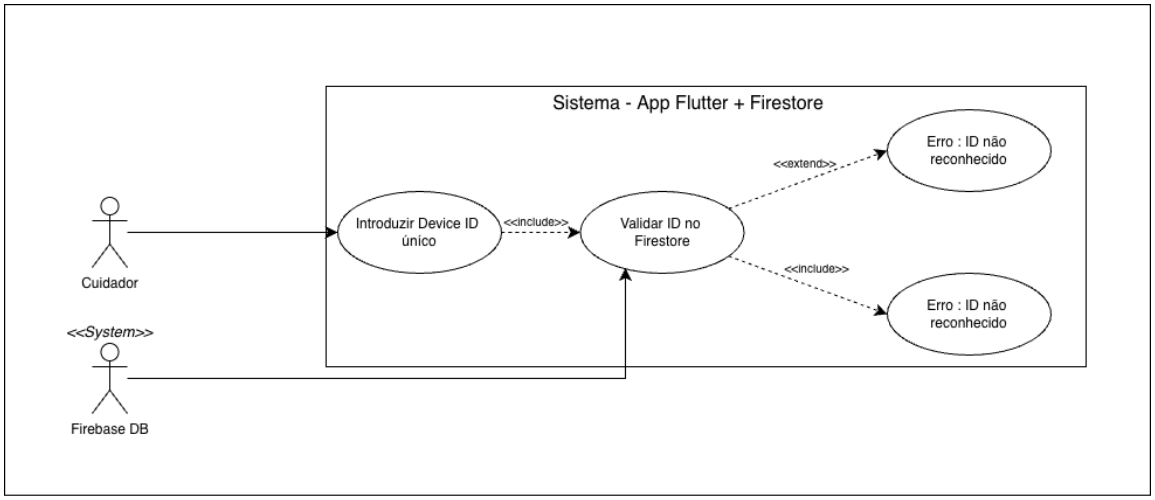


Figura 3.5. – Diagrama de caso de uso UC-02 — Emparelhar Dispositivo.

3.7.5 UC-03 — Visualizar Mapa em Tempo Real

O caso de uso UC-03 descreve a visualização da localização do dispositivo no mapa da aplicação móvel. O cuidador acede ao ecrã do mapa, a aplicação subscreve o fluxo de dados do Firestore e o marcador é atualizado sempre que existe uma nova posição válida associada ao dispositivo monitorizado.

Como ilustrado na Figura 3.6, o fluxo depende da integração entre a aplicação Flutter, o Google Maps e o Firestore. O dispositivo gera telemetria de localização, que é disponibilizada na base de dados, e a aplicação utiliza essa informação para atualizar o marcador no mapa. Caso não exista uma posição recente disponível, o sistema pode apresentar a última posição válida registada.

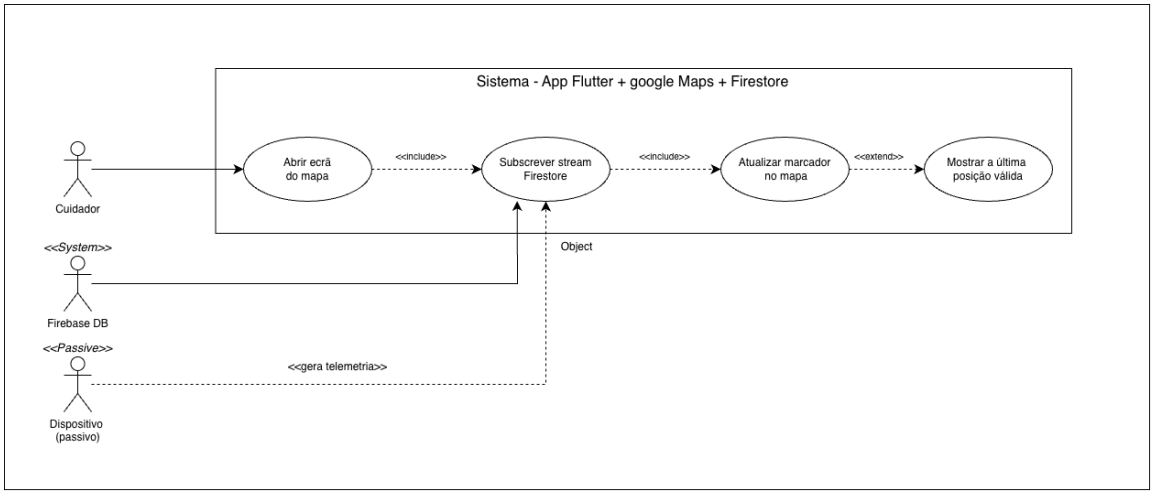


Figura 3.6. – Diagrama de caso de uso UC-03 — Visualizar Mapa em Tempo Real.

3.7.6 UC-04 — Configurar Cerca Virtual

O caso de uso UC-04 descreve a configuração da cerca virtual associada ao dispositivo monitorizado. O cuidador define o centro da geofence através do mapa, ajusta o raio permitido e visualiza graficamente o círculo de segurança antes de guardar a configuração no Firestore.

Como apresentado na Figura 3.7, o fluxo decorre na aplicação Flutter com persistência dos parâmetros no backend. A definição do centro e do raio estabelece o perímetro de segurança utilizado pela lógica de geofencing para detetar situações de saída da área autorizada.

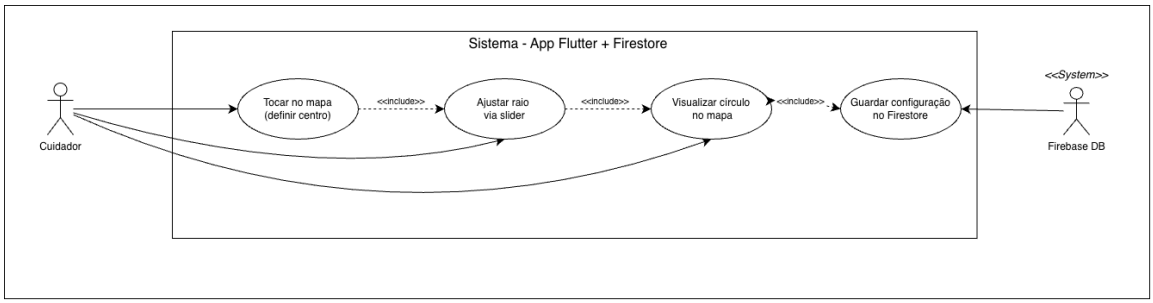


Figura 3.7. – Diagrama de caso de uso UC-04 — Configurar Cerca Virtual.

3.7.7 UC-05 — Receber Alerta de Emergência

O caso de uso UC-05 descreve o processo de geração e receção de alertas quando o sistema deteta uma infração ao perímetro de segurança configurado. A deteção é realizada pela Cloud Function através da lógica de geofencing, seguindo-se o envio de uma notificação para a aplicação móvel do cuidador através do serviço FCM.

Como representado na Figura 3.8, após a deteção da infração, o sistema dispara a notificação, a aplicação recebe e apresenta o alerta ao cuidador e o evento é registado no histórico. Este fluxo permite que situações críticas sejam comunicadas em tempo útil e fiquem documentadas para consulta posterior.

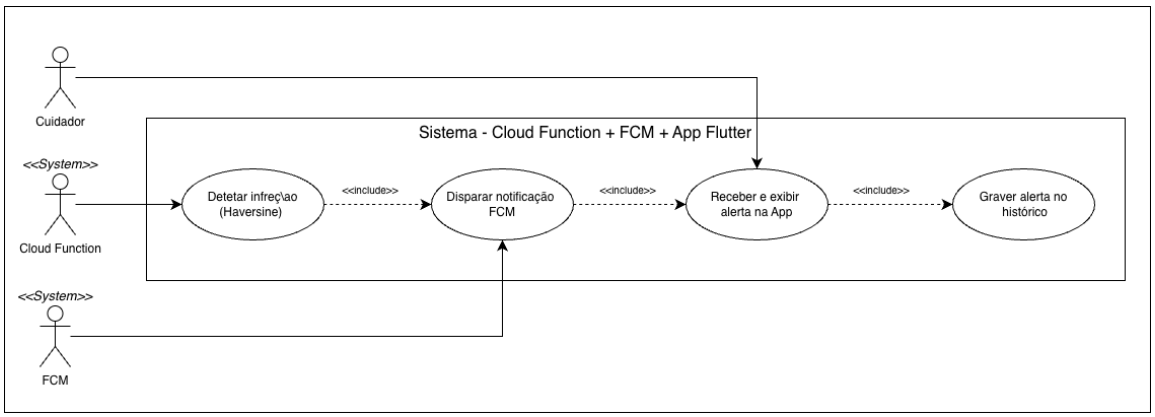


Figura 3.8. – Diagrama de caso de uso UC-05 — Receber Alerta de Emergência.

3.7.8 UC-06 — Consultar Nível de Bateria

O caso de uso UC-06 descreve a consulta do nível de bateria do dispositivo IoT através da aplicação móvel. O cuidador visualiza o estado energético do dispositivo, enquanto a aplicação atualiza o indicador com base nos dados de telemetria armazenados no Firestore.

Como ilustrado na Figura 3.9, o sistema permite apresentar o nível de bateria em tempo real e sinalizar visualmente situações críticas, nomeadamente quando a bateria se encontra abaixo de um limiar definido. Esta funcionalidade é relevante para garantir a continuidade da monitorização e para permitir ao cuidador agir preventivamente antes de o dispositivo ficar sem energia.

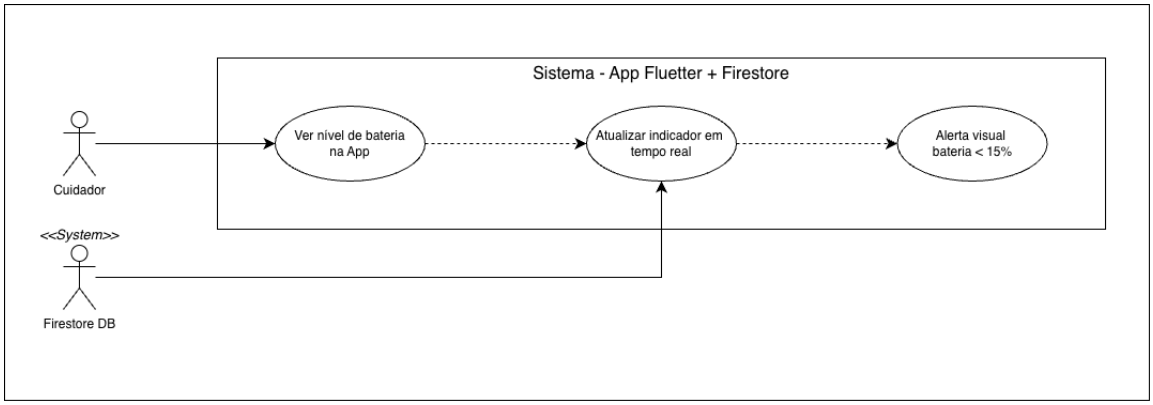


Figura 3.9. – Diagrama de caso de uso UC-06 — Consultar Nível de Bateria.

3.7.9 Diagrama de Sequência — Cenário de Evasão

O diagrama de sequência do cenário de evasão descreve a interação temporal entre o dispositivo IoT, o Firestore, a Cloud Function, o serviço FCM, a aplicação Flutter e o cuidador. O fluxo inicia-se com a aquisição das coordenadas GPS e do nível de bateria, seguindo-se o envio da telemetria para a camada de cloud.

Como apresentado na Figura 3.10, a escrita da telemetria no Firestore aciona a Cloud Function responsável pela lógica de geofencing. Após calcular a distância através da fórmula de Haversine, o sistema regista a posição como segura ou, quando é detetada uma saída do perímetro, cria um alerta e envia uma notificação *push*. A aplicação recebe simultaneamente os dados através da sincronização em tempo real, permitindo ao cuidador consultar o alerta, a posição atual e o trajeto recente.

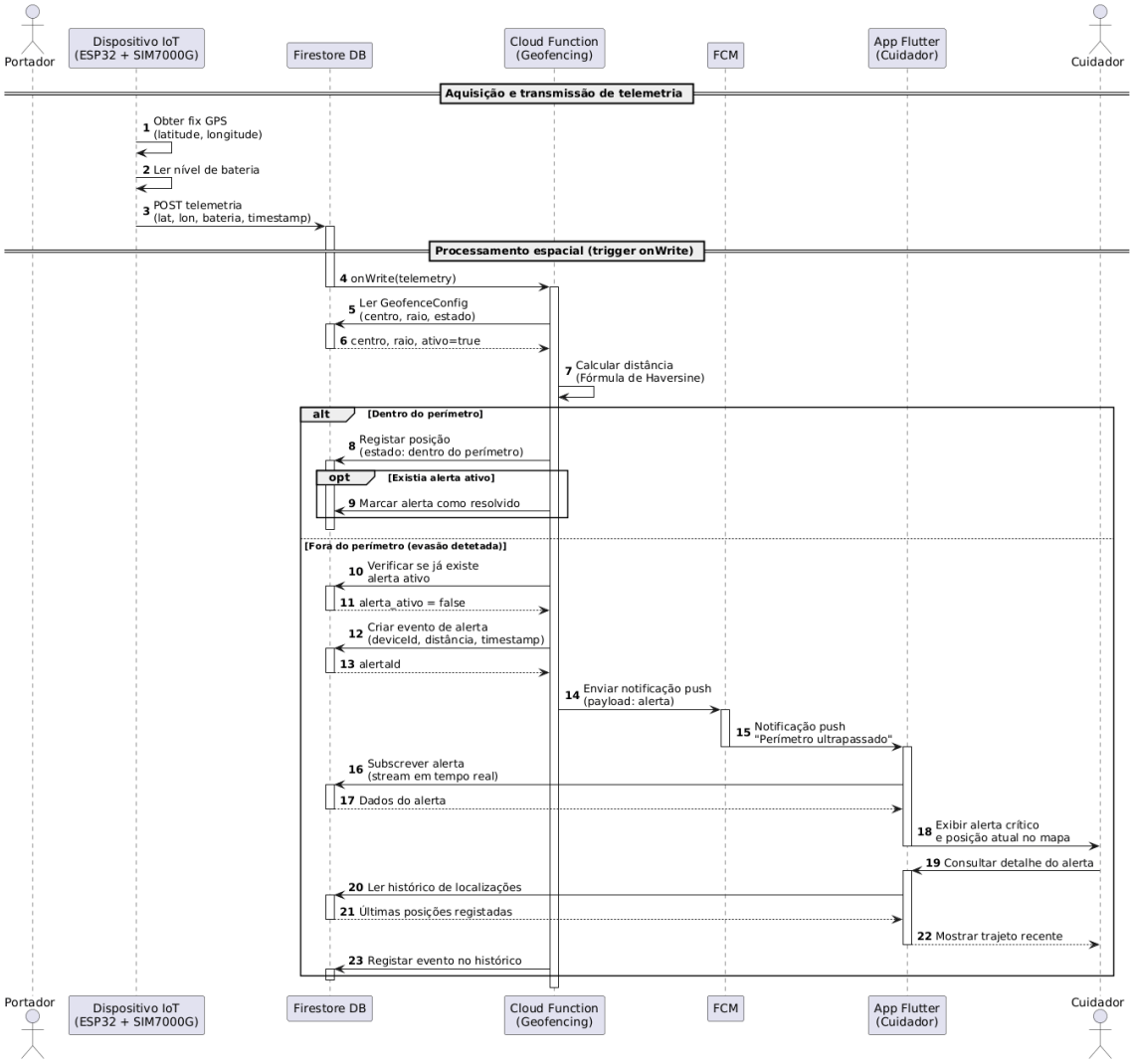


Figura 3.10. – Diagrama de sequência UML do cenário de evasão e geração de alerta.

3.7.10 Diagrama de Atividades — Lógica Haversine

O diagrama de atividades da lógica de Haversine descreve o processamento executado pela Cloud Function sempre que é registada uma nova telemetria no Firestore. O serviço lê a configuração da cerca virtual, verifica se esta se encontra ativa e calcula a distância entre a posição atual do dispositivo e o centro da geofence.

Como apresentado na Figura 3.11, quando o dispositivo permanece dentro do raio definido, a posição é registada como segura e um eventual alerta anterior é marcado como resolvido. Quando a distância ultrapassa o raio permitido, o sistema verifica se já existe um alerta ativo, evitando notificações duplicadas. Caso não exista, cria um evento de alerta no Firestore, envia uma notificação *push* através do FCM e regista o evento no histórico.

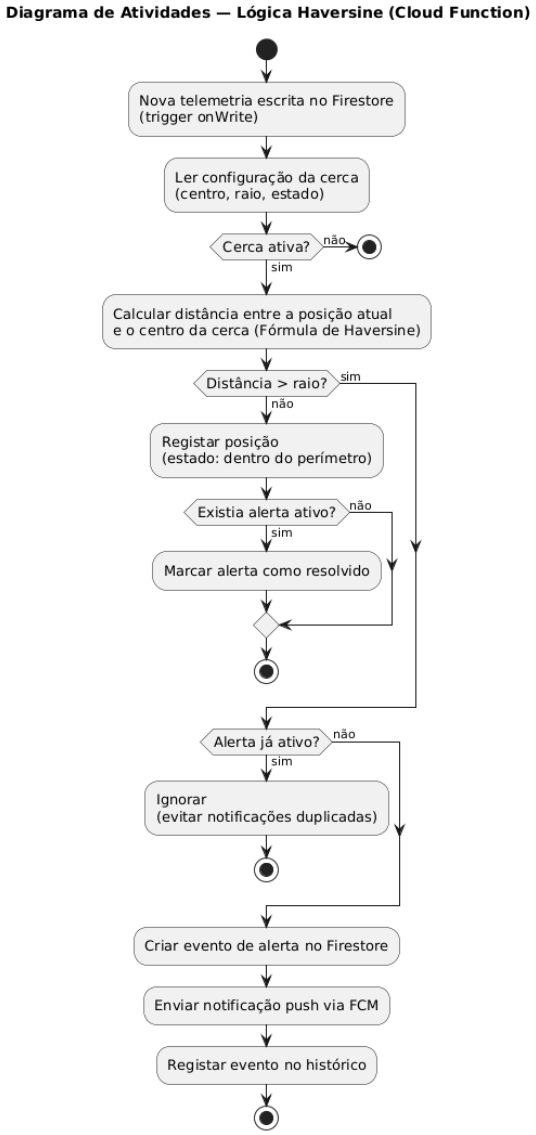


Figura 3.11. – Diagrama de atividades UML da lógica de geofencing baseada na fórmula de Haversine.

3.7.11 Diagrama de Estados do Dispositivo

O diagrama de estados representa os diferentes modos de operação do dispositivo IoT e as condições que determinam a transição entre eles. Após ser ligado, o ESP32 inicializa o modem e o módulo GPS, passando depois para o estado de aquisição de coordenadas através das sentenças National Marine Electronics Association (NMEA).

Como ilustrado na Figura 3.12, quando é obtida uma posição válida e existe conectividade móvel, o dispositivo transmite a telemetria e os dados que se encontrem pendentes. Caso a rede não esteja disponível ou ocorra uma falha no envio, o sistema entra em modo *offline* e armazena o *payload* no cartão MicroSD. Após a conclusão de cada ciclo, o dispositivo desliga os periféricos e entra em *deep sleep*, regressando à aquisição GPS quando é ativado pelo temporizador Real-Time Clock (RTC).

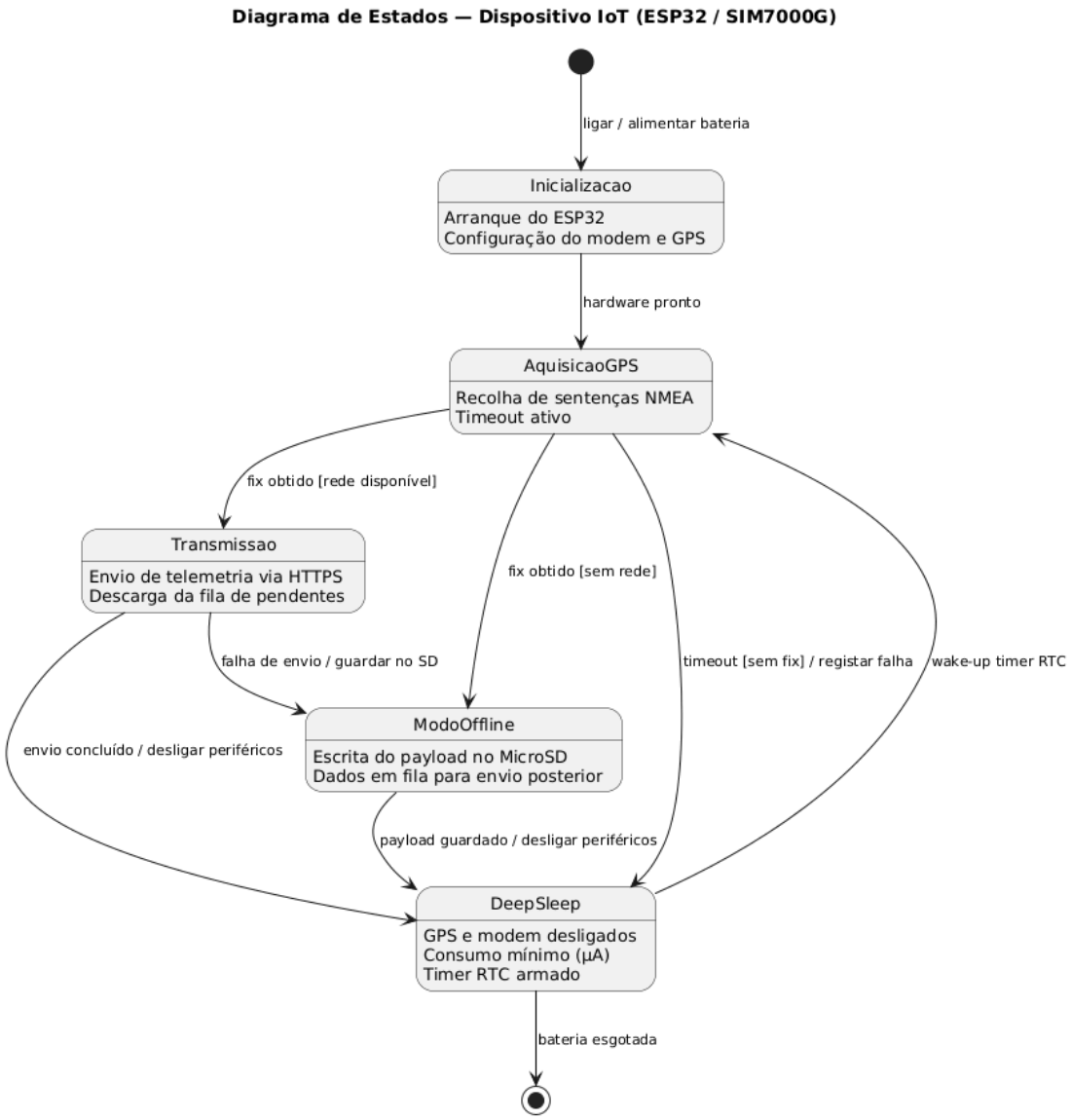


Figura 3.12. – Diagrama de estados UML do dispositivo IoT baseado no ESP32 e no módulo SIM7000G.

3.7.12 Diagrama de Atividades UML — Firmware ESP32

O diagrama de atividades do firmware representa o ciclo de funcionamento do dispositivo IoT, desde o despertar do modo *deep sleep* até ao regresso ao estado de baixo consumo. Após a inicialização do módulo SIM7000G, o sistema tenta obter uma posição GPS válida, lê o nível da bateria e constrói o *payload* de telemetria a transmitir.

Como ilustrado na Figura 3.13, quando existe conectividade celular, o dispositivo envia primeiro os dados pendentes armazenados no cartão MicroSD e, de seguida, transmite a telemetria atual para o Firestore através de HTTPS. Na ausência de rede, os dados são guardados localmente para envio posterior. No final de cada ciclo, o GPS e o modem são desligados e o ESP32 entra novamente em *deep sleep*, reduzindo o consumo energético.

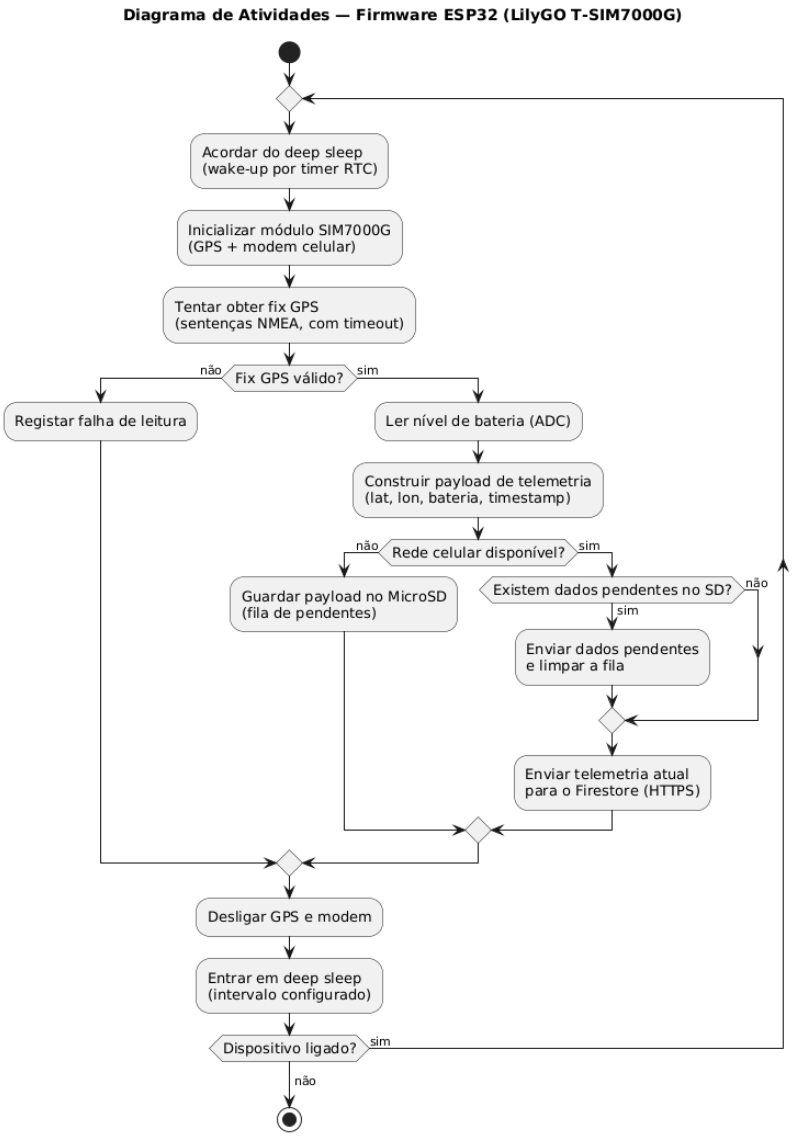


Figura 3.13. – Diagrama de atividades UML do ciclo de funcionamento do firmware ESP32.

3.7.13 Diagrama de Atividades — Registo e Login

O diagrama de atividades de registo e login representa o fluxo de autenticação do cuidador na aplicação móvel. Ao abrir a aplicação, o utilizador pode criar uma nova conta ou iniciar sessão através das credenciais previamente registadas no Firebase Authentication.

Como ilustrado na Figura 3.14, o processo de registo inclui a introdução e validação do endereço de correio eletrónico e da palavra-passe, apresentando uma mensagem de erro quando os dados são inválidos ou a conta já existe. No fluxo de login, as credenciais são validadas pelo Firebase, sendo também disponibilizada a recuperação da palavra-passe. Após uma autenticação válida, a sessão é guardada através de um token JWT e o utilizador é encaminhado para o ecrã principal ou para o emparelhamento inicial do dispositivo.

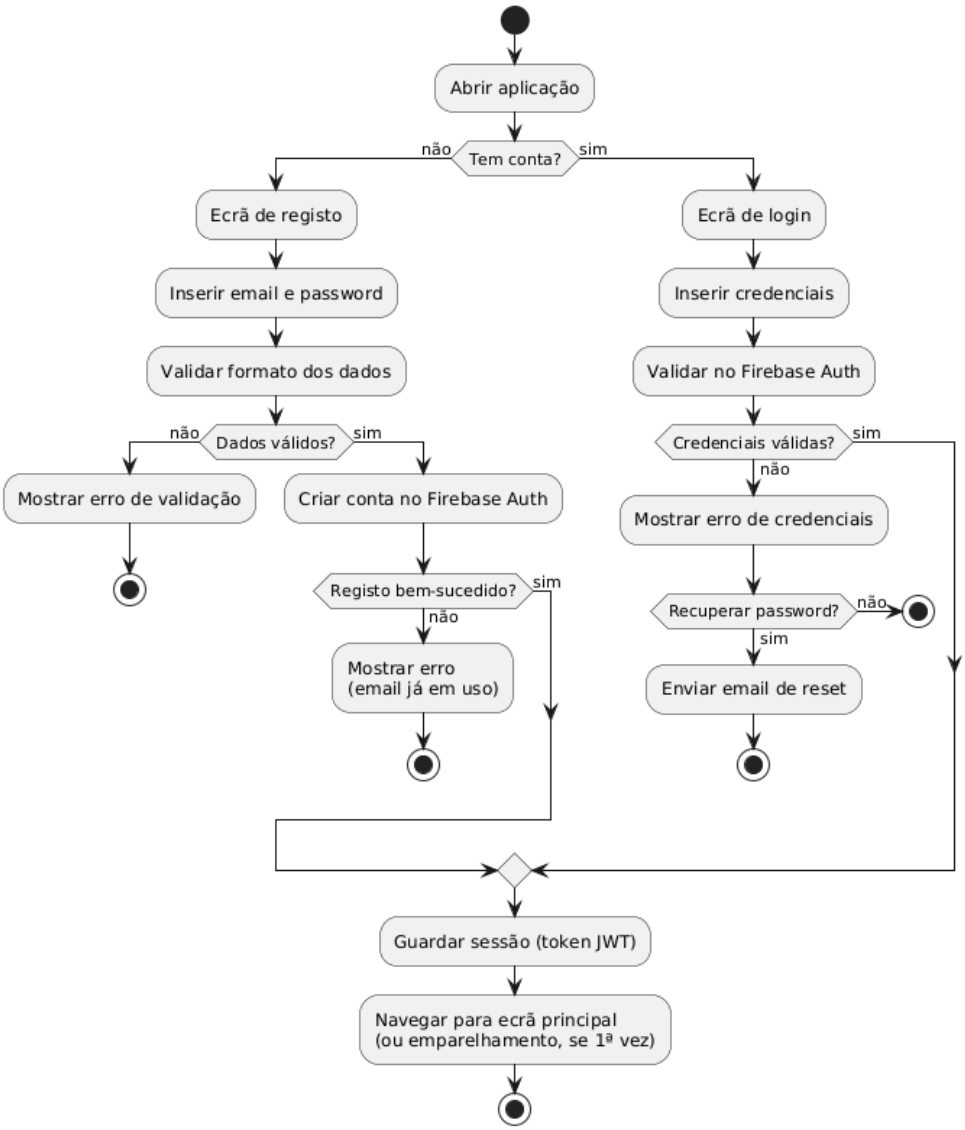


Figura 3.14. – Diagrama de atividades UML dos processos de registo e login.

3.7.14 Diagrama de Atividades — Emparelhamento de Dispositivo

O diagrama de atividades de emparelhamento descreve o processo de associação de um dispositivo IoT à conta autenticada do cuidador. O processo inicia-se no ecrã de emparelhamento, onde é introduzido o identificador único do dispositivo para validação no Firestore.

Como ilustrado na Figura 3.15, caso o identificador não seja reconhecido ou o dispositivo já esteja associado a outra conta, a aplicação apresenta a mensagem de erro correspondente e termina o processo. Quando o identificador é válido e se encontra disponível, o dispositivo é associado à conta do cuidador, a relação é guardada no Firestore e o utilizador é encaminhado para o mapa principal.

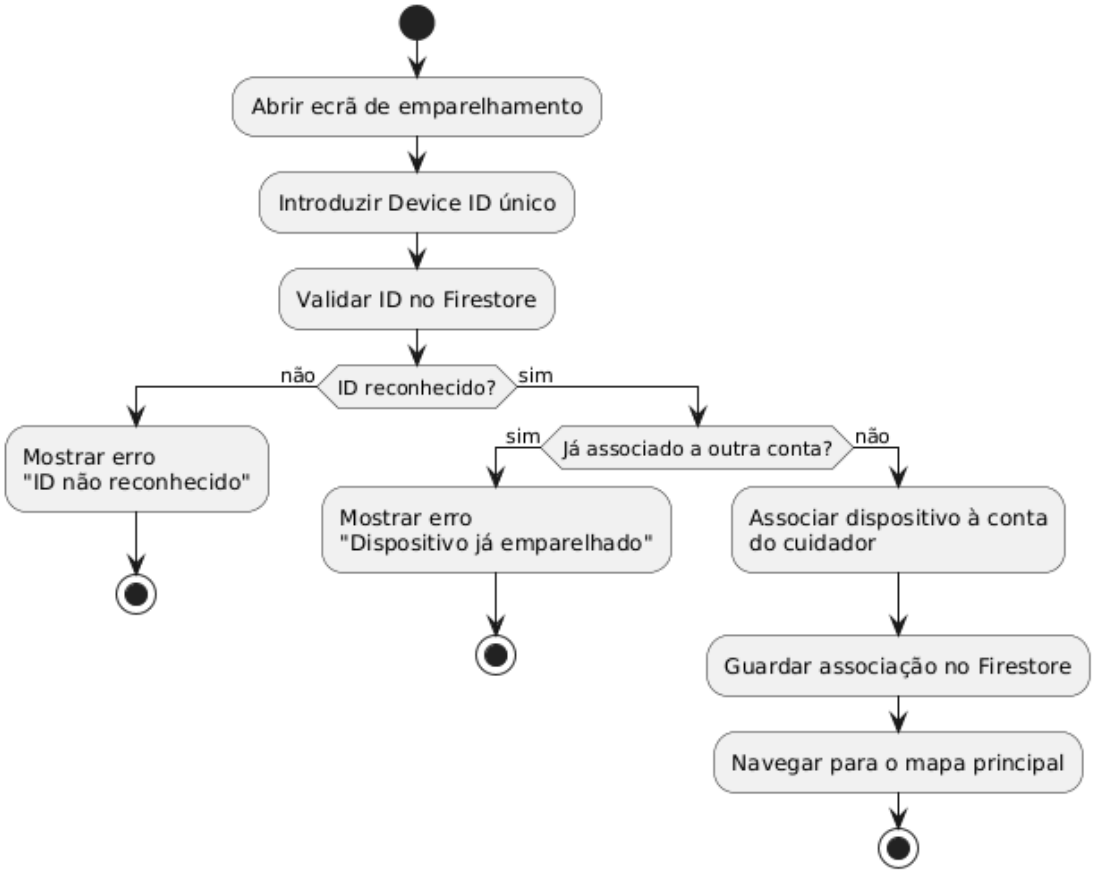


Figura 3.15. – Diagrama de atividades UML do emparelhamento de um dispositivo IoT.

3.7.15 Diagrama de Atividades — Visualização do Mapa

O diagrama de atividades de visualização do mapa descreve o fluxo executado pela aplicação para apresentar a posição do dispositivo em tempo real. Ao abrir o ecrã do mapa, a aplicação subscreve o fluxo de telemetria associado ao dispositivo no Firestore e permanece a aguardar novas coordenadas.

Como ilustrado na Figura 3.16, sempre que é recebida uma nova posição, o marcador é atualizado no mapa. Caso não sejam recebidos dados durante mais de cinco minutos, a aplicação apresenta um aviso de ausência de sinal e mantém visível a última posição válida. O ciclo continua enquanto o ecrã estiver ativo, sendo a subscrição cancelada quando o utilizador abandona o mapa.

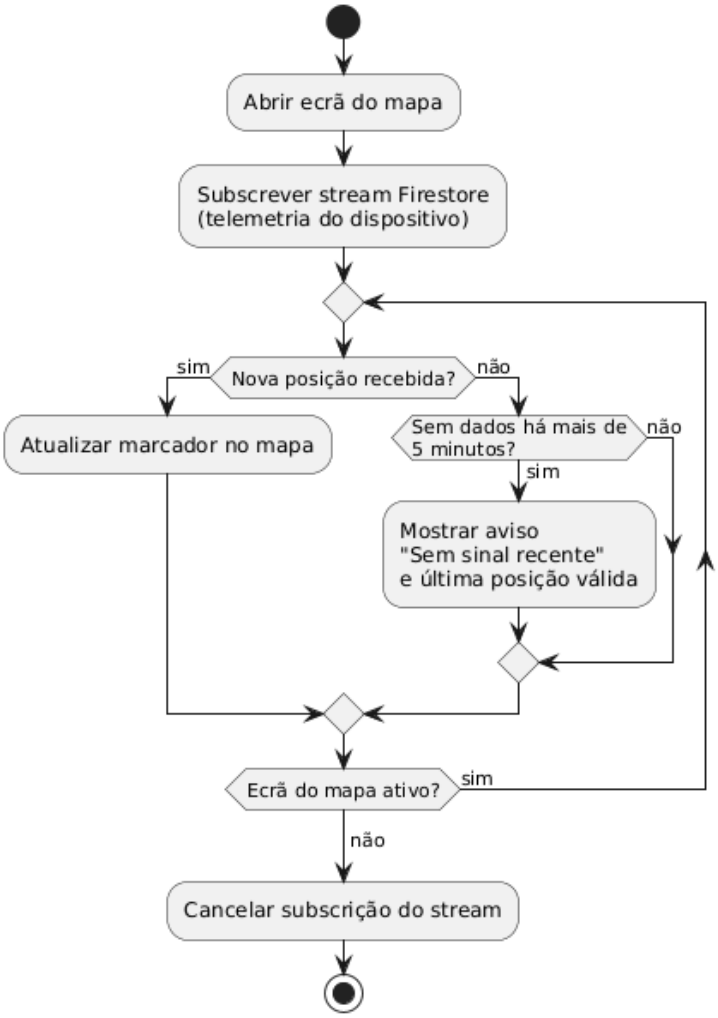


Figura 3.16. – Diagrama de atividades UML da visualização da posição do dispositivo no mapa.

3.7.16 Diagrama de Atividades — Configuração de Geofence

O diagrama de atividades de configuração da geofence descreve o processo utilizado pelo cuidador para definir um perímetro de segurança circular. No ecrã de configuração, o utilizador seleciona o centro diretamente no mapa, ajusta o raio através de um controlo deslizante e visualiza antecipadamente o círculo correspondente.

Como ilustrado na Figura 3.17, após a confirmação são validados os limites permitidos para o raio e os restantes parâmetros. Se os dados forem inválidos, a aplicação apresenta uma mensagem de erro; caso contrário, a configuração é guardada no Firestore. Uma escrita bem-sucedida ativa a geofence, enquanto uma falha de comunicação origina a apresentação de um erro de rede.

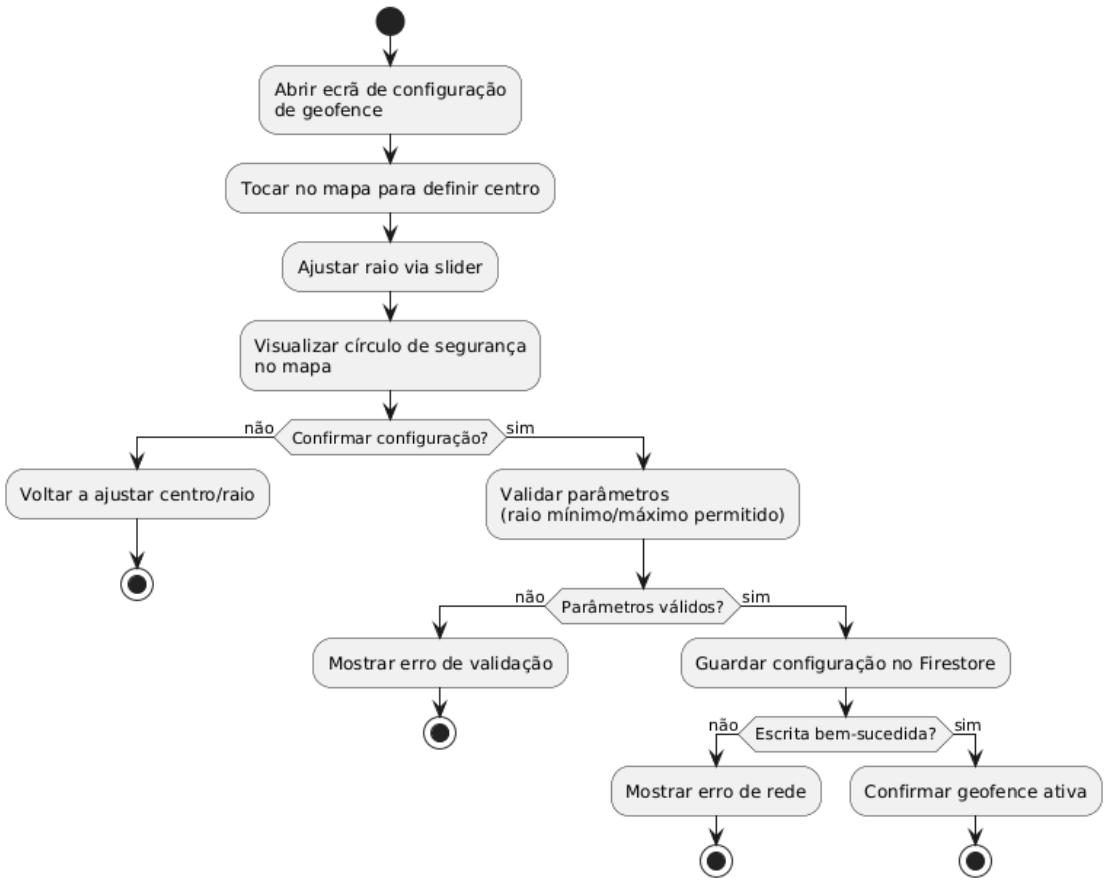


Figura 3.17. – Diagrama de atividades UML da configuração da geofence.

3.7.17 Diagrama de Atividades — Receção de Alerta

O diagrama de atividades de receção de alerta descreve o comportamento da aplicação quando é recebida uma notificação *push* através do FCM. O tratamento do evento depende do estado da aplicação, podendo esta encontrar-se em primeiro plano ou em execução no segundo plano do dispositivo móvel.

Como ilustrado na Figura 3.18, quando a aplicação está em primeiro plano é apresentado imediatamente um alerta modal; caso contrário, é utilizada uma notificação do sistema, que pode ser selecionada pelo cuidador. Em ambos os casos, o utilizador é encaminhado para o detalhe do alerta, onde são carregados do Firestore a posição atual, a distância ao perímetro e o histórico recente. O cuidador pode marcar o alerta como visto ou resolvido, atualizando o respetivo estado, ou mantê-lo ativo.

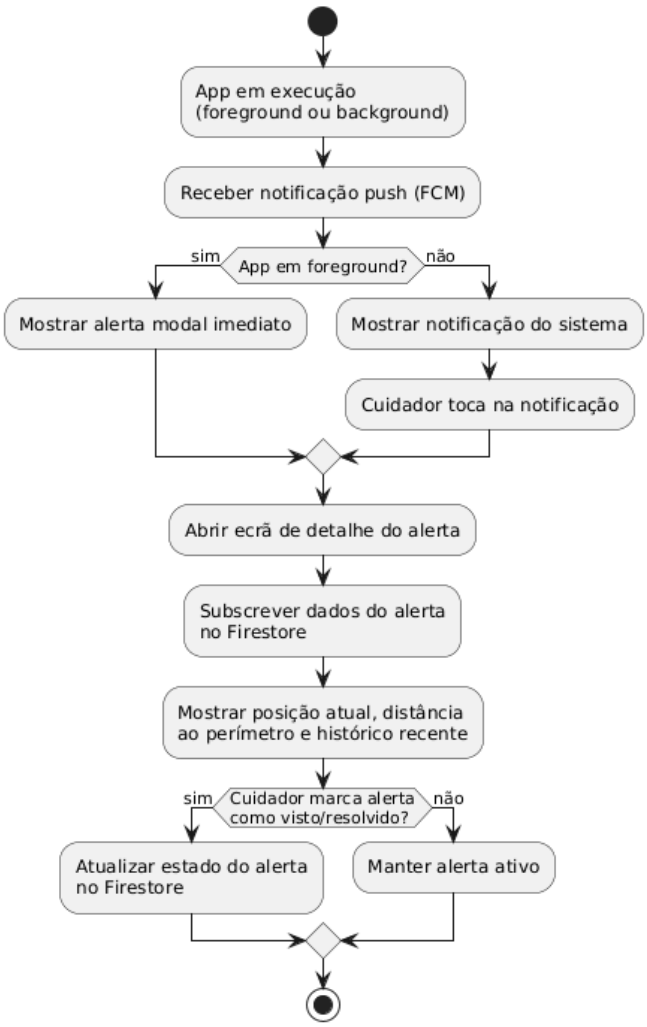


Figura 3.18. – Diagrama de atividades UML da receção e consulta de um alerta.

3.7.18 Diagrama de Atividades — Consulta de Bateria

O diagrama de atividades de consulta de bateria descreve o processo de apresentação do estado energético do dispositivo IoT na aplicação móvel. Ao abrir o ecrã de estado do dispositivo, a aplicação subscreve os dados de telemetria da bateria armazenados no Firestore e atualiza o respetivo indicador em tempo real.

Como ilustrado na Figura 3.19, o valor recebido é comparado com um limiar de 20%. Quando o nível de bateria se encontra abaixo deste valor, a aplicação apresenta um alerta visual de bateria baixa; caso contrário, o indicador mantém o estado normal. Este mecanismo permite ao cuidador identificar antecipadamente a necessidade de carregamento do dispositivo.

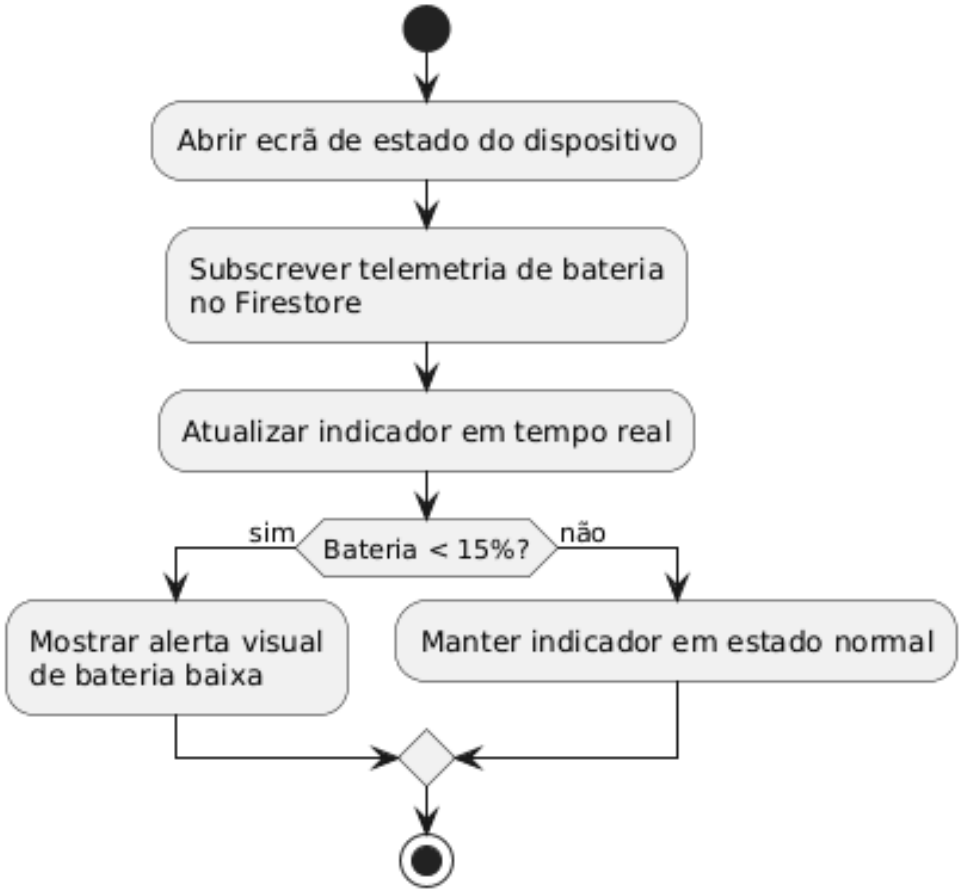


Figura 3.19. – Diagrama de atividades UML da consulta do nível de bateria.

3.8 Arquitetura de Navegação da Aplicação

3.8.1 Estrutura de Ecrãs

A estrutura de ecrãs da aplicação SafeTrack foi organizada de forma hierárquica, separando as funcionalidades de autenticação, emparelhamento do dispositivo e utilização diária. Esta organização permite que o cuidador aceda às principais funcionalidades através de uma navegação simples e coerente com o fluxo de utilização previsto.

Como ilustrado na Figura 3.20, o utilizador inicia o percurso nos ecrãs de autenticação, que incluem o login, o registo e a recuperação da palavra-passe. Após autenticação e emparelhamento do dispositivo, é disponibilizada a área principal da aplicação, composta pelo mapa em tempo real, estado do dispositivo, histórico de localizações, configuração da geofence e gestão de alertas.

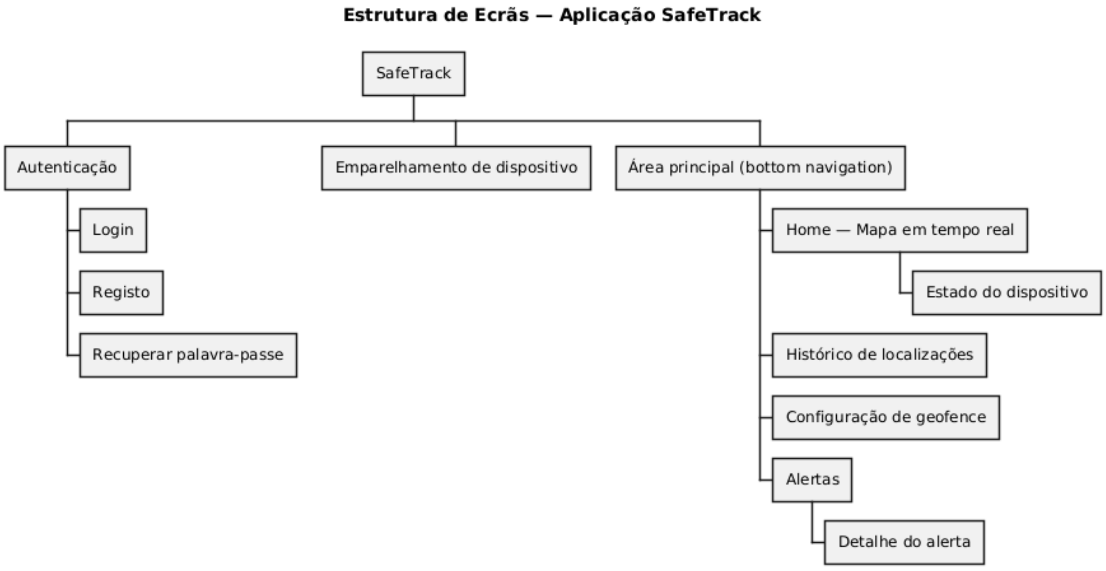


Figura 3.20. – Estrutura hierárquica dos ecrãs da aplicação móvel SafeTrack.

3.8.2 Esquema de Navegação

O esquema de navegação define os percursos disponíveis entre os diferentes ecrãs da aplicação SafeTrack. A navegação foi concebida para encaminhar o utilizador desde a página inicial e os processos de autenticação até à área principal de monitorização, mantendo acessíveis as funcionalidades complementares através de transições diretas.

Como apresentado na Figura 3.21, após o login ou registo, o cuidador pode emparelhar o dispositivo e aceder ao mapa principal. A partir deste ecrã é possível consultar o histórico de localizações, configurar a geofence, visualizar alertas, gerir o perfil e definições da conta ou consultar o estado do dispositivo. As transições identificadas no diagrama permitem documentar de forma inequívoca os caminhos de navegação previstos na aplicação.

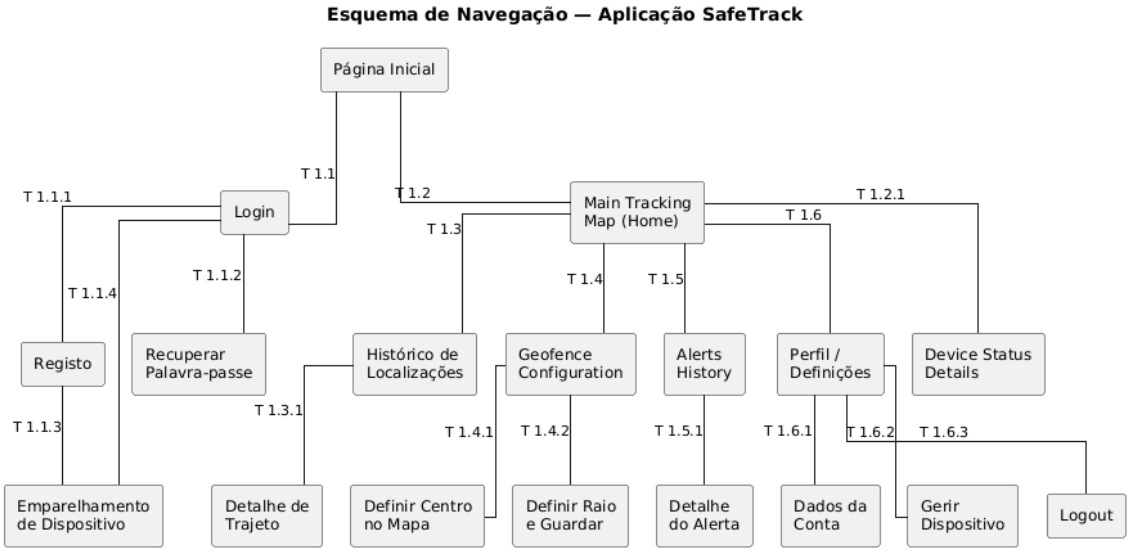


Figura 3.21. – Esquema de navegação entre os ecrãs da aplicação móvel SafeTrack.

3.8.3 Protótipo da Aplicação

O protótipo da aplicação traduz a arquitetura de navegação e os casos de uso definidos anteriormente numa representação visual dos principais ecrãs. Nesta fase são documentadas a hierarquia da informação, a disposição dos controlos e as transições previstas, sem confundir estas representações com capturas da aplicação final. Os ecrãs efetivamente implementados e os respetivos estados de erro serão apresentados no Capítulo 4.

A Figura 3.22 apresenta o logótipo da aplicação: um marcador de localização roxo, com uma abertura circular, sobre fundo branco. A imagem contém apenas este símbolo, sem controlos ou estados de utilização, constituindo o elemento de identificação visual retomado nas maquetas dos ecrãs.



Figura 3.22. – Logótipo da aplicação SafeTrack.

A maqueta de autenticação, associada ao UC-01 e apresentada na Figura 3.23, coloca o símbolo e o nome SafeTrack acima do formulário. São visíveis os campos de endereço de correio eletrónico e palavra-passe, o ícone de visualização da palavra-passe, a opção de memorizar o utilizador, a ligação de recuperação e o botão de início de sessão. A área inferior inclui a opção de continuar com Google, a ligação para criar uma conta e as ligações de privacidade, termos de serviço e ajuda; não é apresentado um formulário de registo distinto nesta imagem.

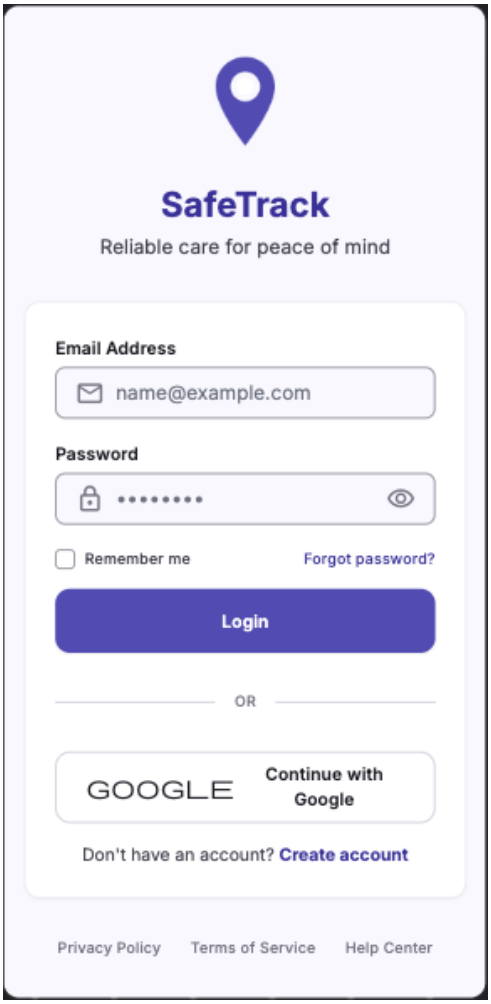


Figura 3.23. – Ecrã de autenticação (Login / Registo).

Na Figura 3.24, correspondente ao UC-04, o mapa ocupa a área superior e apresenta uma zona circular, um marcador identificado como ponto central e controlos de ampliação e redução. O painel inferior reúne o título de configuração da zona segura, o valor do raio, um controlo deslizante e o botão de guardar a configuração. Na barra de navegação inferior, o separador de geofencing encontra-se destacado, enquanto o cabeçalho contém o nome da aplicação, o menu e a imagem de perfil.

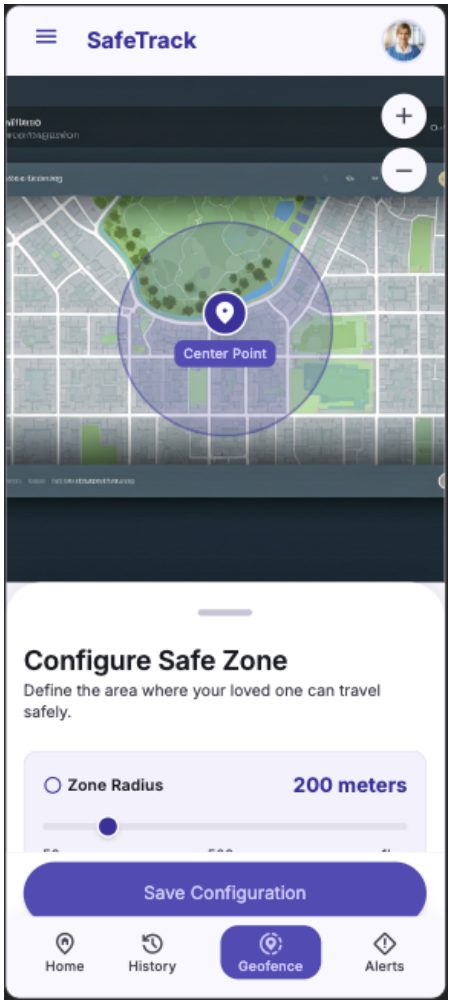


Figura 3.24. – Ecrã de configuração da zona segura.

A Figura 3.25 representa o ecrã principal associado ao UC-03, com o mapa como elemento dominante e uma zona circular desenhada em torno do marcador do dispositivo. Sobre o mapa surge um cartão com o nome do dispositivo, a indicação do tempo desde a atualização, o estado de permanência dentro da zona segura e uma barra de bateria. O cartão apresenta ainda os controlos *History* e *Find*; o cabeçalho inclui o menu, o nome da aplicação, um indicador de bateria e a imagem de perfil, e o separador inicial está selecionado na navegação inferior.

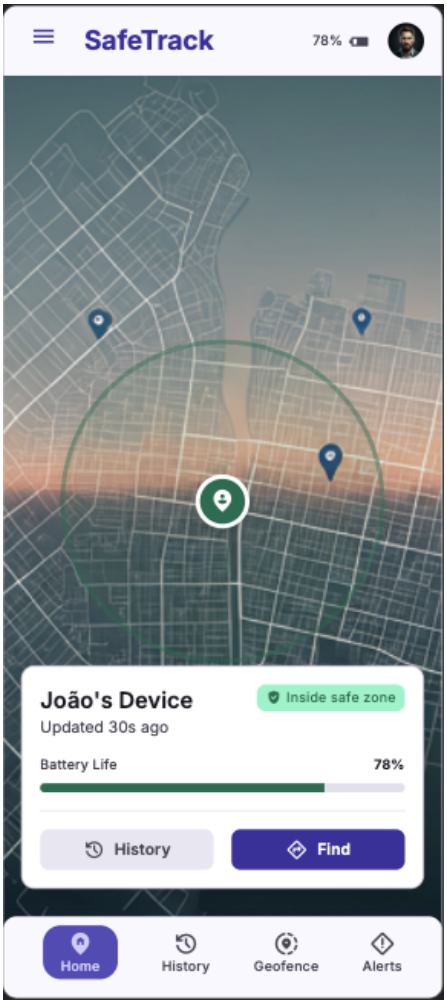


Figura 3.25. – Ecrã principal de rastreamento.

O ecrã da Figura 3.26, associado ao UC-05, organiza os alertas em cartões e disponibiliza filtros para todos os registos, os não lidos e os do próprio dia, acompanhados por contadores. Estão representados eventos de saída de zona, bateria fraca, acionamento de SOS, restabelecimento de ligação e chegada a casa, com ícones, cores e referências temporais distintos. São visíveis as ações de consultar o mapa e dispensar um alerta, bem como uma ação de chamada no cartão de SOS; a barra inferior assinala o separador de alertas como ativo.

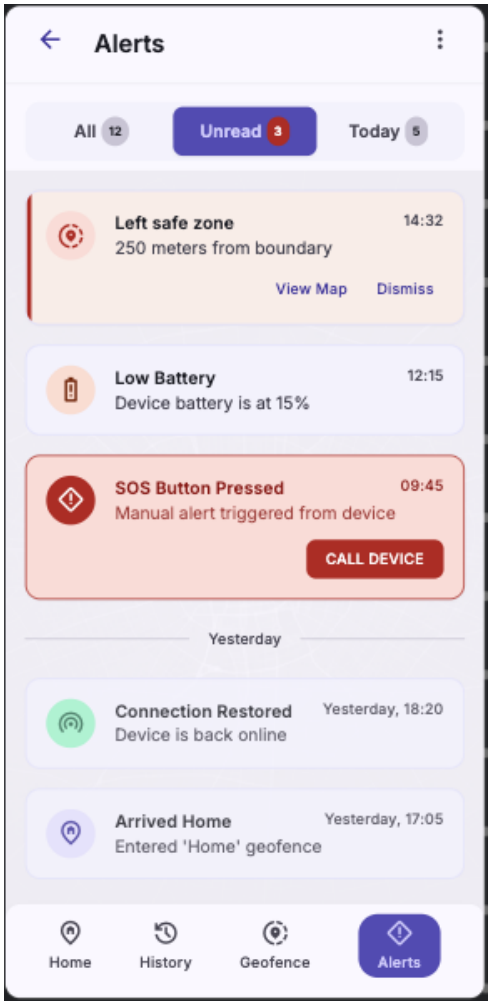


Figura 3.26. – Ecrã de histórico de alertas.

A Figura 3.27, correspondente ao UC-06, apresenta o nome do dispositivo e o estado de ligação no cartão superior. Seguem-se um painel de bateria com percentagem, barra de carga e estimativa de horas restantes, e um painel de última localização com mapa, endereço, indicação temporal e a ação *Update Now*. Na área inferior são visíveis cartões de rede e de estado do SIM, com indicações de tecnologia de acesso, intensidade do sinal, ativação e *roaming*, além da barra de navegação comum aos restantes ecrãs.

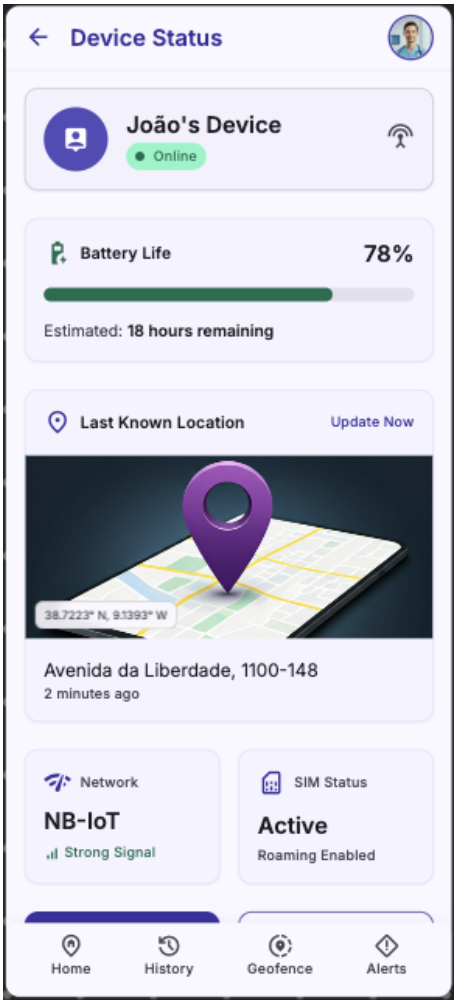


Figura 3.27. – Ecrã de estado do dispositivo.

As maquetas incluem elementos explorados durante a conceção e não implementados no produto mínimo viável: um botão de emergência acionado pelo portador (SOS), uma ação de chamada para o dispositivo, uma estimativa de autonomia restante em horas e um painel de estado do cartão SIM com indicação de *roaming*. Nenhum destes elementos consta dos requisitos funcionais da Tabela 3.1; a sua eventual concretização é remetida para o trabalho futuro da Secção 6.7, sem ampliar retroativamente os requisitos do produto mínimo viável. As maquetas foram produzidas em inglês. As capturas da implementação também incluem ecrãs em inglês, pelo que não se afirma uma tradução integral da interface para português. As capturas reais da aplicação efetivamente implementada encontram-se no Capítulo 4, distinguindo-se destas representações exploratórias.

3.9 Cronograma de Desenvolvimento

O desenvolvimento seguiu uma abordagem faseada, em que cada fase produziu um incremento funcional verificável antes do início da fase seguinte. Esta organização permitiu validar a camada de *backend* com um simulador de dispositivo antes de existir hardware funcional, isolando problemas de integração de problemas de lógica aplicacional.

A Tabela 3.7 apresenta as fases, o respetivo âmbito e o critério de conclusão adotado.

As Fases 3B e 4 concentram-se num único dia porque o *backend* já se encontrava validado com o simulador desde a Fase 1. A camada de aplicação pôde, assim, ser construída sobre um contrato de dados e de serviços estável, reduzindo o tempo necessário para a integração funcional.

Tabela 3.7. – Fases de desenvolvimento do SafeTrack.

Fase	Período	Âmbito	Critério de conclusão
0	anterior a 22/07/2026	Levantamento de requisitos, escolha de hardware, modelação UML e definição da arquitetura de três camadas.	Especificação aprovada e hardware adquirido.
1	22/07/2026	*Backend*: Cloud Functions `ingestTelemetry` e `processTelemetry`, modelo de dados Firestore, regras de segurança e simulador de dispositivo em Node.js.	Alerta gerado a partir de telemetria sintética, com regras de segurança validadas.
2	22–23/07/2026, encerrada a 23/07 às 19:22	Firmware: ciclo de aquisição, integração do modem SIM7000G, transporte HTTPS pela pilha nativa do modem, fila *offline* em MicroSD e *deep sleep*.	Telemetria real do dispositivo recebida pelo *backend*.
3A	23–24/07/2026; maquetas a 23/07 e *commit* a 24/07 às 02:29	Aplicação Flutter: autenticação, emparelhamento e mapa em tempo real.	Posição real visível no mapa da aplicação.
3B	24/07/2026, das 12:14 às 15:17	Aplicação Flutter: editor de cercas, alertas, histórico e integração FCM.	Fluxo completo do cuidador operacional.
4	24/07/2026, das 17:28 às 18:42	Multizona (Fases A a F): modelo de dados de múltiplas zonas, *debounce*, biblioteca de zonas e refinamento de UX.	Zonas circulares e poligonais com semântica partilhada entre aplicação e *backend*.
5	03/08/2026	Otimização e robustez: rede quente, aquisição adaptativa, fallback Wi-Fi de interior, canal Telegram e alertas de saúde do dispositivo.	Aquisição adaptativa operacional, com repouso configurado de 20 s; periodicidade real avaliada na Fase 6.
6	03/08 a 06/09/2026	Validação experimental: ensaios de campo, medição de autonomia, precisão e latência.	Requisitos não funcionais quantificados com veredicto.

4 Desenvolvimento e Implementação

Este capítulo descreve o sistema efetivamente construído e apresenta a evidência da sua implementação. As decisões, requisitos e modelos definidos no Capítulo 3 são aqui retomados apenas quando necessários para contextualizar uma opção de construção. Em particular, a descrição elétrica e o pinout apresentados na Secção 3.2.2 não são repetidos; o foco incide na forma como o firmware utiliza esses recursos, nos problemas encontrados e nas decisões tomadas durante o desenvolvimento.

4.1 Ambiente de Desenvolvimento e Ferramentas

4.1.1 Cadeia de Ferramentas e Versões

O desenvolvimento envolveu três cadeias de ferramentas independentes, uma por camada do sistema, coordenadas a partir de um único repositório. A Tabela 4.1 identifica as ferramentas efetivamente utilizadas e a respetiva função.

Tabela 4.1. – Cadeia de ferramentas por camada do sistema.

Camada	Ferramenta	Versão	Função
Dispositivo	PlatformIO (plataforma `espressif32`)	PlatformIO Core 6.1.19; plataforma `espressif32` 7.0.1	Gestão declarativa de dependências, compilação e gravação do firmware
Dispositivo	Framework Arduino para ESP32	`framework-arduinoespressif32` 3.20017 (Arduino-ESP32 2.0.17, baseado em ESP-IDF v4.4.7)	Camada de abstração de hardware sobre o ESP-IDF

Tabela 4.1 — continuação

Camada	Ferramenta	Versão	Função
Dispositivo	TinyGSM	0.12.0	Controlo do modem por comandos AT (registo de rede, contexto Packet Data Protocol (PDP), GNSS)
Dispositivo	ArduinoJson	7.4.3	Serialização do *payload* de telemetria
Dispositivo	StreamDebugger	1.0.1	Observação do diálogo de comandos AT no monitor série durante a depuração
Backend	Node.js	22	Runtime das Cloud Functions (declarado em `functions/package.json`)
Backend	firebase-functions	5.1.1	Software Development Kit (SDK) de definição de funções (geração 2)
Backend	firebase-admin	12.6.0	Acesso privilegiado ao Firestore, com *bypass* das regras de segurança
Backend	Firebase CLI	15.25.1	Implantação de funções, regras e índices; emulador local
Aplicação	Flutter / Dart SDK	Flutter 3.44.7 (canal *stable*); Dart 3.12.2; SDK mínimo `^3.12.2`	Desenvolvimento *cross-platform* da aplicação do cuidador
Aplicação	Xcode	26.6 (build 17F113)	Compilação e assinatura do *bundle* iOS
Transversal	Visual Studio Code	1.136.1	Ambiente de edição comum às três camadas
Transversal	Git e GitHub Actions	—	Controlo de versões e integração contínua

A escolha do PlatformIO em detrimento do Integrated Development Environment (IDE) Arduino justifica-se por três motivos concretos verificáveis no ficheiro `platformio.ini`: a gestão declarativa de dependências, que fixa as bibliotecas do projeto sem depender do estado global da máquina de desenvolvimento; a integração direta com o controlo de versões, uma vez que a configuração de compilação é um ficheiro de texto versionado; e

o suporte a múltiplos ambientes de compilação a partir da mesma base de código. Esta última capacidade foi determinante no diagnóstico dos problemas de transporte descritos na Secção 4.2.6: além do ambiente de produção (`esp32dev`), o projeto define dois ambientes de diagnóstico — `diag`, que substitui o ciclo normal por uma bateria de testes HyperText Transfer Protocol (HTTP) e HTTPS isolados, e `gnss`, que executa um ciclo de alimentação limpo do recetor GNSS e imprime a trama `CGNSINF` em bruto, distinguindo satélites visíveis de satélites utilizados [30].

Duas opções de compilação merecem registo, por corresponderem a decisões tomadas em resposta a problemas concretos e não a preferências de configuração. A primeira é a definição de `TINY_GSM_MODEM_SIM7000SSL`, que seleciona a variante da biblioteca em que o estabelecimento da ligação segura é delegado na pilha do próprio modem, em vez de ser executado no microcontrolador; esta escolha é coerente com a opção de transporte analisada na Secção 4.2.6. A segunda é o aumento do buffer de receção para 1024 bytes, através de `TINY_GSM_RX_BUFFER`, necessário porque as respostas às interrogações de estado do recetor GNSS e às operações HTTP excedem a dimensão predefinida da biblioteca e eram truncadas, produzindo falhas de interpretação difíceis de distinguir de erros de comunicação.

A configuração fixa igualmente as portas de gravação e de monitorização através do padrão `/dev/cu.*usb*`. A necessidade desta fixação decorreu de um diagnóstico realizado durante o desenvolvimento: em macOS, a deteção automática do PlatformIO seleciona a primeira porta série enumerada pelo sistema, que corresponde à porta de entrada Bluetooth e não à placa. O erro resultante, `Failed to connect to ESP32: No serial data received`, é indistinguível, para quem o observa pela primeira vez, de uma avaria de hardware ou de um problema de alimentação. O padrão adotado cobre os dois circuitos integrados de conversão Universal Serial Bus (USB) para série utilizados nas revisões da placa T-SIM7000G, o CP210x e o CH9102F.

As dependências da aplicação móvel são declaradas em `app/pubspec.yaml` e incluem `firebase_core`, `firebase_auth`, `cloud_firestore`, `firebase_messaging`, `google_maps_flutter`, `google_sign_in`, `provider` para gestão de estado, `google_fonts` e `intl`. Para efeitos de teste é ainda utilizado o pacote `fake_cloud_firestore`, que permite validar a lógica de serviços sem contactar o serviço real.

4.1.2 Organização do Repositório e Controlo de Versões

O projeto está organizado num *monorepo*, com uma pasta de topo por camada e a configuração do Firebase na raiz. A Tabela 4.2 apresenta a estrutura e a dimensão de cada componente.

Tabela 4.2. – Estrutura do repositório e dimensão de cada componente.

Diretório	Conteúdo	Dimensão
`firmware/`	Firmware C++ para a placa LilyGO T-SIM7000G, gerido por PlatformIO	`main.cpp`: 896 linhas; `sim7000_http.h`: 398 linhas
`functions/`	Cloud Functions Node.js (ingestão, geofencing, saúde, notificação, geolocalização)	5 módulos, aproximadamente 1300 linhas
`app/`	Aplicação Flutter do cuidador	44 ficheiros Dart, 5889 linhas
`simulator/`	Simulador de dispositivo em Node.js	221 linhas
`docs/`	Documentação de apoio: estado da arte, autonomia energética e modelo de ameaça	3 documentos
`design/`	Sistema de design e maquetas dos ecrãs	6 conjuntos
Raiz	`firestore.rules`, `firestore.indexes.json`, `firebase.json`	—

A separação em *monorepo*, em vez de repositórios independentes por camada, foi uma decisão deliberada: a semântica de decisão dentro/fora de uma zona existe simultaneamente no *backend* (`functions/geo.js`) e na aplicação (`app/lib/services/geo_utils.dart`), e manter os dois no mesmo repositório permite que alterações ao algoritmo sejam feitas e revistas num único *commit*, evitando a divergência silenciosa entre o que o servidor decide e o que o cuidador vê no ecrã.

A integração contínua está definida em `.github/workflows/ci.yml` e é executada em cada *push* e *pull request*. Executa os testes de lógica pura do *backend* (`npm test`) e, para a aplicação, a análise estática (`flutter analyze lib`) e os testes unitários (`flutter test`). Os testes de integração não integram este fluxo por exigirem o emulador do Firestore, cuja inicialização foi mantida deliberadamente fora do circuito mínimo de verificação.

4.1.3 Gestão de Configuração e Segredos

A configuração do sistema distingue explicitamente elementos públicos de segredos operacionais, e nenhum segredo se encontra sob controlo de versões.

No firmware, todos os parâmetros sensíveis residem em `firmware/src/config.h`, ficheiro incluído no `.gitignore`. O repositório contém apenas `config.example.h`, um modelo documentado com os mesmos campos e valores fictícios. Este ficheiro define a identidade do dispositivo (`DEVICE_ID`), a chave partilhada de autenticação (`DEVICE_KEY`), o *endpoint* da função de ingestão, os parâmetros da rede móvel (Access Point Name (APN) `TM`, sem utilizador nem palavra-passe, correspondente ao operador Things Mobile) e as temporizações do ciclo.

No *backend*, as variáveis sensíveis residem em `functions/.env` — igualmente excluído do controlo de versões, com um `.env.example` versionado como modelo. Este ficheiro contém a `DEVICE_KEY`, que tem de coincidir com a do firmware, a `GEOLOCATION_API_KEY` utilizada no fallback de interior e as credenciais do canal Telegram (`TELEGRAM_BOT_TOKEN` e `TELEGRAM_CHAT_ID`).

Uma decisão relevante em matéria de segurança respeita à separação das três chaves da Google utilizadas pelo sistema. A chave do Maps SDK é usada pela aplicação para desenhar o mapa e está restringida ao *bundle* iOS; a chave da Geolocation API é usada pelo *backend* para resolver listas de redes Wi-Fi em coordenadas e não possui restrição por aplicação, uma vez que a chamada parte de um servidor; e a chave da Geocoding API é usada pela aplicação na pesquisa de endereços do editor de zonas, estando declarada em `lib/config/secrets.dart`. Reutilizar a chave do Maps na chamada de servidor obrigaria a remover a restrição por aplicação, expondo-a a uso indevido, pelo que cada chave fica limitada ao contexto e à Application Programming Interface (API) a que se destina.

Na aplicação móvel, o ficheiro `lib/config/secrets.dart` está excluído do controlo de versões, existindo um `secrets.example.dart` versionado. O fluxo de integração contínua copia o modelo antes de compilar, o que é suficiente para a análise estática e para os testes unitários, uma vez que nenhum teste atual efetua chamadas reais a serviços externos.

A configuração de um novo ambiente de desenvolvimento resume-se, assim, a copiar os três ficheiros de exemplo e introduzir os valores reais, sem que qualquer credencial tenha de circular pelo repositório.

4.2 Implementação da Camada de Dispositivo

4.2.1 Arquitetura do Firmware e Ciclo Principal

O firmware está organizado em blocos funcionais dentro de `firmware/src/main.cpp`, complementado pelo módulo `sim7000_http.h`, que encapsula o transporte HTTPS. A execução não segue o modelo clássico `setup()`/`loop()` do Arduino: o dispositivo executa um ciclo completo dentro de `setup()` e termina em *deep sleep*, do qual acorda por temporizador. Cada ciclo é, por isso, um arranque completo do ESP32, o que elimina qualquer estado residual em memória volátil entre ciclos e simplifica o raciocínio sobre a máquina de estados.

A sequência de um ciclo é a seguinte: despertar por temporizador; inicializar o modem, reaproveitando-o se já estiver ligado; adquirir posição GNSS, com recurso ao varrimento Wi-Fi caso a aquisição falhe; ler o estado da bateria; construir o *payload* JSON; assegurar o registo na rede e o contexto PDP; enviar os *payloads* pendentes guardados no MicroSD; enviar a leitura atual; e entrar em *deep sleep* por `INTERVAL_S` segundos. Se o envio falhar, a leitura atual é gravada na fila do MicroSD em vez de ser perdida.

A principal divergência face ao modelo especificado no Capítulo 3 reside na gestão de energia: o modelo previa desligar completamente o modem entre ciclos, ao passo que a implementação o mantém ligado. Esta alteração, e as suas consequências, são descritas na Secção 4.2.7.

4.2.2 Inicialização e Gestão do Modem SIM7000G

O modem SIM7000G comunica com o ESP32 por Universal Asynchronous Receiver-Transmitter (UART) a 115 200 baud, utilizando os pinos `GPIO27` (transmissão), `GPIO26` (receção) e `GPIO4` (`PWRKEY`). A ligação é estabelecida através de um impulso na linha `PWRKEY`, cuja sequência temporal foi validada experimentalmente em hardware.

A decisão de implementação mais consequente desta camada foi manter o modem ligado entre ciclos, através do parâmetro `KEEP_MODEM_WARM`. A alternativa inicialmente implementada — desligar o modem antes do *deep sleep* — apagava a efeméride do recetor GNSS, obrigando a uma aquisição a frio em cada ciclo, com duração de dezenas de segundos. Mantendo o modem alimentado, a efeméride permanece válida e a aquisição passa a ser *hot*, na ordem do segundo.

Uma segunda decisão, introduzida numa iteração seguinte (`KEEP_NETWORK_WARM`), estende o mesmo princípio ao registo celular: o registo na rede e o contexto PDP são mantidos vivos entre ciclos. A função de ligação à rede é idempotente — se o modem já se encontrar registado e com contexto PDP ativo, retorna de imediato — e a configuração do modo de rede, operação dispendiosa em tempo, apenas é executada quando há efetivamente um registo a estabelecer. Esta alteração foi motivada por uma observação de campo: o intervalo real entre pontos era de aproximadamente 75 segundos quando o intervalo configurado era de 60, sendo os restantes 15 segundos consumidos não pela aquisição GNSS mas pelo restabelecimento do registo celular, do contexto PDP e do *handshake* Transport Layer Security (TLS) a partir do zero em cada ciclo.

4.2.3 Aquisição de Posição GNSS

A aquisição de posição enfrenta um problema de coexistência de radiofrequência documentado durante o desenvolvimento: a transmissão celular provoca *desense* do *front-end* do recetor GNSS, impedindo a aquisição de *fix*. A solução inicial consistiu em colocar o modem em modo avião (`AT+CFUN=4`) durante a aquisição, mantendo o GNSS alimentado — o recetor é independente do estado `CFUN` — e restaurando o rádio (`AT+CFUN=1`) apenas para transmitir. Esta solução resolvia o conflito de Radiofrequência (RF) mas destruía o registo celular em cada ciclo, com o custo de latência descrito na secção anterior.

A implementação final adota uma aquisição adaptativa em duas fases. Na primeira fase, o *fix* é tentado com o rádio celular ligado durante `GPS_FAST_TIMEOUT_S` segundos (20 s na configuração atual): o *desense* degrada sobretudo a aquisição a frio e, com efeméride viva, o *fix* é tipicamente *hot*, pelo que este é o caminho esperado. Só em caso de insucesso se recorre à segunda fase, com `CFUN=4` durante o tempo remanescente de `GPS_TIMEOUT_S` (90 s). A frequência de interrogação do estado do *fix* foi reduzida de 3 s para 1 s, uma vez que o tempo de resposta do modem não a justificava.

O ensaio de 6 de setembro de 2026 registou três intervalos consecutivos de 83 segundos entre posições. Estes intervalos são compatíveis com o insucesso da fase rápida e o recurso à segunda fase, mas não identificam a duração de cada etapa. Sem registos temporais internos, não é possível afirmar que a aquisição decorreu quase até ao limite de 90 segundos: o ciclo inclui também o repouso e as operações de transmissão. A efeméride mantém-se preservada porque o modem permanece ligado.

A Tabela 4.3 resume os parâmetros temporais implementados.

Tabela 4.3. – Parâmetros temporais da aquisição de posição.

Parâmetro	Valor	Justificação
`INTERVAL_S`	20 s	Repouso configurado, não duração do ciclo completo; intervalos de 83 s observados em aquisição degradada
`GPS_TIMEOUT_S`	90 s	Orçamento total de aquisição em condições normais
`GPS_FAST_TIMEOUT_S`	20 s	Fase rápida, com rádio celular ligado
`GPS_INDOOR_TIMEOUT_S`	15 s	Orçamento reduzido após deteção de interior
`INDOOR_APOS_N_FALHAS`	2 ciclos	Ciclos consecutivos sem *fix* que caracterizam interior

O carimbo temporal de cada leitura é obtido do próprio GNSS, em Coordinated Universal Time (UTC), dispensando sincronização por Network Time Protocol (NTP) em operação normal.

4.2.4 Estratégia de Posicionamento em Interior

Em ambiente interior o recetor GNSS não obtém *fix*, e o comportamento inicial do sistema — insistir durante os 90 segundos do orçamento de aquisição — degradava simultaneamente a autonomia e a frequência de reporte, com ciclos observados na ordem

dos 127 segundos em vez dos 20 esperados. Uma vez que o interior é precisamente onde a pessoa monitorizada passa mais tempo, esta era uma lacuna funcional relevante e não apenas uma ineficiência.

A solução implementada tira partido do rádio Wi-Fi já presente no ESP32. Ao fim de `INDOOR_APOS_N_FALHAS` ciclos consecutivos sem *fix*, o firmware assume ambiente interior e reduz o orçamento de aquisição para `GPS_INDOOR_TIMEOUT_S`; em paralelo, efetua um varrimento dos Basic Service Set Identifier (BSSID) visíveis, limitado aos `WIFI_MAX_APS` pontos de acesso de maior Received Signal Strength Indicator (RSSI), e envia essa lista ao *backend*. Um *fix* GNSS válido repõe imediatamente o orçamento normal.

O rádio Wi-Fi opera exclusivamente em modo de varrimento passivo: o dispositivo nunca se associa a qualquer rede. A resolução das coordenadas é efetuada no servidor, e não no dispositivo, por três razões: a chave da API nunca é escrita na memória flash do ESP32, de onde poderia ser extraída por acesso físico; o *payload* transmitido pela rede celular permanece pequeno; e a funcionalidade pode ser testada sem hardware, através do simulador [31].

4.2.5 Registo Local em MicroSD e Reenvio Diferido

A ausência de cobertura celular não pode implicar a perda de leituras. O cartão MicroSD, ligado por Serial Peripheral Interface (SPI) dedicado (`GPIO2`, `GPIO15`, `GPIO14` e `GPIO13`), aloja uma fila de persistência em `/queue`.

Cada *payload* não entregue é gravado como ficheiro independente, cujo nome é o carimbo temporal em milissegundos seguido da extensão `.json`. Esta convenção garante que a ordenação alfabética dos nomes coincide com a ordenação cronológica, dispensando qualquer índice adicional. No ciclo seguinte, antes de transmitir a leitura corrente, o firmware envia os pendentes por ordem cronológica, apagando cada ficheiro apenas após resposta HTTP 200. O envio interrompe-se ao primeiro insucesso, para preservar a ordem, e está limitado a `MAX_FLUSH` ficheiros por ciclo (20 na configuração atual), evitando que a recuperação de uma interrupção prolongada monopolize um único ciclo.

Uma leitura sem carimbo temporal fiável não é enfileirada, uma vez que não seria possível ordená-la nem interpretá-la corretamente à chegada.

4.2.6 Construção e Transmissão do Payload de Telemetria

O *payload* é serializado em JSON com a biblioteca ArduinoJson. Existem duas variantes. A variante GNSS contém o identificador do dispositivo, latitude, longitude, percentagem de bateria, o campo `source` com o valor `"gps"` e o carimbo temporal em milissegundos. A variante de interior substitui as coordenadas por uma lista de pares Media Access Control (MAC)/RSSI, declara `source` igual a `"wifi"` e omite o carimbo temporal quando este não está disponível, caso em que é a função de ingestão a carimbar o instante de chegada.

A transmissão constitui a decisão técnica mais consequente de toda a camada de dispositivo. O envio HTTPS **não** utiliza os *sockets* da biblioteca TinyGSM nem o ArduinoHttpClient: recorre à pilha HTTP nativa do próprio modem, através do conjunto de comandos Attention do modem (AT), nomeadamente `AT+SHCONF`, `AT+SHCONN`, `AT+SHREQ` e `AT+SHREAD`, implementados em `sim7000_http.h`.

Esta opção resultou de um diagnóstico conduzido por etapas no ambiente de compilação `diag` e teve duas causas independentes:

1. O caminho de leitura de *socket* da TinyGSM (`AT+CARECV`) revelou-se não funcional neste módulo, produzindo *timeout* de forma transversal a HTTP e HTTPS. A pilha nativa do modem contorna esse caminho.

2. A ligação TLS é estabelecida com `AT+SHSSL=1,`, isto é, TLS 1.2 com Server Name Indication (SNI) mas **sem validação do certificado do servidor**. Esta não foi uma escolha de conveniência: a firmware **SIM7000G R1529** apresenta um defeito de validação TLS que faz o comando `AT+SHCONN` retornar erro sempre que é pedida a validação de qualquer Certificate Authority (CA). O comportamento foi confirmado com três âncoras de confiança distintas — a raiz auto-assinada, o certificado intermédio e a raiz assinada cruzadamente — todas carregadas e convertidas com sucesso e com o relógio do modem sincronizado por NTP, o que exclui as causas habituais. Confirmou-se ainda que o comando `AT+CSSLCFG="authmode"` não existe nesta família de módulos e que o comando de escrita de ficheiros é `AT+CFSWFILE`, e não o `AT+CFSWRITE` das famílias SIM7070 e SIM7080.

A autenticação do dispositivo perante o *backend* é assegurada pelo cabeçalho `X-Device-Key`, validado pela função de ingestão. As implicações desta configuração em matéria de segurança são analisadas de forma explícita na Secção 4.3.7 e na Discussão.

4.2.7 Gestão de Energia

A gestão de energia do dispositivo resulta de um compromisso explícito entre latência de reporte e autonomia, resolvido a favor da latência [26, 27, 32–34].

Entre ciclos, o ESP32 entra em *deep sleep* com despertar por temporizador. O modem, porém, permanece ligado, tal como o recetor GNSS, cuja alimentação (`CGNSPWR=1`) nunca é ciclada. Esta opção preserva a efeméride e o registo celular. O parâmetro `INTERVAL_S`=20 s define apenas o repouso; os tempos de aquisição e transmissão acrescem à duração do ciclo. No ensaio de campo observaram-se intervalos de 83 s entre posições em aquisição degradada, e não uma periodicidade constante de 20 s. A alimentação contínua torna o consumo do GNSS em *tracking* contínuo a maior componente do consumo médio — superior, inclusivamente, aos picos de transmissão.

É importante notar que aumentar `INTERVAL_S` não resolveria este problema: os termos dominantes do consumo — GNSS em *tracking* e modem celular registado — são contínuos no tempo e não proporcionais ao número de ciclos. Aumentar o intervalo dilui apenas os termos por ciclo, deixando a linha de base praticamente intacta. Os caminhos efetivos para ganhar autonomia são o Power Saving Mode (PSM) e o extended Discontinuous Reception (eDRX) do lado celular e uma gestão mais fina da alimentação do GNSS, ambos identificados como trabalho futuro.

Para permitir a substituição da estimativa analítica por medição real, o firmware regista um registo de energia em `/log/energy.csv` no MicroSD, com uma linha por ciclo contendo carimbo temporal, duração da fase ativa, tensão e percentagem da bateria, origem da posição e resultado do envio. O utilitário `firmware/tools/analyze-energy-log.js` calcula, a partir desse ficheiro, a taxa de descarga e a autonomia extrapolada.

4.2.8 Leitura do Estado da Bateria

O nível de bateria é obtido por leitura do conversor analógico-digital no pino `GPIO35`, ligado à célula através de um divisor de tensão com fator 2. Para reduzir o efeito do ruído introduzido pelas variações de corrente do modem, cada leitura corresponde à média de 16 amostras consecutivas.

A conversão para percentagem é linear entre 3300 mV, considerado 0 %, e 4200 mV, considerado 100 %, valores correspondentes às tensões de corte e de plena carga de uma célula de iões de lítio. Trata-se de uma aproximação deliberadamente simples: a curva real de descarga não é linear, pelo que a percentagem indicada deve ser interpretada como estimativa relativa e não como medida exata de carga remanescente. Esta limitação é relevante para a interpretação do limiar de alerta de bateria fraca definido no *backend*.

No ensaio de campo, a sequência de percentagens reportadas foi 78, 77, 77, 69, 50 e 61 %. Registou-se uma queda de 19 pontos percentuais, seguida de uma subida de 11 pontos em 21 segundos. A amplitude observada não pode ser descrita como ruído de pequena dimensão: a média de 16 amostras não impediu estas oscilações. A variação é compatível com a sensibilidade da leitura de tensão à corrente do modem, mas não foi medida uma referência independente do estado de carga que permita quantificar esse efeito ou atribuir-lhe exclusivamente a anomalia.

4.3 Implementação da Camada de Cloud

As Cloud Functions são executadas na região `europe-west1`, escolhida pela proximidade geográfica a Portugal. A região das funções não determina a localização de todos os serviços Firebase, nomeadamente do Authentication [35]; as implicações são discutidas na Secção 6.5. O máximo de dez instâncias concorrentes limita a concorrência, mas não constitui um teto absoluto de custos nem proteção completa contra abuso.

4.3.1 Modelo de Dados Documental

O modelo de dados do Firestore está organizado pelos caminhos descritos na Tabela 4.4.

Tabela 4.4. – Modelo de dados documental implementado no Firestore.

Caminho	Conteúdo
`users/{uid}`	Perfil do cuidador, incluindo o *token* FCM e, opcionalmente, o identificador de conversa Telegram
`devices/{deviceId}`	Estado do dispositivo: proprietário, código de emparelhamento, último contacto, bateria, contador de leituras consecutivas fora de zona e sinalizadores de alerta ativo
`devices/{deviceId}/telemetry/{pointId}`	Histórico de posições, com origem (GNSS ou Wi-Fi) e precisão associada
`devices/{deviceId}/geofences/{zoneId}`	Zonas seguras: nome, tipo (círculo ou polígono), centro e raio ou lista de vértices, estado de ativação e cor
`alerts/{alertId}`	Alertas gerados, com tipo, dispositivo, posição, zona de referência e estado de resolução

A subcoleção de zonas substituiu um documento único (`geofence/current`), mantido como cópia de segurança. A migração é retrocompatível e executada, no máximo, uma vez por dispositivo: a aplicação cria uma zona inicial a partir da cerca antiga se a subcoleção estiver vazia, e regista um sinalizador no documento do dispositivo para nunca mais o fazer. Esta salvaguarda corrige um defeito identificado durante o desenvolvimento, em que a eliminação de todas as zonas pelo utilizador fazia ressuscitar a zona inicial na abertura seguinte da aplicação.

4.3.2 Função de Ingestão HTTP

A função `ingestTelemetry` constitui o único ponto de entrada de telemetria no sistema. Aceita exclusivamente o método `POST` e autentica o dispositivo por comparação do cabeçalho `X-Device-Key` com o valor guardado nas variáveis de ambiente da função. A ausência ou invalidade da chave produz resposta 401 antes de qualquer acesso ao Firestore, o que limita o custo de pedidos não autorizados.

Os códigos de resposta implementados distinguem situações que seriam indistinguíveis num tratamento genérico de erro: 405 para método não permitido, 401 para chave ausente ou inválida, 404 quando o `deviceId` não existe em `devices/{id}`, 400 para *payload* malformado, 422 para *payload* bem formado cuja lista de pontos de acesso Wi-Fi a Google não consegue resolver em coordenadas, 500 para erro de configuração do servidor e 200 para sucesso. O código 404 devolve o corpo *Dispositivo desconhecido. Faz o emparelhamento primeiro.*. A distinção é intencional: o 401 indica que a função não reconhece a identidade apresentada, enquanto o 404 indica que a autenticação foi aceite, mas o dispositivo indicado não se encontra registado. De igual modo, o 400 identifica um erro do cliente, ao passo que o 422 indica que o dispositivo se comportou corretamente mas o ambiente não permitiu a resolução da posição. O código 405 foi igualmente verificado no sistema em produção.

4.3.3 Processamento Espacial e Deteção de Evasão

A função `processTelemetry` é acionada pela criação de cada documento de telemetria e concentra a lógica de geofencing. A geometria está isolada no módulo `functions/geo.js`, que implementa a fórmula de Haversine com raio terrestre de 6 371 000 m para zonas circulares e o algoritmo de *ray casting* para zonas poligonais.

A semântica adotada é a de segurança por união: o portador é considerado seguro enquanto se encontrar dentro de, pelo menos, uma zona ativa, sendo gerado alerta apenas quando se encontra fora de todas.

O algoritmo de *ray casting* é deliberadamente idêntico no *backend* e na aplicação. Esta duplicação controlada — e não uma reimplementação independente — reduz o risco de divergência entre o indicador apresentado ao cuidador e a decisão que gera o alerta, e ambos os lados são cobertos por testes automatizados.

O tratamento de incerteza aplica-se às posições de origem Wi-Fi, cuja resolução no servidor produz uma estimativa de precisão. Se o círculo de incerteza dessas posições ainda intersetar a zona mais próxima, a leitura é classificada como incerta: não contribui para o contador de evasão nem resolve alertas em aberto. Sem este tratamento, uma posição obtida por Wi-Fi com 50 m de incerteza no interior de uma habitação poderia gerar alertas falsos. As leituras GNSS entram atualmente no sistema com `accuracy=0`, porque a função de ingestão inicializa esse campo a zero e apenas lhe atribui outro valor quando a posição é

resolvida por Wi-Fi no servidor. A avaliação utiliza a distância à fronteira da zona, e não a distância ao centro, sendo esta última utilizada apenas na composição do texto do alerta.

4.3.4 Histerese e Supressão de Alertas Duplicados

O alerta de evasão apenas é gerado na segunda leitura consecutiva fora de todas as zonas ativas, através de um contador mantido no documento do dispositivo. Este *debounce* evita que uma única leitura ruidosa junto à fronteira desencadeie uma notificação, ao custo de um ciclo adicional de latência — um compromisso cuja consequência quantitativa é analisada na Secção 5.5.4.

De igual modo, o alerta de bateria fraca é emitido quando a carga desce a 20 % ou menos, mas só volta a armar-se acima de 30 %. Sem esta banda morta, um dispositivo cuja leitura oscilasse entre 19 % e 21 % — situação verosímil, dada a variação do Analog-to-Digital Converter (ADC) com a corrente do modem — geraria notificações repetidas em poucos minutos.

O ensaio de campo revelou, contudo, oscilações de 19 e 11 pontos percentuais, superiores às variações de um ou dois pontos consideradas no dimensionamento desta banda morta. A histerese de 20 %/30 % não é suficiente para tornar o alerta robusto perante oscilações desta ordem: uma subestimação da leitura pode provocar um disparo mesmo com carga real bastante acima de 20 %, sem que esse falso disparo tenha sido demonstrado no ensaio.

Todos os alertas são emitidos apenas na transição de estado, registada por sinalizadores no documento do dispositivo.

4.3.5 Monitorização de Saúde do Dispositivo

O geofencing só é capaz de avisar enquanto o dispositivo comunica. Se este ficar sem bateria, sem cobertura ou avariar, deixa de emitir telemetria e o cuidador não é notificado — precisamente no momento em que a pessoa deixa de estar localizável. Esta lacuna é coberta por dois mecanismos de naturezas distintas.

A bateria fraca é avaliada em cada telemetria, uma vez que o dado acompanha o *payload*. A avaliação é feita no início da função de processamento, antes da lógica de geofencing, porque vários ramos desta última terminam antecipadamente.

O silêncio prolongado, pelo contrário, não pode ser detetado por um acionador de eventos, porque a ausência de eventos não gera eventos. É detetado pela função `checkDeviceHealth`, agendada de cinco em cinco minutos, que executa uma única consulta a dispositivos cujo último contacto seja anterior ao limiar. O custo desta consulta é constante e independente do número de dispositivos saudáveis; dispositivos que nunca comunicaram são naturalmente excluídos pela desigualdade.

Um dispositivo sem cuidador associado é contabilizado pela varredura de saúde, mas não gera alerta, uma vez que não existe destinatário a notificar. Consequentemente, a vigilância do silêncio só produz alertas depois de concluído o emparelhamento do dispositivo com uma conta de cuidador.

O limiar de silêncio está fixado em 15 minutos, valor deliberadamente superior aos 5 minutos que a aplicação utiliza para apresentar o indicador de ausência de sinal. A diferença é intencional: o indicador na aplicação é passivo, ao passo que o alerta desperta o telemóvel do cuidador. A correspondência anteriormente estimada de cerca de 45 ciclos pressupunha um ciclo completo de 20 segundos. Como os intervalos reais podem atingir 83 segundos em aquisição degradada, o limiar deve ser interpretado como tempo sem telemetria, não como uma contagem fixa de ciclos falhados.

A resolução de alertas é filtrada por tipo. O regresso a uma zona segura não diz nada sobre o estado da bateria nem sobre a comunicação, pelo que fechar indiscriminadamente todos os alertas em aberto ocultaria problemas reais ao cuidador. A filtragem é efetuada em memória, e não por consulta, porque exigiria um índice composto no Firestore para um conjunto de documentos que é, por construção, muito reduzido.

O alerta de silêncio transporta a última posição conhecida do dispositivo — uma leitura adicional efetuada apenas nos dispositivos em falha. É o único dado acionável que resta quando um localizador deixa de comunicar.

4.3.6 Notificações Push e Canal Alternativo

A notificação do cuidador utiliza dois canais em paralelo, disparados de forma independente, de modo que a falha de um não afete o outro.

O canal FCM corresponde ao mecanismo nativo previsto na especificação. A sua utilização em iOS exige, contudo, uma chave Apple Push Notification service (APNs), que apenas o Apple Developer Program permite gerar. Sem essa chave, o alerta era corretamente criado no Firestore mas nunca chegava ao telemóvel.

Foi por isso implementado um canal alternativo baseado no Telegram, que envia a mensagem de alerta acompanhada de um marcador de localização, sobre o qual o cuidador pode tocar para abrir o mapa. O canal notifica igualmente o regresso à zona segura. A latência observada deste canal situa-se entre um e dois segundos, pelo que não constitui a componente dominante da latência total do alerta.

O canal FCM permanece integralmente implementado e operacional, aguardando apenas a disponibilização de uma chave APNs ou a utilização de um dispositivo Android, onde não existe essa restrição.

4.3.7 Autenticação e Regras de Segurança

O sistema distingue três identidades: o cuidador, autenticado por Firebase Authentication; o dispositivo, autenticado por chave partilhada no cabeçalho HTTP; e as próprias Cloud Functions, que operam através do Admin SDK e, por essa via, ignoram as regras de segurança do Firestore.

As regras implementadas em `firestore.rules` obedecem a quatro princípios. Primeiro, o cuidador acede exclusivamente aos seus próprios dados, verificados por comparação do identificador de utilizador autenticado. Segundo, a telemetria e os alertas são de escrita exclusivamente reservada às Cloud Functions: qualquer escrita direta por parte de um cliente é negada. Terceiro, o dispositivo nunca comunica diretamente com o Firestore, pelo que não necessita de regras próprias [36]. Quarto, o cliente não pode criar documentos

de dispositivo: o provisionamento é reservado ao Admin SDK, e o emparelhamento é uma atualização de um documento previamente provisionado.

O emparelhamento exige o identificador do dispositivo e um código de seis dígitos. A regra de atualização apenas aceita a reivindicação de um dispositivo sem proprietário se o código enviado pelo cliente coincidir com o código guardado. O cliente reenvia o código mas nunca o lê: a leitura do documento do dispositivo continua reservada ao proprietário. Deste modo, conhecer o identificador de um dispositivo deixou de ser suficiente para o reivindicar.

O quarto princípio resultou de um defeito identificado em 6 de setembro de 2026 durante a execução de TF-02.2. Uma escrita sobre um documento inexistente é avaliada como criação, não como atualização. A regra de criação então em vigor, `allow create: if isSignedIn() && request.resource.data.ownerId == request.auth.uid`, permitia a qualquer utilizador autenticado criar um documento em `devices/{identificador}` declarando-se proprietário, sem apresentar o código de emparelhamento. Um atacante poderia assim apropriar-se de um identificador ainda não provisionado; quando o equipamento real começasse a transmitir, a função de ingestão encontraria o documento existente e aceitaria a telemetria, permitindo ao atacante receber a localização da pessoa monitorizada. A previsibilidade dos identificadores agravava este risco.

A correção foi aplicada e implantada no mesmo dia, substituindo a regra por `allow create: if false`. Os dispositivos passaram a ser provisionados exclusivamente pelo Admin SDK, conforme o Anexo C.7; o cliente apenas pode atualizar um dispositivo existente segundo as condições de emparelhamento. A verificação posterior de TF-02.6 confirmou que a reivindicação legítima, a libertação e a nova reivindicação continuavam a funcionar, sem regressão no fluxo testado. Os resultados após correção constam da Tabela 5.5.

O sistema apresenta duas limitações de segurança conhecidas, identificadas e documentadas de forma explícita: a ausência de validação do certificado do servidor no troço dispositivo–nuvem, descrita na Secção 4.2.6, e a utilização de uma chave partilhada única, sem rotação nem individualização por dispositivo, sem assinatura do corpo do pedido e sem proteção contra repetição. Ao contrário do defeito de criação de dispositivos, estas limitações permanecem por resolver. O modelo de ameaça correspondente, o mapa de risco qualitativo e o caminho de mitigação encontram-se desenvolvidos na Secção 6.3.

4.3.8 Implantação, Cold Starts e Custos Operacionais

A implantação é efetuada pela CLI do Firebase, separadamente para as funções e para as regras e índices do Firestore. A função agendada de verificação de saúde exige a ativação prévia da Cloud Scheduler API no projeto.

Durante o desenvolvimento identificou-se um comportamento relevante do processo de implantação: alterações efetuadas apenas ao ficheiro de variáveis de ambiente não são detetadas pela CLI, que compara o resumo criptográfico do código. A propagação de uma nova chave exige, por isso, uma alteração ao código ou uma implantação forçada.

4.4 Implementação da Aplicação Móvel

4.4.1 Arquitetura da Aplicação e Gestão de Estado

A aplicação está estruturada em camadas — modelos, serviços, fornecedores de estado, ecrãs e componentes reutilizáveis — distribuídas por 44 ficheiros Dart num total aproximado de 5900 linhas.

A gestão de estado assenta no pacote `provider`. Existem dois fornecedores principais: um responsável pela autenticação e outro pelo estado do dispositivo emparelhado, ligados por `ChangeNotifierProxyProvider`, de modo que a alteração do utilizador autenticado reconfigura automaticamente o estado do dispositivo. A navegação de topo é decidida por um componente único que observa o estado de autenticação e de emparelhamento e encaminha para o ecrã adequado, evitando transições imperativas dispersas pelo código.

O tema visual está implementado como extensão de tema, com variantes clara e escura. Todas as cores são obtidas a partir dessa extensão, e nunca literalmente, o que garante que qualquer componente novo se adapta automaticamente ao modo do sistema; os estilos do mapa seguem a mesma convenção.

4.4.2 Autenticação: Login, Registo e Recuperação

A autenticação recorre ao Firebase Authentication, com registo e autenticação por correio eletrónico e palavra-passe e recuperação de palavra-passe por mensagem de reposição. A validação dos campos está isolada num módulo próprio, sendo os erros do serviço traduzidos para mensagens legíveis apresentadas num componente de erro comum a todos os ecrãs de autenticação.

A autenticação através de conta Google está implementada, mas não se encontra operacional na configuração atual, por motivo documentado na Secção 4.6.

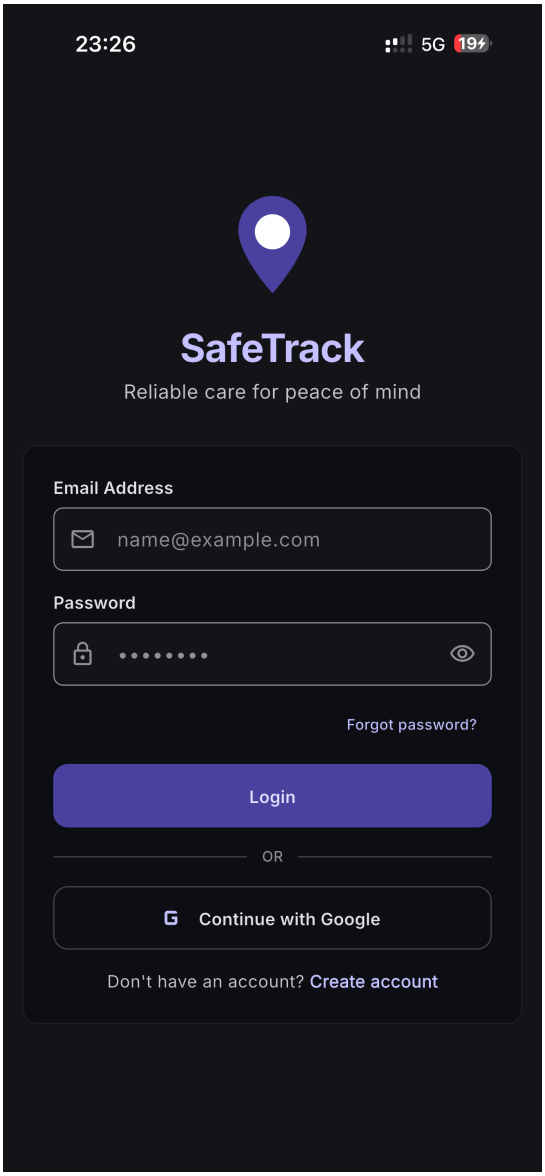


Figura 4.1. – Ecrã de autenticação da aplicação SafeTrack.

4.4.3 Emparelhamento por Código

O ecrã de emparelhamento solicita o identificador do dispositivo e o código de seis dígitos. A operação é uma única atualização ao documento do dispositivo, em que o cliente declara a sua propriedade e reenvia o código, sendo a validação efetuada pelas regras do Firestore, do lado do servidor, conforme descrito na Secção 4.3.7.

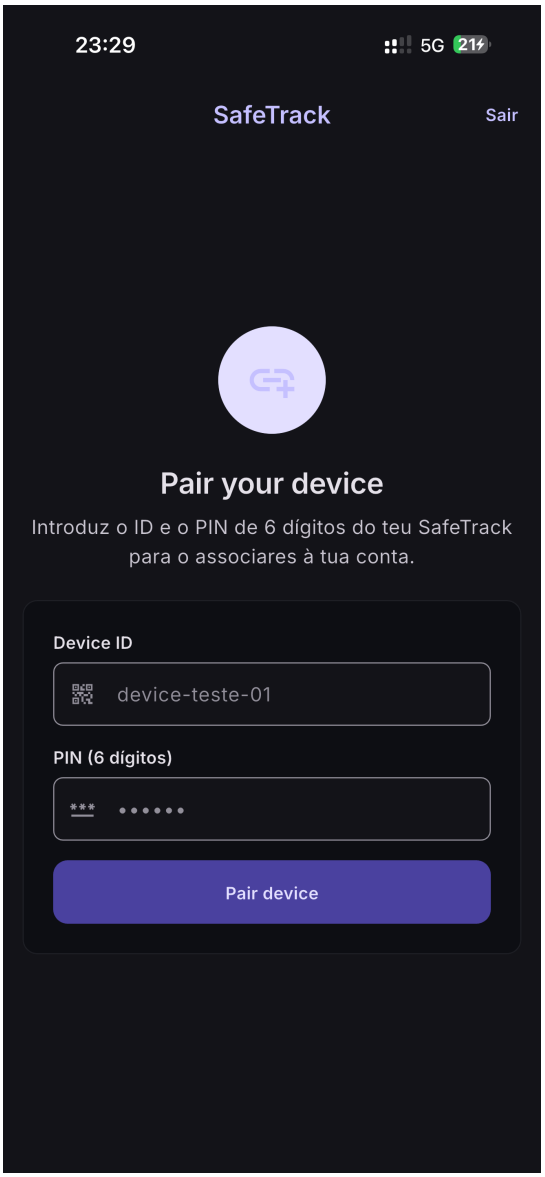


Figura 4.2. – Ecrã de emparelhamento de um dispositivo através de identificador e código.

4.4.4 Mapa em Tempo Real

O ecrã principal apresenta um mapa Google com a última posição conhecida do dispositivo, as zonas seguras configuradas e um cartão de estado com o nível de bateria e o indicador de dentro ou fora de zona. A subscrição do documento de telemetria é feita em tempo real, sendo o fluxo memorizado por dispositivo para evitar subscrições duplicadas na reconstrução da interface.

Quando a última telemetria é anterior a cinco minutos, o marcador passa a cinzento e é apresentado um aviso de ausência de sinal. Os marcadores são gerados em tempo de execução por desenho em tela, uma vez que a paleta predefinida do componente de mapa não disponibiliza a variante cinzenta necessária para representar este estado.

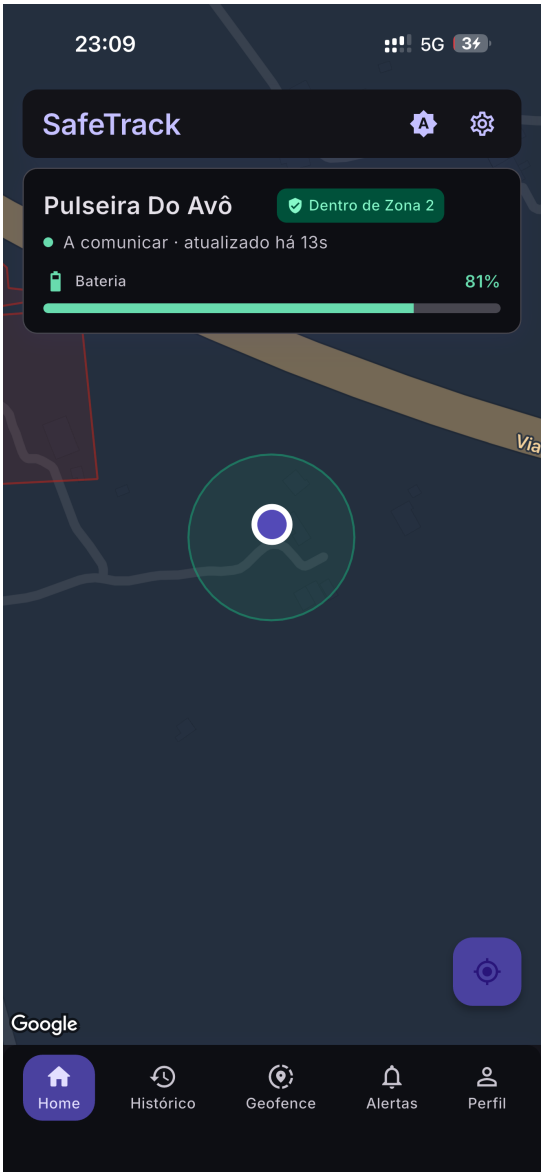


Figura 4.3. – Ecrã principal com a localização do dispositivo, a zona segura e o estado da bateria.

4.4.5 Editor de Cercas Virtuais

O separador de geofencing apresenta a biblioteca de zonas do dispositivo e permite criar, editar, ativar, desativar e eliminar zonas. O editor suporta zonas circulares, definidas por centro arrastável e raio ajustável, e zonas poligonais, definidas por uma sequência de vértices. É ainda possível localizar um endereço por pesquisa, evitando a navegação manual no mapa.

A decisão de pertença a uma zona é feita pelo mesmo algoritmo utilizado no *backend*, conforme descrito na Secção 4.3.3.

Durante o desenvolvimento foi identificado e corrigido um defeito na composição do documento de zona: a operação de limpeza de campos do tipo oposto, válida em atualizações, era rejeitada na criação de um documento novo e provocava a terminação abrupta da aplicação. A correção está coberta por teste automatizado.

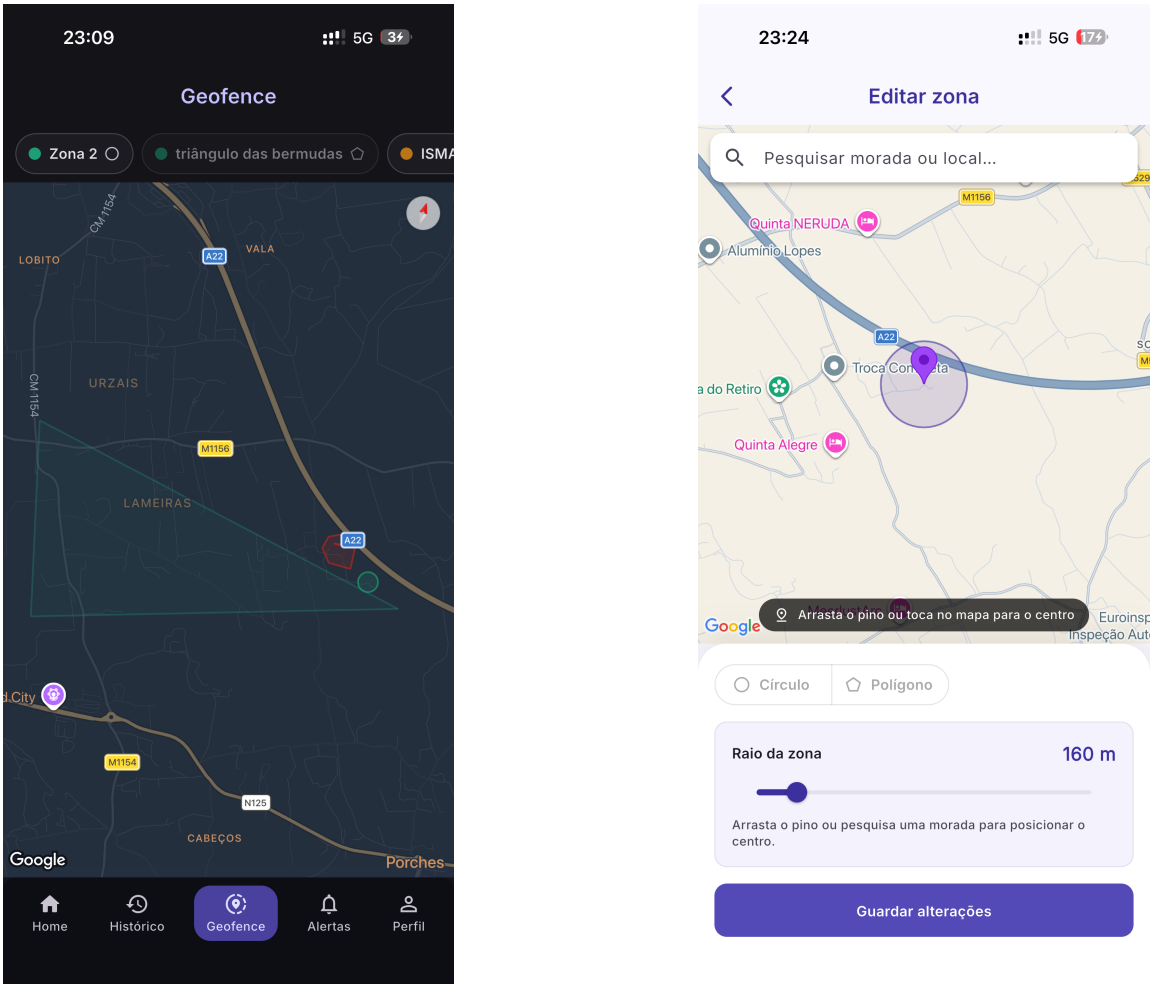


Figura 4.4. – Biblioteca de zonas seguras e editor de uma cerca virtual circular.

4.4.6 Histórico de Localizações

O separador de histórico apresenta o percurso do dispositivo sob a forma de trilho no mapa e de lista cronológica de posições, com filtros temporais e um painel de estatísticas do período selecionado.

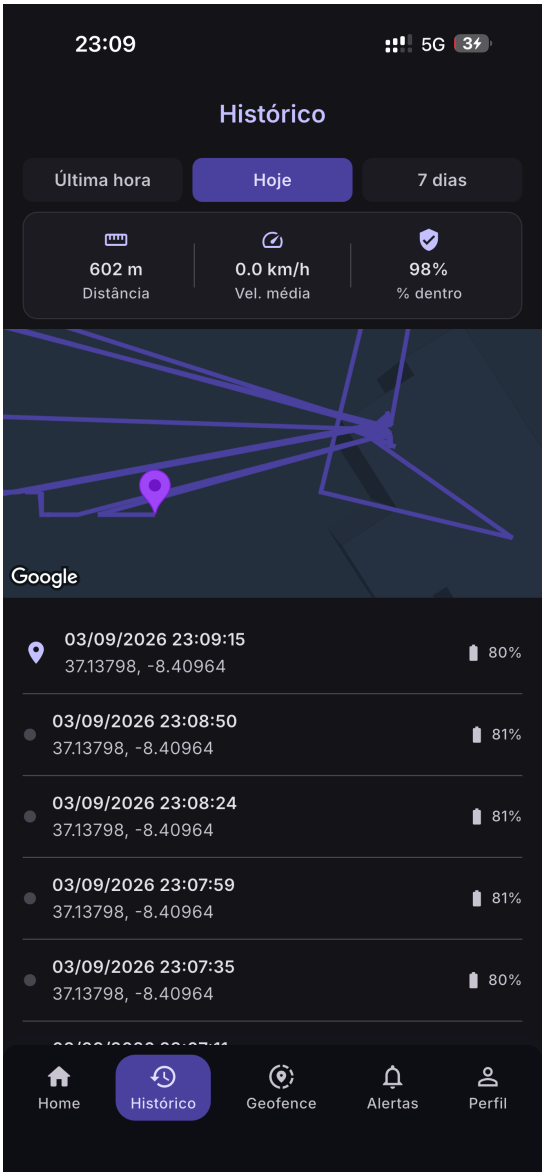


Figura 4.5. – Ecrã de histórico com o percurso, as posições e as estatísticas do período selecionado.

4.4.7 Alertas: Notificação, Lista e Detalhe

O separador de alertas apresenta, em tempo real, a lista de alertas do dispositivo, permitindo consultar o detalhe de cada um e marcá-lo como resolvido. Com a aplicação em primeiro plano, a receção de uma mensagem apresenta um aviso imediato; caso contrário, o alerta chega por notificação do sistema. O detalhe apresenta a posição, a distância ao perímetro e o histórico recente.

A gestão do *token* de notificação é feita à entrada da área autenticada, sendo o *token* guardado no perfil do utilizador.



Figura 4.6. – Lista de alertas gerados pelo sistema e respetivo estado de resolução.

4.4.8 Estado do Dispositivo e Bateria

O ecrã de definições do dispositivo permite consultar o estado de comunicação e de bateria, renomear o dispositivo e desfazer o emparelhamento. O indicador de bateria assinala visualmente os níveis baixos, em coerência com o limiar utilizado pelo *backend*.

O perfil do cuidador centraliza o acesso à gestão do dispositivo emparelhado, às preferências de aparência e ao encerramento da sessão. A partir deste ecrã, o cuidador pode abrir a gestão do dispositivo para alterar o nome apresentado, copiar o identificador ou desfazer o emparelhamento.

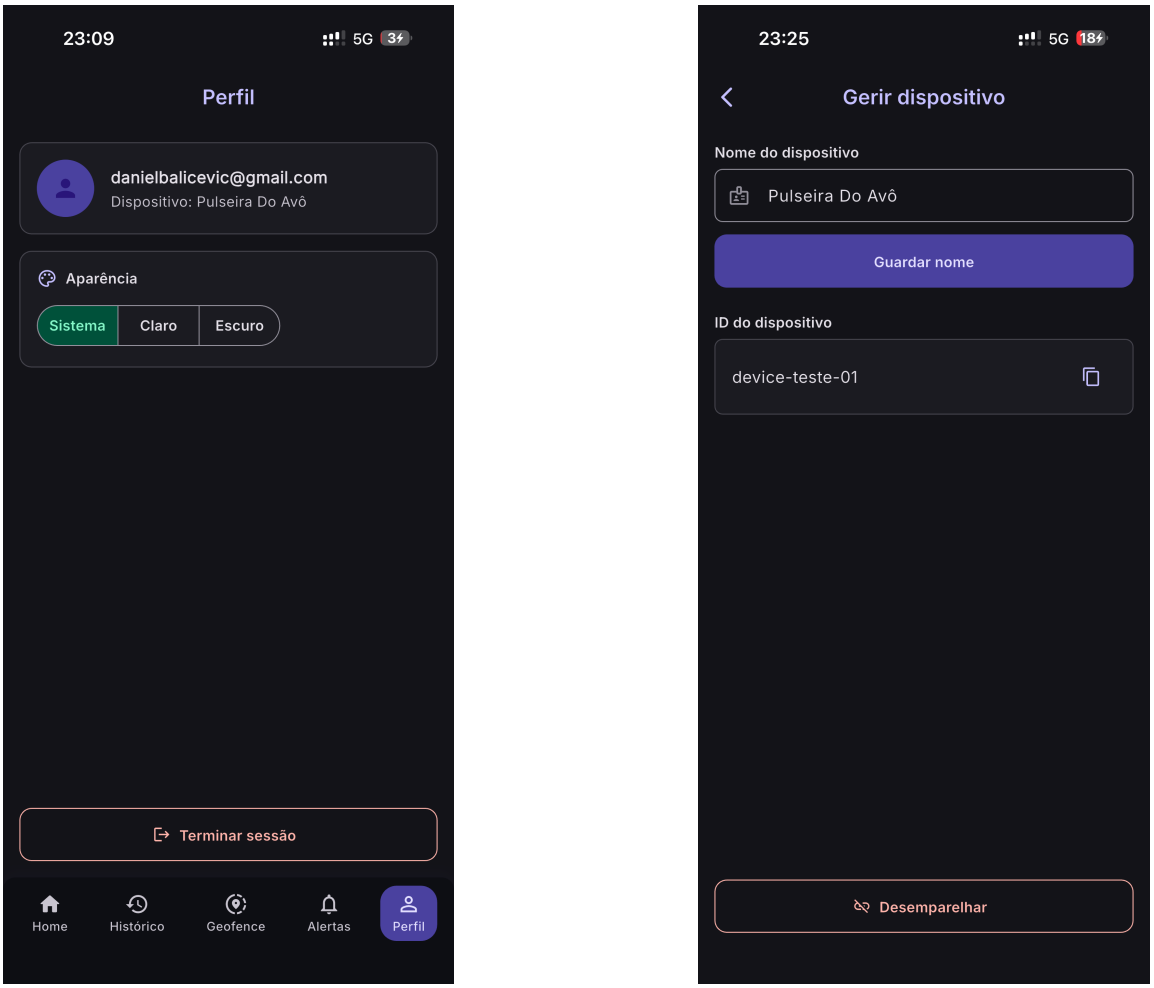


Figura 4.7. – Ecrãs de perfil do cuidador e de gestão do dispositivo emparelhado.

4.4.9 Permissões, Estados de Erro e Degradação

A aplicação apresenta estados explícitos para as situações de degradação previstas: ausência de dispositivo emparelhado, ausência de telemetria recente, ausência de zonas configuradas e falha de autorização de notificações. Cada um destes estados corresponde a um ecrã ou componente próprio, e não a um ecrã vazio, o que evita que a ausência de dados seja interpretada como avaria.

4.5 Integração e Montagem do Protótipo

A Figura 4.8 apresenta duas vistas da montagem do protótipo numa caixa de plástico transparente aberta. Na vista da placa são visíveis a plataforma LilyGO T-SIM7000G, o cartão MicroSD, a antena GNSS ligada por cabo e a antena celular externa; a vista oposta evidencia a célula 18650 instalada no respetivo suporte. As fotografias documentam o conjunto sobre uma superfície de apoio, sem o telemóvel do cuidador, não constituindo um registo do ensaio exterior.

O invólucro é uma caixa genérica, sem vedação nem fixação, suficiente para proteção mecânica básica durante o ensaio, mas não para utilização continuada. A resistência ambiental e a adequação ao uso pelo portador não foram validadas experimentalmente, por ausência de ensaios específicos; um invólucro adequado permanece como trabalho futuro.

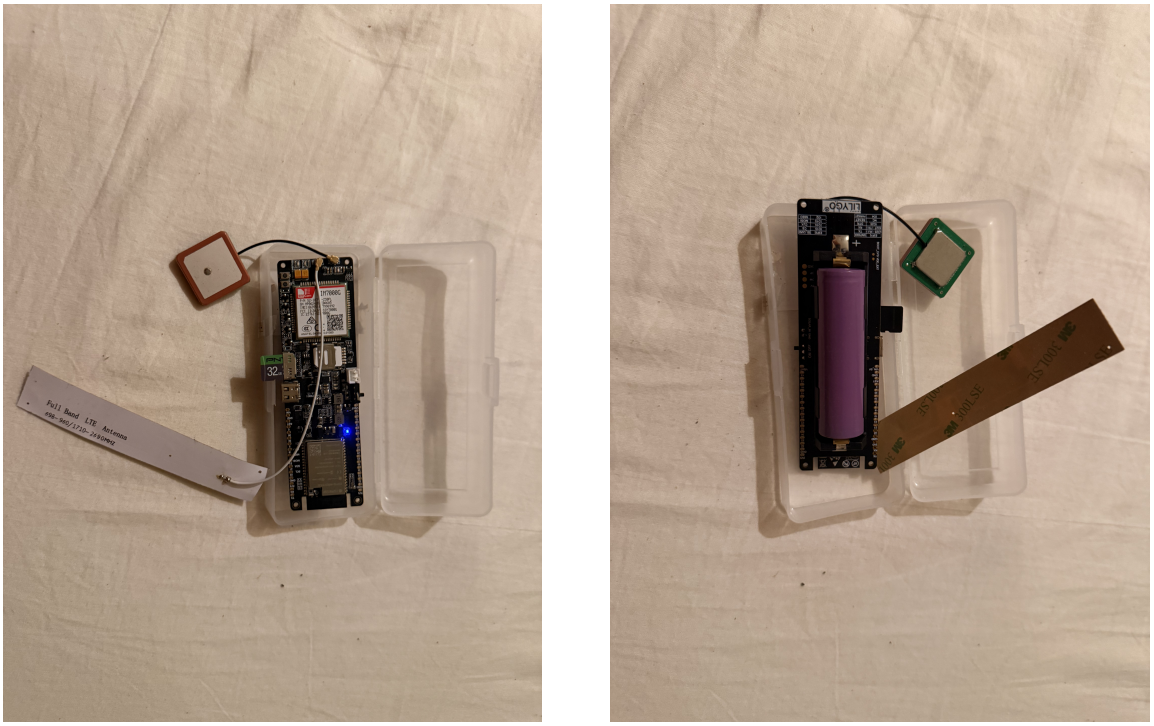


Figura 4.8. – Montagem do protótipo em caixa transparente: vista da placa e das antenas à esquerda e da célula 18650 à direita.

4.6 Desvios Face à Especificação

A implementação divergiu da especificação inicial em seis pontos, todos documentados e justificados.

Intervalo do ciclo. O intervalo de repouso configurado foi reduzido de 60 para 20 segundos, na sequência da mitigação do conflito de coexistência entre o GNSS e o rádio celular. Este parâmetro não representa o ciclo completo: o ensaio de campo registou intervalos de 83 segundos em aquisição degradada. As consequências para a latência, a autonomia e as projeções de consumo de dados são analisadas no Capítulo 5.

Revisão da autonomia mínima. O RNF01 previa inicialmente 24 horas de autonomia. A decisão de manter o modem e o recetor GNSS ligados entre ciclos, necessária para reduzir a latência de aquisição, levou à revisão formal do critério para 6 horas, conforme registado na Tabela 3.2 e justificado na Secção 6.4.

Cercas múltiplas e poligonais. A especificação previa uma única cerca circular por dispositivo. A implementação suporta múltiplas zonas, circulares ou poligonais, com semântica de segurança por união.

Posicionamento em interior. Não previsto na especificação inicial, foi acrescentado por se ter verificado que o sistema ficava cego precisamente no ambiente onde a pessoa monitorizada passa mais tempo.

Validação do certificado do servidor. A especificação assumia HTTPS com validação completa. O defeito de firmware do módulo SIM7000G, documentado na Secção 4.2.6, torna essa validação tecnicamente inatingível neste hardware. O canal permanece cifrado, mas não autenticado quanto ao servidor.

Canal de notificação. O *push* nativo em iOS exige uma chave APNs associada a uma conta paga. Foi implementado um canal alternativo via Telegram, mantendo o canal FCM integralmente funcional para quando essa restrição for removida. Pelo mesmo motivo — a alteração do identificador de *bundle* da aplicação para uma equipa de programador pessoal — a autenticação por conta Google deixou de estar operacional, uma vez que o cliente OAuth se encontra associado ao identificador anterior.

5 Testes e Validação

Este capítulo apresenta a evidência de validação do sistema. Nas tabelas, NR significa não registado individualmente; NM, não medido; NT, não testado; e NV, não validado. Estas indicações não representam resultados favoráveis. Nos requisitos funcionais, a indicação qualitativa remete para os casos de teste documentados, sem pressupor uma medição numérica. Os resultados são separados da sua interpretação: neste capítulo registam-se protocolos, condições, medições e veredictos; o significado dos resultados e os compromissos de engenharia são discutidos no Capítulo 6.

5.1 Estratégia de Testes

A validação do sistema foi organizada em quatro níveis, correspondentes aos ramos ascendentes do Modelo em V adotado no Capítulo 3.

Ao nível unitário foi verificada a lógica pura do *backend* e da aplicação. Ao nível de integração foram recolhidas evidências de interação com o Firestore. Ao nível de sistema foi definido um protocolo ponta a ponta, mas falta o registo completo dos instantes até à receção do alerta de saída. Os ensaios de campo acrescentam observações reais, sem concluir a validação de todos os requisitos não funcionais.

Foram utilizados três ambientes distintos, cuja distinção é relevante para a interpretação dos resultados: o emulador local do Firestore, para ensaios determinísticos sem custo nem efeitos laterais; o projeto Firebase real, na região `europe-west1`, para ensaios ponta a ponta; e o hardware físico com cartão Subscriber Identity Module (SIM) ativo, para os ensaios de campo. O simulador de dispositivo em Node.js permite injetar telemetria sintética no *endpoint* real, o que possibilita exercitar o *backend* e a aplicação sem depender da disponibilidade do hardware.

As ferramentas de execução foram o executor de testes nativo do Node.js (`node --test`), o executor de testes do Flutter (`flutter test`), a análise estática (`flutter analyze`), o monitor série do PlatformIO e a consola do Firestore.

5.2 Testes Unitários Automatizados

A suite de testes automatizados do *backend* é constituída por 43 testes de lógica pura, executados sem emulador nem acesso à rede. Na execução de verificação realizada em 6 de setembro de 2026, a totalidade dos testes foi aprovada, sem falhas, em 40,7 milissegundos. A Tabela 5.1 apresenta a distribuição por ficheiro de teste, a duração observada e a lógica coberta.

Tabela 5.1. – Distribuição dos testes unitários do *backend* por ficheiro.

Ficheiro de teste	Testes	Duração	Lógica coberta
`geo.test.js`	13	27,8 ms	Pertença a zonas circulares e poligonais; distância à fronteira; pontos sobre a fronteira; polígonos côncavos; coordenadas extremas; equivalência com o algoritmo da aplicação
`health.test.js`	17	28,4 ms	Deteção de silêncio prolongado; histerese do alerta de bateria (limiares de 20 % e 30 %); emissão apenas na transição de estado; exclusão de dispositivos sem contacto anterior; formatação da duração do silêncio
`indoor.test.js`	13	32,0 ms	Validação e normalização de endereços MAC; rejeição de entradas malformadas; tratamento de listas vazias ou não conformes; número mínimo de pontos de acesso exigido pela trilateração
Total (`npm test`)	**43**	**40,7 ms**	**Todos aprovados; 0 falhas**

A escolha de isolar a lógica pura em módulos próprios — em vez de a manter embebida nos manipuladores das Cloud Functions — foi motivada precisamente pela testabilidade: nenhum destes 43 testes necessita de emulador, credenciais ou rede, pelo que a suite completa é executada em menos de um décimo de segundo e integra o fluxo de integração contínua sem custo apreciável.

A aplicação dispõe de quatro ficheiros de testes, incidindo sobre o serviço de dispositivo, os utilitários geométricos, o modelo de zona e um componente de interface. A

execução com Flutter 3.44.7 e Dart 3.12.2 aprovou os 24 testes em 2 segundos, conforme discriminado na Tabela 5.2.

Tabela 5.2. – Distribuição dos testes unitários da aplicação por ficheiro.

Ficheiro de teste	Testes	Duração	Lógica coberta
`device_service_test.dart`	12	NR	Serviço de dispositivo e interação com os dados persistidos
`geo_utils_test.dart`	7	NR	Decisão geométrica dentro/fora utilizada pela aplicação
`zone_test.dart`	4	NR	Modelo e validação das zonas seguras
`widget_test.dart`	1	NR	Componente de interface
Total (`flutter test`)	**24**	**2 s**	**Todos aprovados; 0 falhas**

A análise estática, executada através de `flutter analyze lib`, terminou em 1,1 segundos com a indicação *No issues found*, sem avisos nem erros.

Casos não cobertos. A cobertura incide sobre lógica determinística. Não são cobertos por testes automatizados o firmware, cuja verificação exige hardware, nem os caminhos que dependem de serviços externos — envio efetivo de notificações, resolução de posições pela Geolocation API e autenticação —, validados por ensaio manual.

5.3 Testes de Integração Ponta-a-Ponta

O protocolo de ensaio integrado destina-se a percorrer o sistema completo com relógios sincronizados. A recolha completa dos instantes não foi realizada; a Tabela 5.3 apresenta as etapas e os suportes de evidência previstos, não uma execução integral validada.

Protocolo. (i) Colocar o dispositivo dentro de uma zona ativa e confirmar, no monitor série, um ciclo com envio bem-sucedido; (ii) confirmar a criação do documento de telemetria na consola do Firestore; (iii) confirmar a invocação da função de processamento nos registos; (iv) atravessar a fronteira da zona; (v) confirmar a criação do documento de alerta; (vi) registar o instante de receção da notificação no telemóvel.

Para ensaios determinísticos sem hardware, o simulador permite injetar telemetria com coordenadas controladas, reproduzindo o cenário de evasão de forma repetível.

Tabela 5.3. – Protocolo de integração ponta a ponta e estado dos registos.

#	Etapa	Instante	Suporte previsto
1	Leitura adquirida no dispositivo	NR	Monitor série
2	Envio HTTPS concluído (200)	NR	Monitor série
3	Documento de telemetria criado	NR	Consola Firestore
4	Função de processamento invocada	NR	Registos Firebase
5	Alerta persistido	NR	Consola Firestore
6	Notificação recebida	NR	Captura de ecrã

5.4 Testes Funcionais por Caso de Uso

A validação de cada caso de uso está organizada em casos de teste positivos e negativos, registados segundo um formato comum: identificador, condição ou passos, resultado esperado, resultado obtido e veredicto. Apenas as linhas com evidência de execução apresentam resultado obtido e veredicto; os casos não testados são identificados explicitamente. Os casos negativos foram incluídos deliberadamente, uma vez que o comportamento do sistema perante entradas inválidas é tão relevante para a segurança do cuidador como o comportamento perante entradas válidas.

Os resultados apresentados nas linhas preenchidas das Tabelas 5.8 e 5.9 foram obtidos em 6 de setembro de 2026 contra o sistema em produção, utilizando um dispositivo dedicado aos ensaios.

5.4.1 UC-01 — Registar e Efetuar Login

Estes testes verificam o ciclo de autenticação, desde a criação da conta até à recuperação de acesso. O registo com dados válidos criou uma conta e iniciou a sessão; os casos de correio eletrónico já existente, palavra-passe fraca e credenciais incorretas não foram testados e permanecem identificados como não testados na Tabela 5.4.

O fluxo completo de recuperação foi verificado em 6 de setembro de 2026: abertura do ecrã de recuperação, confirmação *Check your inbox*, receção da mensagem, acesso à página de reposição, confirmação *Password changed* e início de uma nova sessão com a palavra-passe alterada. A confirmação apresenta a formulação “se existir uma conta para este endereço, receberá uma ligação”, que não confirma nem desmente a existência da

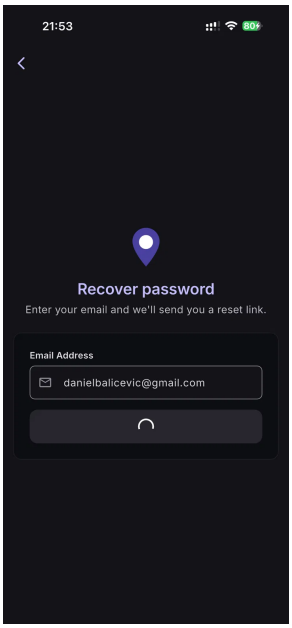
conta. Esta resposta constitui uma proteção contra enumeração de utilizadores registados, coerente com o princípio de não revelar qual das credenciais falhou, enunciado no resultado esperado de TF-01.4; esse caso específico permanece por testar. O controlo de acesso aos dados é avaliado separadamente na Secção 5.5.7.

Foi observada uma limitação na entrega e apresentação da mensagem de reposição: o correio foi recebido na pasta de spam, com o identificador interno do projeto Firebase como remetente e assunto, em vez do nome da aplicação. O modelo de correio e o nome público do projeto mantêm os valores por omissão na consola Firebase, sem personalização. A ligação de reposição funcionou corretamente, mas a página de reposição foi apresentada em inglês, no domínio por omissão do Firebase, e as capturas do fluxo da aplicação também incluem texto em inglês. A personalização destes elementos e a configuração de um domínio de envio próprio são remetidas para a Secção 6.7, como medidas destinadas a melhorar a identificação da aplicação e a reduzir a probabilidade de classificação como spam.

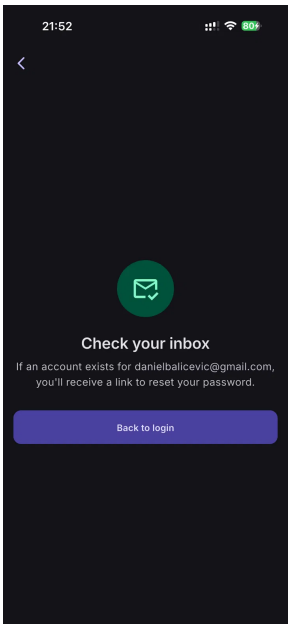
Tabela 5.4. – Testes funcionais do caso de uso UC-01.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-01.1	Registo com dados válidos	Conta criada e sessão iniciada	Conta criada e sessão iniciada	Passou
TF-01.2	Registo com correio eletrónico já existente	Mensagem de conta duplicada	NT	NV
TF-01.3	Palavra-passe fraca	Registo recusado com indicação do critério	NT	NV
TF-01.4	Credenciais incorretas	Autenticação recusada, sem revelar qual o campo errado	NT	NV
TF-01.5	Pedido de recuperação de palavra-passe	Mensagem de reposição enviada	Fluxo completo verificado; palavra-passe alterada e nova sessão iniciada	Passou

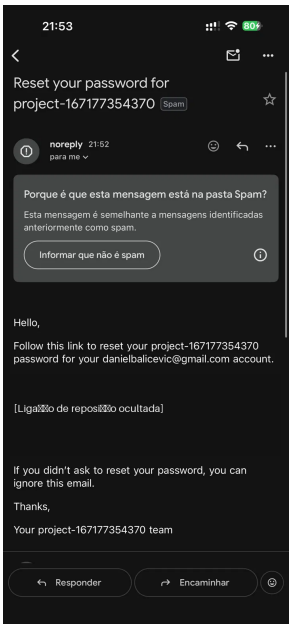
As cinco etapas documentadas pelas capturas são apresentadas na Figura 5.1. As capturas evidenciam o pedido e a confirmação de envio, a mensagem recebida na pasta de spam, a página de reposição e a confirmação da alteração; o início de uma nova sessão foi verificado no ensaio, sem captura anexada dessa etapa. A ligação de reposição foi ocultada na cópia da mensagem destinada ao relatório.



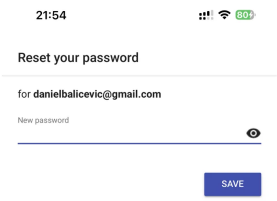
(a) Pedido de recuperação.



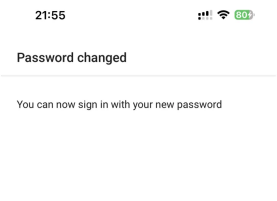
(b) Confirmação de envio.



(c) Mensagem recebida na pasta de spam.



(d) Página de reposição.





(e) Confirmação da alteração.

Figura 5.1. – Evidências do fluxo de recuperação de palavra-passe (TF-01.5), verificado em 6 de setembro de 2026. A ligação de reposição foi ocultada.

5.4.2 UC-02 — Emparelhar Dispositivo

O caso negativo TF-02.2 revelou, em 6 de setembro de 2026, um defeito nas regras do Firestore: a escrita sobre um identificador inexistente era avaliada como criação e permitia criar um dispositivo sem código de emparelhamento. A correção, aplicada e implantada no mesmo dia, passou a negar a criação de documentos de dispositivo pelo cliente. A Tabela 5.5 apresenta os resultados após correção, incluindo a verificação de não regressão do emparelhamento legítimo em TF-02.6. Este é um exemplo concreto da importância dos casos negativos: um teste positivo com um dispositivo previamente provisionado nunca teria revelado este defeito. A causa e a correção são descritas na Secção 4.3.7.

A evidência de TF-02.1 e TF-02.3 provém do ensaio de segurança de 6 de setembro de 2026, apresentado na Tabela 5.15: a reivindicação com código correto devolveu HTTP 200 e a reivindicação com código errado devolveu HTTP 403. As verificações foram conduzidas a partir de um cliente externo autenticado por Firebase Authentication, com uma conta descartável criada e eliminada durante o ensaio. Foram exercitadas as regras do servidor, e não a interface da aplicação; relativamente à imposição do controlo de acesso, esta verificação é mais forte do que um ensaio apenas pela aplicação, pois confirma que a recusa é imposta pelo servidor.

O caso TF-02.5 complementa essa avaliação: o dispositivo de ensaio `device-teste-01` foi provisionado sem o campo `pairingCode` e não é reclamável pelo fluxo atual. Trata-se de uma limitação de provisionamento no fluxo protegido pelas regras do Firestore, não apenas de uma restrição visual da aplicação. A Tabela 5.15 fornece o enquadramento da verificação das regras, mas não contém uma linha específica para a ausência de código; não se atribui, por isso, a este caso um código HTTP não registado.

5.4.3 UC-03 — Visualizar Mapa em Tempo Real

Todos os casos da Tabela 5.6 foram verificados na aplicação em 6 de setembro de 2026, com resultados conformes ao esperado. A evidência abrange a atualização do mapa, os estados sem ligação ou sem telemetria recente, a apresentação da origem Wi-Fi, as zonas ativas e a consulta do histórico.

Tabela 5.5. – Testes funcionais do caso de uso UC-02.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-02.1	Identificador e código corretos	Dispositivo reclamado e associado ao cuidador	HTTP 200 (Tabela 5.15, controlo de reivindicação)	Passou
TF-02.2	Identificador inexistente	Emparelhamento recusado	HTTP 403, *permission denied*	Passou
TF-02.3	Código de emparelhamento incorreto	Recusado pelas regras do Firestore	HTTP 403 (Tabela 5.15)	Passou
TF-02.4	Dispositivo já emparelhado por outro cuidador	Recusado; o dispositivo mantém o proprietário anterior	HTTP 403, *permission denied*	Passou
TF-02.4b	(Controlo) O proprietário anterior manteve-se	Dispositivo inacessível à conta atacante	Leitura recusada, HTTP 403	Passou
TF-02.5	Dispositivo sem código definido	Não reclamável; comportamento documentado como limitação	`device-teste-01` provisionado sem `pairingCode`	Passou
TF-02.6	Desfazer emparelhamento e voltar a emparelhar	Dispositivo libertado e reclamável de novo	1.ª reivindicação 200; libertação 200; 2.ª reivindicação 200	Passou

5.4.4 UC-04 — Configurar Cerca Virtual

Estes casos verificam a criação e a gestão das zonas circulares e poligonais utilizadas pelo geofencing. As entradas inválidas foram incluídas para confirmar que geometrias incompletas e valores fora dos limites não são persistidos como configurações utilizáveis. Todos os casos da Tabela 5.7 foram verificados na aplicação em 6 de setembro de 2026, com resultados conformes ao esperado.

O caso TF-04.8, relativo à remoção de todas as zonas sem recriação automática na abertura seguinte, constitui um teste de não regressão do defeito de migração corrigido durante o desenvolvimento. A salvaguarda que impede a recriação da zona antiga encontra-se descrita na Secção 4.3.1 e aplica-se à gestão de zonas apresentada na Secção 4.4.5; distingue-se do defeito de criação de documentos novos também descrito nessa última secção.

Tabela 5.6. – Testes funcionais do caso de uso UC-03.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-03.1	Chegada de nova telemetria	Marcador atualizado sem intervenção do utilizador	O marcador foi atualizado sem intervenção do utilizador	Passou
TF-03.2	Telemóvel sem ligação à Internet	Estado de erro explícito, sem terminação abrupta	Foi apresentado um estado de erro explícito, sem terminação abrupta	Passou
TF-03.3	Sem telemetria há mais de cinco minutos	Marcador cinzento e aviso de ausência de sinal	O marcador ficou cinzento e foi apresentado um aviso de ausência de sinal	Passou
TF-03.4	Posição de origem Wi-Fi	Posição apresentada com a precisão associada	A posição foi apresentada com a precisão associada	Passou
TF-03.5	Zonas ativas configuradas	Zonas desenhadas sobre o mapa	As zonas foram desenhadas sobre o mapa	Passou
TF-03.6	Consulta do histórico do período	Trilho e lista coerentes com a telemetria	O trilho e a lista apresentaram-se coerentes com a telemetria	Passou

Tabela 5.7. – Testes funcionais do caso de uso UC-04.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-04.1	Criação de zona circular	Zona persistida e visível no mapa	A zona foi persistida e ficou visível no mapa	Passou
TF-04.2	Criação de zona poligonal	Zona persistida com os vértices definidos	A zona foi persistida com os vértices definidos	Passou
TF-04.3	Alteração do raio de uma zona	Alteração refletida na aplicação e no *backend*	A alteração foi refletida na aplicação e no *backend*	Passou
TF-04.4	Remoção de uma zona	Zona eliminada, sem recriação automática	A zona foi eliminada, sem recriação automática	Passou
TF-04.5	Raio abaixo ou acima dos limites	Valor limitado ao intervalo admissível	O valor foi limitado ao intervalo admissível	Passou
TF-04.6	Polígono com menos de três vértices	Criação recusada	A criação foi recusada	Passou
TF-04.7	Desativação de uma zona	Zona ignorada na avaliação de geofencing	A zona foi ignorada na avaliação de geofencing	Passou
TF-04.8	Remoção de todas as zonas	Nenhuma zona é recriada na abertura seguinte	Nenhuma zona foi recriada na abertura seguinte	Passou

5.4.5 UC-05 — Receber Alerta de Emergência

O conjunto de testes acompanha o alerta desde a deteção da saída de zona até à sua apresentação e resolução. A evidência de campo acrescentada a TF-05.1 e TF-05.7 confirma a sequência dentro/fora/regresso e a notificação de regresso; não comprova uma notificação de saída, mantendo-se TF-05.2 e TF-05.3 sem resultado documentado. Os casos negativos e de fronteira verificam a supressão de leituras isoladas, a ausência de duplicados e o tratamento de posições classificadas como incertas.

5.4.6 UC-06 — Consultar Nível de Bateria

Estes testes verificam a apresentação do nível de bateria e os alertas associados à saúde do dispositivo. Os casos em torno dos limiares foram incluídos para avaliar a histerese e impedir que pequenas oscilações originem notificações repetidas.

Os tipos de alerta observados nestes ensaios correspondem aos identificadores `geofence_exit`, `low_battery` e `offline`.

5.5 Validação dos Requisitos Não Funcionais Quantificados

5.5.1 Protocolo Experimental Comum

O ensaio de campo foi realizado na tarde de 6 de setembro de 2026, em deslocação pedonal no exterior, na zona costeira da Praia da Galé, em Albufeira. Não foi utilizado qualquer equipamento ou ponto de referência independente de posição. Esta é a limitação central do ensaio: apenas permite caracterizar a dispersão de leituras sucessivas, não a sua exatidão.

Os instantes foram obtidos do relógio do telemóvel e dos carimbos temporais registados pela aplicação. A sincronização entre estes relógios e o dispositivo ou o servidor não foi validada experimentalmente; não foi quantificado o respetivo desvio. Por isso, os intervalos

Tabela 5.8. – Testes funcionais do caso de uso UC-05.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-05.1	Saída de zona observada na aplicação	Estado fora de todas as zonas; aviso de saída a verificar	Dentro às 14:11 (79 %); fora às 14:13 (78 %) e 14:16 (79 %). Sem captura do aviso de saída	Parcial
TF-05.2	Saída de zona com a aplicação em segundo plano	Notificação do sistema	NT	NV
TF-05.3	Saída de zona com a aplicação fechada	Notificação entregue pelo canal disponível	NT	NV
TF-05.4	Uma única leitura fora da zona	Nenhum alerta gerado (*debounce*)	0 alertas; `outsideCount=1`	Passou
TF-05.5	Duas leituras consecutivas fora	Alerta gerado uma única vez	1 alerta `geofence_exit`, a 599 m	Passou
TF-05.6	Permanência fora da zona	Nenhum alerta repetido	Mantém 1 alerta após 4 leituras	Passou
TF-05.7	Regresso à zona	Regresso assinalado; resolução do alerta de evasão	Às 14:17, dentro de "posto 1", 78 %; notificação Telegram de regresso. Resolução já verificada no ensaio anterior	Passou
TF-05.8	Leitura GNSS com precisão declarada pelo cliente	Precisão normalizada segundo o contrato de ingestão	`accuracy=0`; alargamento apenas para Wi-Fi	Passou
TF-05.9	Marcação manual do alerta como resolvido	Estado atualizado na lista	NT	NV

Tabela 5.9. – Testes funcionais do caso de uso UC-06.

ID	Condição / passos	Resultado esperado	Resultado obtido	Ver.
TF-06.1	Leitura de bateria em nível normal	Valor apresentado sem alerta	NT	NV
TF-06.2	Leitura igual ou inferior a 20 %	Alerta de bateria fraca gerado uma vez	1 alerta `low_battery`	Passou
TF-06.3	Oscilação entre 19 % e 21 %	Nenhum alerta repetido (histerese)	0 alertas novos	Passou
TF-06.4	Recuperação acima de 30 %	Alerta rearmado e resolvido	Alerta rearmado e resolvido	Passou
TF-06.5	Sem comunicação há mais de 15 minutos	Alerta de silêncio com a última posição conhecida	Alerta `offline`: 27 min de silêncio; detetado em 96 s	Passou
TF-06.6	Retoma da comunicação	Alerta de silêncio resolvido de imediato	Resolvido 17 s depois	Passou

entre pontos não constituem uma medição da latência ponta-a-ponta entre a travessia física e a notificação.

Nenhuma amostra foi excluída. O subconjunto de quatro leituras com o dispositivo aparentemente parado é apresentado separadamente da leitura atípica após reconexão, situada a 101 m do centroide, que permanece reportada. As estatísticas disponíveis limitam-se à dispersão fornecida para esse subconjunto; não foram obtidos percentis de erro ou de latência de campo. As consequências da ausência de referência, da amostra reduzida e da ausência de cegagem são discutidas na Secção 5.9.

5.5.2 RNF01 — Autonomia Energética

Critério. Autonomia mínima de 6 horas com intervalo de repouso configurado de 20 segundos; a duração efetiva do ciclo depende da aquisição e da transmissão.

Previsão analítica. Com base nos valores de consumo de folha de dados dos componentes e na configuração efetivamente implementada — modem e GNSS mantidos ligados entre ciclos — a estimativa central situa-se em aproximadamente 1,8 dias, com um intervalo

plausível entre 1,2 e 3,0 dias para uma célula de 3000 mAh. A componente dominante do consumo é o recetor GNSS em *tracking* contínuo, e não os picos de transmissão.

Esta estimativa não resulta de medição da corrente da placa completa. O ensaio disponível começou com 82 % de bateria e terminou com 27 %, após 5 h 28 min. A extrapolação de cerca de 10 h é aproximadamente quatro vezes inferior à previsão central de 43,2 h. O modelo analítico não reproduz, portanto, o comportamento observado; sem medição de corrente e dada a oscilação da percentagem reportada, não é possível atribuir quantitativamente a discrepância a um componente.

Protocolo previsto para validação completa, não executado integralmente. Carregar a célula a 100 %; operar o dispositivo exclusivamente com alimentação por bateria, sem cabo USB ligado a um computador, que falsearia a medição; deixar funcionar até à paragem por descarga; retirar o cartão MicroSD e processar o registo `/log/energy.csv` com o utilitário de análise incluído no repositório, que calcula a taxa de descarga em pontos percentuais por hora e a autonomia extrapolada.

Tabela 5.10. – Resultados da validação do requisito RNF01 — autonomia energética.

Métrica	**Valor medido**	**Critério / estado**
Duração total do ensaio	5 h 28 min	—
Bateria no início	82 %	—
Bateria no fim	27 %	—
Descarga total	55 pontos percentuais	—
Taxa média de descarga	Aproximadamente 10 %/h	—
Autonomia extrapolada a partir de carga completa	Aproximadamente 10 h	≥ 6 h
Resultado	**Parcialmente validado**	Extrapolação, não descarga completa

A extrapolação anterior é preservada como registo, mas não comprova seis horas de autonomia: o ensaio documentado durou 5 h 28 min e não foi uma descarga completa. A amplitude das oscilações de bateria observada no ensaio de campo fragiliza a inferência a partir da percentagem reportada. Mantém-se o critério de seis horas, mas o seu cumprimento não está validado experimentalmente. A classificação de **parcialmente validado** reconhece a evidência direta de operação durante 5 h 28 min com redução de 55 pontos percentuais da carga reportada e a projeção de autonomia superior a seis horas a partir de carga completa; não converte essa extrapolação numa observação direta de seis horas de funcionamento.

5.5.3 RNF02 — Precisão de Localização

Critério. O percentil 95 do erro das leituras GNSS válidas deve ser inferior a 10 metros.

Ensaio realizado. Foram consideradas quatro leituras consecutivas com o dispositivo aparentemente parado, em exterior costeiro: às 14:24:05, 14:25:28, 14:26:51 e 14:28:14, respetivamente nas coordenadas 37.08838/-8.32520, 37.08841/-8.32518, 37.08843/-8.32517 e 37.08844/-8.32517. O centroide reportado é 37.08842, -8.32518.

O ensaio mede a dispersão de leituras sucessivas no mesmo ponto, e não o erro face a uma posição de referência. O critério do RNF02, formulado como erro inferior a 10 m, não foi validado por não ter havido referência independente. A dispersão observada é compatível com esse limite, mas não o demonstra.

Tabela 5.11. – Dispersão observada e limites da validação do RNF02.

Métrica	**Resultado**
Leituras no subconjunto aparentemente estacionário	$n = 4$
Distância média ao centroide	2,4 m
Distância máxima ao centroide	4,3 m
Amplitude total entre extremos	7,2 m
Leitura atípica após reconexão, às 14:32:37	37.08916/-8.32584; a 101 m do centroide
Erro face a referência independente	Não validado experimentalmente: referência inexistente
Veredicto do RNF02	**Não validado**

A leitura atípica é reportada separadamente, sem ser omitida ou eliminada do ensaio. Não foi confirmada a imobilidade nesse instante, pelo que os 101 m não podem ser interpretados como erro de posição. Os restantes ambientes previstos e o percentil 95 do erro não foram validados experimentalmente, por ausência de uma campanha com referência independente.

5.5.4 RNF03 — Latência de Alerta

Critério. A latência entre a travessia física da fronteira e a receção da notificação deve ser inferior a 30 segundos.

Previsão analítica. Para um intervalo T entre leituras consecutivas, a espera até à primeira leitura após a travessia situa-se entre zero e T. A confirmação por uma segunda leitura fora da zona acrescenta outro intervalo:

$$L_{\text{espera+debounce}} \in [T, 2T]. \tag{5.1}$$

O cenário inicial assume um ciclo de 23 s, associado ao repouso configurado de 20 s. Nesse cenário, a espera e a confirmação por segunda leitura somam entre 23 e 46 s; não se estima a latência total sem quantificar os restantes termos. Com o intervalo observado de 83 s, os dois termos estruturais passam a 83 a 166 s. Este segundo intervalo é uma previsão condicionada à repetição de ciclos de 83 s, não uma latência de notificação medida nem um limite superior global.

Tabela 5.12. – Cenários de previsão da latência de alerta (RNF03).

Componente	**Cenário configurado**	**Cenário observado em campo**
Repouso configurado (`INTERVAL_S`)	20 s	20 s
Intervalo entre leituras usado no modelo	23 s (hipótese anterior)	83 s (observado)
Espera pelo ciclo seguinte	0 a 23 s	0 a 83 s
Confirmação por segunda leitura fora	23 s	83 s
Soma: espera e *debounce*	23 a 46 s	83 a 166 s
Latência total prevista	Não quantificada	Não quantificada
Latência de notificação medida	Não validada experimentalmente	Não validada experimentalmente
Veredicto analítico	**Não cumprido**	**Não cumprido**

Os intervalos entre pontos já refletem a operação do dispositivo; somar-lhes novamente toda a aquisição produziria dupla contagem. Os tempos residuais de transporte, processamento e entrega não foram medidos isoladamente. Não foram registados os instantes necessários para medir a latência entre a travessia e uma notificação de saída; não se apresenta, por isso, um intervalo numérico de latência total medida.

O RNF03 permanece **não cumprido na previsão analítica**, agora com margem substancialmente superior no cenário de aquisição degradada. A remoção da SIM produziu ainda uma interrupção distinta, de 263 s entre pontos, que não é utilizada como ciclo nominal neste cálculo.

5.5.5 RNF04 — Disponibilidade de Rede e Fallback

Ensaio realizado. A SIM foi removida por volta das 14:22. Entre os pontos das 14:28:14 e das 14:32:37 registou-se um intervalo de 263 s, seguido de outro ponto às 14:32:58, 21 s depois. Esta sequência é compatível com acumulação e escoamento da fila do dispositivo no MicroSD, mas não comprova esse mecanismo sem registos de enfileiramento e confirmação de reenvio.

O instante exato de reposição da SIM não foi registado. Assim, os 263 s quantificam a ausência de pontos entre os dois carimbos, não a duração exata da remoção nem o tempo de reconexão contado desde a reposição. As quantidades enfileiradas e recuperadas, a taxa de recuperação e a preservação da ordem não foram validadas experimentalmente, por não terem sido contadas ou auditadas; as respetivas células indicam a ausência de medição ou validação.

Tabela 5.13. – Resultados da validação do requisito RNF04.

Métrica	Valor medido	Veredicto
Duração da interrupção observada entre pontos	263 s	Parcial
Leituras geradas durante a interrupção	NM	—
Leituras enfileiradas no MicroSD	NM	—
Leituras recuperadas após restabelecimento	NM	NV
Taxa de recuperação	NM	100 % esperado
Ordem cronológica preservada	NM	NV
Restabelecimento observado	14:32:37, após 263 s sem pontos	Tempo desde reposição da SIM não medido

5.5.6 RNF05 e RNF08 — Usabilidade e Desempenho da Interface

Protocolo de usabilidade previsto, sem resultados disponíveis. Avaliação exploratória com um mínimo de três participantes sem conhecimento prévio do sistema, executando quatro tarefas: autenticar-se, emparelhar um dispositivo, criar uma zona segura e interpretar um alerta recebido. São registados o tempo de execução, o número de erros e as dificuldades observadas.

Tabela 5.14. – Resultados da avaliação exploratória de usabilidade (RNF05).

Tarefa	P1	P2	P3	Dificuldades observadas
Registo e autenticação	NT	NT	NT	NT
Emparelhamento	NT	NT	NT	NT
Criação de zona segura	NT	NT	NT	NT
Interpretação de alerta	NT	NT	NT	NT

Não foram disponibilizados resultados de participantes. Este protocolo permanece pendente e não sustenta um veredicto de usabilidade.

Desempenho da interface (RNF08). Não foi medida a percentagem de fotogramas acima de 16,7 ms. O protocolo previsto consiste na execução da aplicação em modo *profile*, com as ferramentas de desenvolvimento do Flutter, durante interação contínua com o mapa, registando a percentagem de fotogramas com tempo de composição superior a 16,7 ms.

5.5.7 RNF06 — Segurança dos Dados

Protocolo. As verificações foram executadas em 6 de setembro de 2026 a partir de um cliente externo autenticado por Firebase Auth, sem recurso ao Admin SDK, que ignora as regras e não permitiria validá-las. Foi utilizada uma conta descartável no papel de segundo cuidador, eliminada no final do ensaio. O veredicto separa explicitamente o controlo de acesso da confidencialidade em trânsito, admitindo cumprimento parcial.

As onze verificações ativas produziram o resultado esperado. Os controlos positivos são necessários porque uma tabela composta apenas por recusas não permitiria distinguir regras de segurança corretas de um sistema que negasse indiscriminadamente todos os pedidos. A restrição da chave da aplicação ao *bundle* iOS constitui uma medida antiabuso de quota e não um mecanismo de controlo de acesso aos dados.

O último registo da tabela corresponde a uma limitação conhecida e documentada: o canal é cifrado, protegendo contra interceção passiva, mas o certificado do servidor não é validado, pelo que a autenticação do servidor não é assegurada. O veredicto deste requisito é, por conseguinte, de **cumprimento parcial**, sendo a análise de risco correspondente apresentada na Secção 6.3.

Tabela 5.15. – Resultados da validação do requisito RNF06 — segurança dos dados.

Teste	Resultado esperado	Obtido
Leitura do Firestore sem autenticação	Negada pelas regras	403, *permission denied*
Leitura de dispositivo de outro cuidador	Negada pelas regras	403, *permission denied*
Leitura do próprio dispositivo (controlo)	Permitida pelas regras	200
Escrita direta em `telemetry` por cliente	Negada pelas regras	403, *permission denied*
Criação direta de alerta por cliente	Negada pelas regras	403, *permission denied*
Ingestão sem cabeçalho de chave	Resposta 401	401
Ingestão com chave inválida	Resposta 401	401
Ingestão com chave válida (controlo)	Resposta 200	200
Reivindicação com código de emparelhamento errado	Negada pelas regras	403
Reivindicação com código correto (controlo)	Permitida pelas regras	200
Ingestão através de `GET`	Resposta 405	405
Protocolo do troço dispositivo–nuvem	TLS 1.2 cifrado, servidor não autenticado	Confirmado no firmware

5.5.8 RNF07 — Consumo de Dados

Critério. Consumo mensal estimado inferior a 100 MB.

Análise. O *payload* de telemetria GNSS tem cerca de 116 bytes; somando os cabeçalhos do pedido e a resposta, o tráfego aplicacional por ciclo situa-se em cerca de 460 bytes, já com margem para o encapsulamento Transmission Control Protocol (TCP)/Internet Protocol (IP). Contudo, a implementação estabelece **uma ligação TLS nova em cada ciclo**: o dispositivo entra em *deep sleep* entre ciclos e a sequência `SHCONN`/`SHDISC` encerra a ligação após cada envio. O *handshake* TLS — que inclui a cadeia de certificados do servidor — domina, por isso, o tráfego real.

A Tabela 5.16 apresenta a projeção para os dois cenários e para os dois intervalos de ciclo relevantes.

A projeção indica incumprimento do requisito de 100 MB no cenário temporal assumido, mas não é uma medição de consumo mensal. Os intervalos de 83 s observados em campo invalidam a interpretação de 20 s como periodicidade constante. Sem a distribuição

Tabela 5.16. – Projeção do consumo mensal de dados (RNF07).

Intervalo	Cenário	Bytes/ciclo	Ciclos/dia	Consumo/mês
60 s (especificado)	Sem *handshake* por ciclo (hipotético)	aprox. 460	aprox. 1370	aprox. 19 MB
60 s (especificado)	Com *handshake* TLS por ciclo	aprox. 5400	aprox. 1370	aprox. 223 MB
20 s (repouso configurado)	Sem *handshake* por ciclo (hipotético)	aprox. 460	aprox. 3760	aprox. 52 MB
20 s (repouso configurado)	Com *handshake* TLS por ciclo	aprox. 5400	aprox. 3760	aprox. 610 MB

temporal dos ciclos e a contabilização do tráfego efetivo, não se recalcula aqui o consumo mensal nem se extrapola o pequeno subconjunto de campo para todo o mês.

5.5.9 RNF09 — Robustez Ambiental

Na ausência de câmara climática, o requisito é declarado como **não validado experimentalmente**. É apresentada evidência documental indireta, baseada nas gamas de operação declaradas pelos fabricantes dos componentes, que não substitui o ensaio do protótipo completo e não permite emitir veredicto sobre o conjunto montado.

5.5.10 RNF10 — Escalabilidade

Protocolo. Em 6 de setembro de 2026 foi realizada a injeção paralela de telemetria para dez identificadores de dispositivo distintos, cada um com uma zona circular ativa de 200 m, ao ritmo sintético de um pedido a cada 20 segundos, durante 30 minutos. Este ritmo de injeção não representa os intervalos de 83 segundos posteriormente observados no dispositivo em campo. Os tempos apresentados correspondem ao *round-trip* observado pelo cliente e não ao tempo interno de execução da função.

Tabela 5.17. – Resultados da validação do requisito RNF10 — escalabilidade.

Configuração	Invocações	Tempo médio	Percentil 95	Erros
1 dispositivo (infraestrutura a frio)	30	290 ms	393 ms	0
10 dispositivos (infraestrutura quente)	900	126 ms	159 ms	0

A latência inferior observada com dez dispositivos não significa que o sistema escale melhor à medida que a carga aumenta. A linha de base de um dispositivo iniciou-se com a infraestrutura parada e incorporou o custo de instanciação, enquanto a carga contínua de dez dispositivos manteve as instâncias ativas durante 30 minutos e diluiu esse custo por 900 pedidos.

Este resultado valida experimentalmente a restrição R-06 da Tabela 3.3, que antecipava o impacto dos arranques a frio e o classificava como baixo. A diferença de cerca de 230 ms no percentil 95 é reduzida face ao orçamento de 30 segundos do RNF03, sustentando a classificação atribuída na especificação.

Veredicto. O RNF10 foi cumprido, sem erros nas 900 invocações realizadas com dez dispositivos.

5.6 Testes de Campo em Mobilidade Urbana Real

Ensaio realizado. O percurso pedonal decorreu em 6 de setembro de 2026, entre as 14:11 e as 14:33, na Praia da Galé, em Albufeira, em exterior aberto e passadiço. O ensaio não executou o protocolo anteriormente previsto de passagem por ambiente urbano denso e interior; esses ambientes continuam não validados experimentalmente nesta campanha.

As indicações de ausência de registo correspondem a grandezas não validadas experimentalmente: não foram medidos a extensão do percurso nem o total de ciclos, e não foram contabilizadas as tentativas de aquisição, as resoluções Wi-Fi ou as leituras recuperadas da fila. As condições meteorológicas também não foram registadas.

Os pontos consecutivos disponíveis delimitam os intervalos 14:24:05–14:25:28 (83 s), 14:25:28–14:26:51 (83 s), 14:26:51–14:28:14 (83 s), 14:28:14–14:32:37 (263 s, no episódio de remoção da SIM) e 14:32:37–14:32:58 (21 s). Estes registos fundamentam a revisão da previsão do RNF03 e a validação apenas parcial da recuperação de conectividade, sem demonstrar a recuperação integral da fila.

Tabela 5.18. – Resultados do ensaio de campo na Praia da Galé.

Métrica	Valor observado
Data e janela horária	6 de setembro de 2026, 14:11–14:33
Local e ambientes	Praia da Galé, Albufeira; percurso pedonal, exterior aberto e passadiço
Zonas configuradas	Duas: circular e poligonal, simultaneamente ativas e visíveis no mapa
Travessias de fronteira	Uma saída e um regresso, confirmados na aplicação
Notificação documentada	Um regresso a zona segura, entregue pelo Telegram
Interrupção induzida	SIM removida por volta das 14:22
Anomalia de posição	Leitura a 101 m do centroide após reconexão
Extensão do percurso	NM
Número total de ciclos	NM
Taxa de obtenção de *fix*	NM
Ciclos resolvidos por Wi-Fi	NM
Leituras recuperadas do MicroSD	NM

5.7 Testes de Robustez e Cenários de Falha

Os testes de robustez avaliam o comportamento do sistema quando componentes ou serviços deixam de responder nas condições previstas. Foram incluídas falhas de energia, armazenamento, comunicação e validação de dados para verificar se cada camada degrada de forma controlada, sem transformar uma falha localizada em perda silenciosa de informação.

As linhas preenchidas da Tabela 5.19 correspondem a ensaios executados em 6 de setembro de 2026 contra o sistema em produção.

Os registos do sistema em produção mostram ainda 17 execuções consecutivas da função agendada `checkDeviceHealth` entre as 05:43 e as 07:08 de 6 de setembro de 2026, com periodicidade de cinco minutos e sem qualquer falha. Esta evidência confirma a execução regular do mecanismo responsável pela deteção de silêncio prolongado durante a janela observada.

Tabela 5.19. – Cenários de falha induzida e comportamento observado.

Cenário induzido	Comportamento esperado	Comportamento observado
Perda de energia a meio de um ciclo	Retoma no ciclo seguinte, sem corrupção da fila	NT
Cartão MicroSD ausente	Ciclo prossegue; leitura não entregue é perdida com registo explícito	NT
Cartão MicroSD cheio	Gravação falha com registo; ciclo prossegue	NT
Modem não responsivo	Ciclo termina sem envio; leitura enfileirada	NT
Payload malformado enviado ao *endpoint*	Resposta 400, sem escrita no Firestore	Resposta 400; nenhuma escrita no Firestore
Token FCM inválido	Canal Telegram continua a notificar	NT
Remoção de zona com alerta ativo	Alerta permanece consultável; sem exceção	Alerta permaneceu consultável; sem exceção
Leitura sem carimbo temporal	A função de ingestão atribui o instante de chegada	Resposta 200; instante de chegada atribuído pela ingestão

5.8 Matriz Consolidada de Rastreabilidade

A matriz de rastreabilidade estabelece a ligação entre cada requisito, a secção que documenta a respetiva implementação, a secção que apresenta a sua validação, o valor medido quando aplicável e o veredicto final. A sua função é dupla: identificar os requisitos com validação ainda incompleta e evitar funcionalidades implementadas sem requisito de origem. O veredicto funcional reflete a evidência disponível para cada requisito e não implica que todos os casos de teste negativos do respetivo caso de uso tenham sido executados; estes são identificados como não testados nas tabelas anteriores.

5.9 Ameaças à Validade

Validade interna. Os ensaios foram conduzidos pelo próprio autor, sem cegagem e com conhecimento do comportamento esperado do sistema, o que constitui uma fonte potencial de enviesamento na classificação de anomalias e na decisão de repetir ensaios.

Validade externa. Os resultados foram obtidos com um único exemplar de hardware, um único operador de rede móvel, uma única região geográfica e uma janela temporal

Tabela 5.20. – Matriz de rastreabilidade entre requisitos, implementação e validação.

Req.	Descrição resumida	Impl.	Validação	Valor medido	Ver.
RF01	Aquisição de posição	4.2.3, 4.2.4	5.5.3, 5.6	NR	NV
RF02	Transmissão de telemetria	4.2.6, 4.3.2	5.3	NR	NV
RF03	Autenticação do cuidador	4.3.7, 4.4.2	5.4.1, 5.5.7	Qualitativo	Passou
RF04	Mapa em tempo real	4.4.4	5.4.3	Qualitativo	Passou
RF05	Configuração de cercas	4.3.1, 4.4.5	5.4.4	Qualitativo	Passou
RF06	Deteção de evasão	4.3.3	5.4.5	Qualitativo	Passou
RF07	Geração de alertas	4.3.3, 4.3.4	5.4.5	Qualitativo	Passou
RF08	Notificação do cuidador	4.3.6	5.3, 5.5.4	NR	NV
RF09	Estado da bateria	4.2.8, 4.3.5, 4.4.8	5.4.6	Qualitativo	Passou
RF10	Persistência do histórico	4.3.1, 4.3.5	5.4.3	Qualitativo	Passou
RF11	Emparelhamento de dispositivo	4.3.7, 4.4.3	5.4.2	Qualitativo	Passou
RF12	Histórico visual de percursos	4.4.6	5.4.3	Qualitativo	Passou
RF13	Posicionamento alternativo em interior	4.2.4, 4.3.2	5.5.3, 5.6	NR	NV
RNF01	Autonomia ≥ 6 h; repouso configurado de 20 s	4.2.7	5.5.2	$\sim$10 h (extrap.)	Parcial
RNF02	Precisão < 10 m (p95)	4.2.3	5.5.3	Dispersão, não erro	Não val.
RNF03	Latência < 30 s	4.3.3, 4.3.4	5.5.4	Não medida	Não cumpre (modelo)
RNF04	Tolerância a falha de rede	4.2.5	5.5.5	263 s sem pontos	Parcial
RNF05	Usabilidade	4.4	5.5.6	NM	NV
RNF06	Segurança dos dados	4.3.7	5.5.7	11/11 verificações	Parcial
RNF07	Consumo de dados	4.2.6	5.5.8	NM	Não cumpre (modelo)
RNF08	Desempenho da interface	4.4.1	5.5.6	NM	NV
RNF09	Robustez ambiental	—	5.5.9	—	Não val.
RNF10	Escalabilidade	4.3	5.5.10	p95 159 ms; 0/900 erros	Passou

reduzida. Não é possível generalizar o desempenho observado a outras combinações de hardware, cobertura ou ambiente.

Validade de construção. Não houve referência independente de posição no ensaio de campo, pelo que a dispersão de quatro leituras não permite estimar a exatidão nem validar o erro exigido pelo RNF02. Os horários provêm do telemóvel e da aplicação, sem verificação independente da sincronização; intervalos entre pontos não equivalem à latência de notificação. O valor atípico de 101 m permanece reportado, sem exclusão silenciosa.

Validade de conclusão. O subconjunto aparentemente estacionário contém apenas quatro leituras, pelo que os resultados de dispersão têm carácter descritivo e não suportam inferência sobre o erro de posição. A avaliação de usabilidade continua por realizar e não é apresentada como um ensaio concluído.

6 Discussão

6.1 Grau de Cumprimento dos Objetivos

Os quatro objetivos definidos na Secção 1.3 são retomados individualmente.

Monitorizar a localização em tempo real. Implementado e observado em campo, sem garantia de periodicidade constante. O sistema utiliza `INTERVAL_S`=20 s como repouso configurado, mas foram observados intervalos de 83 s entre posições em condições de aquisição degradada. A posição é apresentada num mapa interativo na aplicação do cuidador, com indicação da origem da posição e do tempo decorrido desde a última comunicação. A implementação excede a especificação ao acrescentar posicionamento alternativo por Wi-Fi em ambiente interior, onde o GNSS não obtém *fix*.

Definir perímetros de segurança configuráveis. Cumprido e excedido. A especificação previa uma única cerca circular por dispositivo; a implementação suporta múltiplas zonas, circulares ou poligonais, com semântica de segurança por união e com o mesmo algoritmo de decisão no *backend* e na aplicação.

Receber alertas automáticos de emergência. Parcialmente validado. A geração e persistência de alertas foram verificadas, assim como uma notificação de regresso à zona. Falta evidência suficiente da entrega da notificação de saída nos estados previstos da aplicação. A latência inferior a 30 segundos não foi medida de ponta a ponta; o modelo da Secção 5.5.4 indica incumprimento na configuração considerada.

Consultar o estado de carga da bateria. Implementado, com limitações na estimativa. O nível de bateria acompanha cada leitura de telemetria e é apresentado na aplicação, mas as oscilações observadas em campo excedem a banda morta do alerta. A apresentação do valor não demonstra a exatidão do estado de carga nem garante a ausência de falsos alertas.

Para além dos objetivos definidos, foram entregues quatro funcionalidades não previstas na especificação inicial: geofencing poligonal, histórico visual de percursos, monitorização autónoma da saúde do dispositivo — silêncio prolongado e bateria fraca — e uma suite de 43 testes automatizados sobre a lógica crítica do *backend*.

A coerência da decisão geométrica entre servidor e aplicação é sustentada por testes dos dois lados: `geo.test.js` verifica a implementação do *backend*, enquanto `geo_utils_test.dart` verifica a implementação correspondente na aplicação. Esta cobertura bilateral sustenta a afirmação da Secção 4.3.3 de coerência da classificação dentro/fora nos casos testados entre o estado processado pelo servidor e o apresentado ao cuidador.

6.2 Comparação com Soluções Existentes

A comparação sistematizada na Tabela 2.1 permite situar o sistema desenvolvido face às alternativas existentes.

O SafeTrack não supera qualquer das soluções analisadas na dimensão em que cada uma se especializa. Não possui a deteção de anomalias por aprendizagem de rotina do AngelSense, que dispensa o cuidador de antecipar manualmente cada padrão de risco. Não alcança a autonomia do Jiobit, cujo hardware dedicado é otimizado para consumo, ao passo que o SafeTrack assenta numa placa de desenvolvimento genérica. Não iguala a maturidade de protocolo do Traccar, que suporta milhares de modelos de hardware comercial. E não atinge a precisão em interior ao nível da divisão descrita na literatura baseada em BLE, que exige, contudo, infraestrutura instalada no espaço monitorizado.

A contribuição do trabalho reside na integração e documentação do protótipo, não na exclusividade das funcionalidades isoladas. O cuidador administra o projeto e os dados, permanecendo dependente dos fornecedores dos serviços utilizados. Os alertas de silêncio e bateria fraca também existem no Traccar [20]; no SafeTrack, foram implementados com limiares, histerese e resolução por tipo definidos para este protótipo. Os canais de notificação e a aquisição adaptativa são decisões de implementação cuja utilidade deve ser apreciada juntamente com os limites da validação experimental.

6.3 Requisitos Não Cumpridos e Sua Justificação Técnica

Os veredictos distinguem incumprimento previsto, validação parcial e ausência de validação experimental. O RNF02 permanece não validado por ausência de referência independente; o ensaio de campo também não demonstra a recuperação integral exigida pelo RNF04 e limita a interpretação da autonomia extrapolada do RNF01. Os casos seguintes são analisados quanto à causa, ao risco associado e ao caminho de mitigação.

RNF03 — Latência de alerta

O incumprimento previsto decorre do intervalo entre leituras e da confirmação por duas leituras consecutivas fora de todas as zonas ativas. Este *debounce* acrescenta um ciclo completo à latência para suprimir saídas isoladas. A Tabela 5.12 compara um ciclo hipotético de 23 s com ciclos observados de 83 s: neste último, a espera e a confirmação somam 83 a 166 s, antes dos termos residuais de entrega não medidos.

O compromisso é, portanto, entre latência e taxa de falsos positivos. Num sistema de segurança destinado a cuidadores, um alerta falso repetido tem custo elevado: conduz à desativação das notificações e, por essa via, à perda do alerta verdadeiro. O ensaio impede, contudo, que este compromisso seja apresentado como um acréscimo de apenas dez a vinte segundos: com aquisição degradada, a confirmação acrescenta um intervalo de 83 s. A configuração deve ser reavaliada, e não justificada apenas pela redução de falsos positivos, pois a margem de incumprimento do RNF03 é substancialmente superior à inicialmente prevista.

Existem três caminhos de resolução: reduzir adicionalmente o intervalo do ciclo, o que agrava o consumo energético e de dados; diferenciar o *debounce* em função da confiança da leitura, dispensando-o quando a posição tem precisão elevada e a distância à fronteira é significativa; ou rever formalmente o requisito para um valor compatível com a arquitetura, justificando a revisão. A segunda alternativa depende, contudo, de uma alteração prévia ao firmware: atualmente não são transmitidos o HDOP, o número de satélites nem outra medida de qualidade do *fix* GNSS, pelo que as posições de satélite entram no sistema

com precisão igual a zero. Sem essa informação, o *debounce* diferenciado em função da confiança é tecnicamente impossível para leituras GNSS.

RNF06 — Segurança dos dados

O requisito é parcialmente cumprido. O controlo de acesso encontra-se integralmente implementado e verificado: o cuidador acede apenas aos seus dados, a telemetria e os alertas são de escrita reservada às funções privilegiadas, e o emparelhamento exige um código que o cliente nunca chega a ler.

Durante a execução de TF-02.2, em 6 de setembro de 2026, foi identificado um defeito adicional no controlo de acesso, entretanto corrigido: a regra de criação de dispositivos permitia que uma conta autenticada criasse um documento com um identificador arbitrário, declarando-se proprietária sem apresentar o código de emparelhamento. Como a escrita sobre um documento inexistente é uma criação e não uma atualização, as condições de emparelhamento não eram aplicadas. Esta falha permitia a apropriação antecipada de um identificador ainda não provisionado; se o equipamento real começasse posteriormente a transmitir, a ingestão aceitaria a telemetria para o documento existente e a conta atacante poderia receber a localização. A previsibilidade dos identificadores agravava o risco.

A criação pelo cliente foi negada através de `allow create: if false`, com correção aplicada e implantada no mesmo dia. O provisionamento ficou reservado ao Admin SDK, mantendo-se o emparelhamento como atualização. Após a correção, o identificador inexistente foi recusado com HTTP 403 e TF-02.6 confirmou, sem regressão, a reivindicação, libertação e nova reivindicação legítimas (Tabela 5.5). Ao contrário das limitações de validação do certificado e de chave partilhada descritas a seguir, este defeito foi corrigido, constituindo um resultado concreto do processo de verificação.

A confidencialidade em trânsito, no troço entre o dispositivo e a nuvem, é assegurada apenas parcialmente. O canal é cifrado com TLS 1.2, o que protege contra interceção passiva, mas o certificado do servidor não é validado. Esta limitação não resulta de omissão: foi diagnosticada metodicamente e atribuída a um defeito da firmware do módulo, confirmado com três âncoras de confiança distintas e com o relógio do modem sincronizado. Trata-se, por conseguinte, de uma impossibilidade técnica no hardware utilizado, e não de uma escolha de conveniência.

O risco correspondente deve, ainda assim, ser enunciado sem atenuação. Perante um atacante ativo capaz de se interpor entre o dispositivo e a rede móvel, o dispositivo aceita qualquer certificado, o que permite decifrar, ler e alterar o *payload* antes de o reencaminhar — incluindo forjar coordenadas ou ocultar uma saída de zona — bem como capturar a chave de autenticação em trânsito. A probabilidade é baixa para um atacante oportunista, por exigir capacidade de interceção de tráfego celular, mas não é desprezável no perfil de ameaça específico deste sistema, em que o adversário mais plausível não é genérico: é alguém com motivo concreto para manipular a localização de uma pessoa vulnerável.

A este risco acresce o da chave partilhada única, comum a todos os dispositivos, sem rotação, sem assinatura do corpo do pedido e sem proteção contra repetição. Um pedido capturado pode ser reenviado mais tarde, o que permitiria repetir uma posição antiga enquanto a pessoa já se encontrava noutro local.

As mitigações identificadas, por ordem de relação entre impacto e esforço, são: chave individual por dispositivo, gerada no emparelhamento e nunca legível pelo cliente, à semelhança do código de emparelhamento já implementado; assinatura do corpo do pedido por Hash-based Message Authentication Code (HMAC) sobre o conteúdo e um carimbo temporal, com janela de validade curta, o que confere integridade e resistência a repetição mesmo sobre um canal não autenticado; migração para um módulo em que a validação de certificado seja funcional; e cifra da memória flash do ESP32, que protege a chave contra extração por acesso físico.

RNF07 — Consumo de dados

A projeção apresentada na Secção 5.5.8 indica consumo superior ao especificado sob a hipótese temporal então adotada e com estabelecimento de uma ligação TLS completa em cada envio. Não corresponde a consumo real medido: reduzir o repouso de 60 para 20 segundos não implica triplicar o número de transmissões quando a aquisição prolonga os ciclos. Os intervalos de 83 s observados em campo exigem que uma futura projeção utilize a distribuição efetiva dos ciclos e o tráfego contabilizado, sem generalizar a partir deste pequeno subconjunto.

A mitigação com melhor relação entre impacto e esforço não é aumentar o intervalo de reporte, mas evitar o restabelecimento da ligação: manter a sessão TLS entre ciclos

é incompatível com o *deep sleep* do ESP32, mas a retoma de sessão, quando suportada pelo modem, reduziria substancialmente o custo por ciclo. Em alternativa, agregar várias leituras num único envio periódico reduziria o número de ligações na proporção do fator de agregação, ao custo de latência acrescida — compromisso que interage diretamente com o RNF03.

RNF09 — Robustez ambiental

O requisito não foi validado experimentalmente, por indisponibilidade de câmara climática. A evidência documental relativa às gamas de operação dos componentes individuais não permite emitir veredicto sobre o conjunto montado, pelo que o requisito é declarado como não validado, e não como cumprido.

6.4 Limitações Assumidas

Para além dos requisitos analisados na secção anterior, o sistema apresenta as seguintes limitações, assumidas de forma explícita.

A autonomia energética é significativamente inferior à de hardware dedicado de baixo consumo, o que decorre de duas opções combinadas: a utilização de uma placa de desenvolvimento genérica, com reguladores e periféricos não otimizados, e a decisão de manter o modem e o recetor GNSS permanentemente ligados para privilegiar a latência de reporte.

Justificação do RNF01. A autonomia mínima de 6 horas foi adotada após a decisão de manter o modem e o GNSS ligados, preservando a efeméride e o registo celular. O repouso configurado de 20 s não garante, porém, esse intervalo entre posições: o cenário de aquisição degradada implica uma previsão de 83 a 166 s apenas para a espera e o *debounce*, conforme a Secção 5.5.4. A estimativa anterior de aproximadamente 10 h de autonomia foi extrapolada das percentagens reportadas, não medida por descarga completa. As oscilações observadas em campo fragilizam essa extrapolação; por isso, não se mantém a conclusão de cumprimento experimental do RNF01 sem um ensaio de duração suficiente e uma avaliação independente do estado de carga. O requisito fica parcialmente validado pela operação

diretamente observada durante 5 h 28 min e pela projeção a partir da carga consumida, mantendo-se a distinção entre evidência parcial e demonstração integral do critério.

O sistema não implementa deteção de anomalias por aprendizagem de padrões de deslocação, dependendo exclusivamente de zonas definidas manualmente pelo cuidador. Não dispõe de comunicação por voz nem de mecanismo de fixação anti-remoção, pelo que não constitui uma solução completa de segurança pessoal, mas um sistema de localização, geofencing e monitorização de estado.

A gestão suportada é de um dispositivo por cuidador. A arquitetura de dados não impede a extensão a múltiplos dispositivos, mas a interface e os fornecedores de estado assumem um dispositivo único.

O firmware não transmite qualquer medida de qualidade do *fix* GNSS, como o HDOP ou o número de satélites utilizados. Todas as posições de satélite são, por isso, tratadas como exatas pelo *backend*; apenas as posições resolvidas por Wi-Fi transportam uma estimativa de precisão. Esta limitação impede atualmente a estratégia de *debounce* diferenciado discutida na Secção 6.3, sendo a transmissão da precisão estimada do GNSS o seu pré-requisito técnico.

O protótipo foi ensaiado numa caixa de plástico transparente genérica, sem vedação nem fixação, conforme descrito na Secção 4.5. Esta solução oferece proteção mecânica básica durante o ensaio, mas não é adequada a utilização continuada pelo portador. A resistência a água e poeiras não foi validada experimentalmente, por ausência de ensaios de proteção ambiental.

A leitura de bateria apresentou uma queda de 19 pontos percentuais e uma recuperação de 11 pontos em 21 segundos. A banda morta de 20 %/30 %, dimensionada para oscilações de um ou dois pontos, é insuficiente perante esta amplitude. Existe o risco de disparo com carga real bastante acima de 20 %; como não houve referência independente de carga, não se afirma que esse falso alerta tenha ocorrido. Esta limitação afeta tanto a informação apresentada ao cuidador como qualquer autonomia inferida apenas da percentagem reportada.

6.5 Privacidade, Proteção de Dados e Considerações Éticas

O sistema trata dados de localização contínua de uma pessoa singular, os quais constituem dados pessoais na aceção do Regulamento Geral sobre a Proteção de Dados (RGPD). Acresce que o titular dos dados é, por definição do caso de uso, uma pessoa em situação de vulnerabilidade — idoso com défice cognitivo ou criança — cuja capacidade de prestar consentimento informado é reduzida ou inexistente. Esta circunstância desloca a questão do plano estritamente técnico para o plano ético.

A região `europe-west1` foi escolhida para as Cloud Functions, mas não demonstra a residência de todos os dados na União Europeia: segundo a documentação do fornecedor, o Firebase Authentication é executado nos Estados Unidos. O cliente administra o projeto, mas a Google atua geralmente como subcontratante no tratamento dos dados [35]. A localização e as condições de tratamento devem ser verificadas por serviço; a escolha da região não demonstra, isoladamente, conformidade com o RGPD. O Telegram acrescenta outro serviço externo ao percurso das mensagens. As regras do Firestore restringem o acesso dos clientes aos dados do cuidador proprietário, sem eliminar o processamento pelos fornecedores.

A minimização de dados é apenas parcialmente observada: o sistema conserva o histórico completo de posições sem política de retenção definida. A definição de um período de retenção, com eliminação automática das posições mais antigas, constitui uma melhoria identificada e não implementada.

Do ponto de vista ético, a tensão fundamental é entre segurança e autonomia da pessoa monitorizada. A monitorização contínua de um adulto, ainda que com défice cognitivo, constitui uma restrição da sua privacidade que só se justifica na medida do risco que previne. O sistema foi concebido para alertar em caso de saída de uma zona segura, e não para vigilância permanente do quotidiano: as zonas são definidas pelo cuidador e o alerta é o evento central, não a consulta contínua da posição. Esta distinção é de desenho, não de aplicação — nada impede a consulta contínua — pelo que a utilização responsável do sistema depende, em última análise, do cuidador.

Um sistema desta natureza deveria, num contexto de utilização real, pressupor o consentimento informado do titular sempre que este o possa prestar, ou a intervenção do seu representante legal quando não o possa, bem como a informação clara sobre que dados são recolhidos, por quanto tempo e com que finalidade.

6.6 Viabilidade Económica

Tabela 6.1. – Custo de materiais do protótipo.

Componente	Quantidade	Custo
Placa LilyGO T-SIM7000G (inclui ESP32, GNSS e antenas)	1	Custo não documentado
Antena GNSS	1	Incluída na placa
Antena celular	1	Incluída na placa
Célula de iões de lítio 18650 (3000 mAh)	1	2,00 EUR
Cartão MicroSD	1	10,00 EUR
Cartão SIM (Things Mobile)	1	10,00 EUR
Subtotal conhecido (exclui placa)	—	**22,00 EUR**

O subtotal documentado é de 22,00 EUR, excluindo a placa, cujo custo não foi disponibilizado; não representa o custo total do protótipo. A operação depende do tráfego de dados móveis e da utilização dos serviços de nuvem e geolocalização. Sem registos de faturação e utilização, não se confirma que estes serviços permaneçam dentro das respetivas quotas gratuitas.

O consumo e o custo mensal efetivos não foram documentados. A projeção de tráfego da Secção 5.5.8 é analítica e não substitui uma medição do operador ou uma fatura.

A comparação económica exige considerar aquisição, dados móveis, nuvem e manutenção ao longo do período de utilização. A ausência de subscrição comercial de rastreamento não permite concluir, por si só, que a operação seja gratuita ou que o custo total seja inferior ao das alternativas.

6.7 Trabalho Futuro

As linhas de trabalho futuro identificadas organizam-se em quatro grupos. Os elementos exploratórios das maquetas — botão SOS acionado pelo portador, chamada para

o dispositivo, estimativa de autonomia restante em horas e painel de estado do SIM com indicação de *roaming* — não integram o produto mínimo viável nem os seus requisitos funcionais. A sua eventual implementação constitui trabalho futuro, sujeito a especificação e validação próprias.

Segurança. Individualização da chave por dispositivo, gerada no emparelhamento; assinatura do corpo do pedido com carimbo temporal, para conferir integridade e resistência a repetição; migração para um módulo celular em que a validação de certificado seja funcional; e cifra da memória flash do microcontrolador. A estas acresce uma medida de deteção de anomalias no servidor — rejeição de saltos de posição fisicamente impossíveis entre leituras consecutivas — que mitiga parcialmente a injeção de coordenadas falsas mesmo em caso de comprometimento da chave.

Autonomia. Adoção do PSM e do eDRX do lado celular, que permitem ao modem permanecer registado consumindo uma fração da corrente atual, e gestão adaptativa da alimentação do recetor GNSS, aceitando aquisições menos rápidas em troca de não manter o recetor permanentemente ativo. Complementarmente, um ciclo adaptativo que reduza a frequência de reporte quando a carga desce abaixo de um limiar poderá prolongar a autonomia útil. Antes de fundamentar esta decisão na percentagem reportada, é necessário introduzir filtragem temporal da leitura de bateria ou sincronizar a amostragem com períodos de repouso do modem e validar a estimativa resultante contra uma referência independente.

Funcionalidade. Suporte a múltiplos dispositivos por cuidador; zonas com ativação por horário; deteção de anomalias por aprendizagem de padrões de deslocação; definição de uma política de retenção de dados históricos; e transmissão, pelo firmware, de métricas de qualidade do *fix* GNSS, nomeadamente HDOP e número de satélites, permitindo aplicar um *debounce* ajustado à confiança da leitura.

Produto. Conclusão do canal de notificação nativo mediante obtenção de chave APNs; construção de um invólucro adequado ao uso por uma pessoa, com fixação e resistência a intempéries; e ensaio do conjunto montado em condições ambientais controladas. Acrescem a personalização do modelo de correio de autenticação e do nome público do projeto Firebase, a configuração de um domínio de envio próprio para reduzir a probabilidade de classificação das mensagens como spam e a adaptação da página de reposição para português. Estas medidas respondem ao ensaio em que a mensagem foi entregue na pasta

de spam com o identificador interno do projeto, e a página de reposição foi apresentada em inglês no domínio por omissão do Firebase, apesar de a ligação funcionar corretamente.

7 Conclusão

7.1 Síntese do Trabalho Realizado

O presente trabalho teve por objetivo a prototipagem de um sistema IoT autónomo de rastreamento e geofencing para pessoas vulneráveis, integrando três camadas tecnológicas distintas: um dispositivo baseado em microcontrolador com GNSS e comunicações celulares, uma infraestrutura de nuvem sem servidor para ingestão e processamento espacial de telemetria, e uma aplicação móvel para o cuidador.

O sistema foi desenvolvido de forma faseada, com validação incremental de cada camada. A camada de nuvem foi construída e validada com um simulador de dispositivo antes da existência de firmware funcional, o que permitiu separar problemas de lógica aplicacional de problemas de integração de hardware. O firmware foi desenvolvido em seguida, e a aplicação móvel por último, sobre um *backend* já verificado.

Durante o desenvolvimento foram identificados e analisados problemas de integração que não eram previsíveis na fase de especificação, dos quais se destacam o conflito de coexistência de radiofrequência entre o recetor GNSS e o transmissor celular, resolvido por uma estratégia de aquisição adaptativa em duas fases, e a inviabilidade da validação de certificado no módulo utilizado, diagnosticada de forma sistemática e documentada como limitação assumida com modelo de ameaça explícito.

A leitura conjunta do Capítulo 5 e da Tabela 5.20 permite distinguir, entre os dez requisitos não funcionais, um cumprido (RNF10), três parcialmente validados (RNF01, RNF04 e RNF06), dois com incumprimento indicado pelas previsões analíticas (RNF03 e RNF07) e quatro sem validação experimental concluída (RNF02, RNF05, RNF08 e RNF09); entre os treze requisitos funcionais, nove passaram a estar validados (RF03, RF04, RF05, RF06, RF07, RF09, RF10, RF11 e RF12), enquanto quatro permanecem por validar (RF01, RF02, RF08 e RF13), devido à insuficiência da evidência disponível para concluir a respetiva validação. Os resultados favoráveis decorrem dos ensaios de escalabilidade e das verificações parciais de autonomia, conectividade e segurança, enquanto

os incumprimentos previstos resultam dos intervalos entre leituras, da confirmação de evasão e do custo de transmissão considerado nos modelos, não de medições completas de latência ou consumo mensal. Os requisitos restantes carecem de ensaios ou de referências independentes suficientes, não sendo a extrapolação de autonomia, a dispersão das posições ou a evidência documental substitutos da validação experimental exigida.

A campanha de validação identificou e corrigiu um defeito de segurança nas regras de acesso do Firestore, revelado pelo caso negativo TF-02.2 em 6 de setembro de 2026; a verificação posterior confirmou a recusa da criação não autorizada e a ausência de regressão no emparelhamento legítimo, constituindo em si um resultado do processo de verificação.

7.2 Contribuições

O trabalho apresenta quatro contribuições identificáveis.

Do ponto de vista da integração de sistemas, a construção de um protótipo que articula aquisição de posição, geofencing, persistência de alertas e canais de notificação, sem subscrição comercial de rastreamento e com o projeto administrado pelo utilizador. A validação integral da entrega de alertas de saída permanece por concluir.

Do ponto de vista da engenharia de sistemas embebidos, a documentação de uma estratégia de aquisição adaptativa num módulo combinado, com redução do repouso configurado de 60 para 20 segundos e eliminação do restabelecimento do registo celular em cada ciclo. Esta configuração não garante igual periodicidade de reporte: o intervalo entre posições observado em campo atingiu 83 segundos em condições de aquisição degradada.

Do ponto de vista da conceção de sistemas de segurança, a implementação de alertas de silêncio e bateria fraca, com histerese e resolução por tipo. Estas funções não são exclusivas do SafeTrack; o contributo reside na sua integração e avaliação no protótipo.

Do ponto de vista da qualidade de *software*, a construção de uma suite de testes automatizados sobre a lógica crítica, com o algoritmo de decisão geométrica partilhado e testado de forma independente no *backend* e na aplicação, o que reduz o risco de divergência entre o estado apresentado ao cuidador e o estado que gera o alerta.

7.3 Reflexão Final

O presente trabalho começou com mais ideias do que tempo. Na fase inicial, a possibilidade de construir um sistema completo — do sensor à notificação no telemóvel — trouxe um entusiasmo que, olhando para trás, se traduziu numa ambição mal calibrada. Idealizei funcionalidades que o âmbito de um produto mínimo viável não comportava e subestimei o peso da integração entre três camadas tecnológicas que, isoladamente, me pareciam dominadas.

O maior erro que cometi não foi técnico. Foi de comunicação. Trabalhei durante demasiado tempo de forma isolada, adiando o contacto com o orientador e com o docente auxiliar para momentos em que já tinha decisões tomadas, em vez de as discutir enquanto ainda eram reversíveis. Várias das dificuldades que enfrentei — a definição do âmbito, a escolha do hardware, o critério de autonomia energética — teriam sido resolvidas mais depressa, ou evitadas, com uma conversa de vinte minutos no momento certo. Aprendi que procurar orientação não é sinal de fragilidade num projeto individual; é o mecanismo que impede um projeto individual de se fechar sobre si próprio.

A meio do desenvolvimento tornou-se evidente que algumas limitações não decorriam do software, mas do hardware escolhido. A placa de desenvolvimento utilizada não expõe qualquer medida de qualidade do fix GNSS, a firmware do módulo celular apresenta um defeito de validação de certificado que nenhuma alteração ao meu código resolve, e o consumo em repouso está longe do que uma solução dedicada permitiria. Perceber que o passo seguinte exigia componentes de custo substancialmente superior — precisamente aquilo que o projeto se propunha evitar — foi desmotivante, e houve semanas em que o trabalho avançou pouco.

Foi também o momento em que o trabalho mudou de natureza. Deixei de tentar esconder essas limitações e passei a medi-las, a documentá-las e a explicar as suas causas. O conflito de coexistência de radiofrequência entre o recetor de satélite e o transmissor celular, a impossibilidade de validar o certificado do servidor, a distância entre o intervalo de reporte configurado e o observado em campo, a amplitude real da estimativa de bateria — nenhum destes resultados é o que eu queria apresentar, e todos são mais úteis do que a alternativa. Um sistema que sabe onde falha é mais defensável do que um sistema que afirma não falhar.

Se recomeçasse, faria três coisas de forma diferente: definiria o âmbito com o orientador antes de comprar seja o que for, marcaria pontos de situação regulares em vez de os deixar acontecer por necessidade, e reservaria desde o início tempo para validação experimental, em vez de a empurrar para o fim. Fica, ainda assim, um protótipo integrado, com validação ainda parcial, um conjunto de problemas de integração diagnosticados até à causa, e a noção clara — que não tinha em julho — de que a parte difícil da engenharia raramente é escrever o código.

Bibliografia

[1] Alzheimer's Association, "Wandering and getting lost: Who's at risk and how to be prepared," 2023, [Online]. Available: https://www.alz.org/media/Documents/alzheimers-dementia-wandering-behavior-ts.pdf.

[2] WiseGuy Reports, "Dementia gps tracker market — comprehensive analysis 2032," 2024, [Online]. Available: https://www.wiseguyreports.com/reports/dementia-gps-tracker-market.

[3] u-blox, "Wearable devices for elderly and child tracking," 2023, [Online]. Available: https://www.u-blox.com/en/wearable-devices-for-elderly-and-child-tracking.

[4] M. Doyle *et al.*, "Implementing global positioning system trackers for people with dementia who are at risk of wandering," *Dementia*, Apr. 2024.

[5] T. G. Stavropoulos, A. Papastergiou, K. Mpaltadoros, E. Nikolopoulos, and S. Kompatsiaris, "Iot wearable sensors and devices in elderly care: A literature review," *Sensors*, vol. 20, no. 10, p. 2826, May 2020.

[6] S. Mahapatra *et al.*, "An iot-based wristband for automatic people tracking, contact tracing and geofencing for covid-19," *Sensors*, vol. 22, no. 24, p. 9902, Dec. 2022.

[7] LiveViewGPS, "How geofencing works: A step-by-step guide," 2024, [Online]. Available: https://www.lvgps.com/.

[8] C. Veness, “Calculate distance between two points — haversine formula,” [Online]. Available: https://www.movable-type.co.uk/scripts/latlong.html.

[9] M. M. Raikar, K. Angadi, S. Reddy, N. Naik, and A. Doddwada, “Fleet tracking and geofencing using the internet of things (IoT),” in *2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC)*. Jalandhar, India: IEEE, 2023, pp. 463–468.

[10] M. D. Mughni and P. A. R. Devi, “Implementation of teacher presence system using mobile-based geofencing & haversine formula methods,” *Applied Technology and Computing Science Journal*, vol. 6, no. 1, pp. 31–40, Jun. 2023.

[11] Onyax, “NB-IoT vs. LTE-M: A comparison,” 2024, [Online]. Available: https://onyax.com/.

[12] Particle, “Comparing LTE-M vs. NB-IoT,” [Online]. Available: https://www.particle.io/blog/lte-m-vs-nb-iot/.

[13] KORE Wireless, “NB-IoT vs. LTE-M: What’s the difference?” [Online]. Available: https://www.korewireless.com/.

[14] L. C. Keen, “Mobile app architecture with flutterfire,” Dec. 2022, [Online]. Available: https://medium.com/@limcheekeen.63/mobile-app-architecture-with-flutterfire-72daa2affb93.

[15] S. Getachew, “The ultimate flutter guide for cross-platform apps in 2024,” Sep. 2024, [Online]. Available: https://medium.com/@solomongetachew112/the-ultimate-flutter-guide-for-building-cross-platform-apps-in-2024-0b2b3d1fa2e1.

[16] ThinkSys, “How to develop cross-platform mobile apps with flutter,” 2024, [Online]. Available: https://thinksys.com/development/powerful-cross-platform-app-development-with-flutter/.

[17] AngelSense, “AngelSense vs. Jiobit: Best GPS tracker for autistic children,” 2026, [Online]. Available: https://www.angelsense.com/blog/gps-tracker-comparison-autism-angelsense-vs-jiobit/.

[18] GPSDad, “Jiobit GPS tracker review 2026,” 2026, [Online]. Available: https://gpsdad.com/reviews/jiobit-review-2026/.

[19] Traccar, “Traccar — open source GPS tracking platform,” [Online]. Available: https://www.traccar.org/.

[20] ——, “Events and notifications,” [Online]. Available: https://www.traccar.org/events/. Consultado em 6 de setembro de 2026.

[21] E. R. Pratama, F. Renaldi, F. R. Umbara, and E. C. Djamal, “Geofencing technology in monitoring of geriatric patients suffering from dementia and Alzheimer,” in *2020 3rd International Conference on Computer and Informatics Engineering (IC2IE)*, 2020, pp. 106–111.

[22] G. Guerrero-Ulloa, C. Rodríguez-Domínguez, and M. J. Hornos, “Agile methodologies applied to the development of internet of things (iot)-based systems: A review,” *Sensors*, vol. 23, no. 2, p. 790, Jan. 2023.

[23] N. Ahmad *et al.*, “Hybrid project management between traditional software development lifecycle and agile based product development for future sustainability,” *Sustainability*, vol. 15, no. 2, p. 1121, Jan. 2023.

[24] LilyGO, “T-sim7000g — esp32 with sim7000g module,” 2023, [Online]. Available: https://github.com/Xinyuan-LilyGO/LilyGO-T-SIM7000G.

[25] Cirkit Designer, “T-sim7000g,” [Online]. Available: https://docs.cirkitdesigner.com/component/5901959b-e405-4785-a4c5-b01c6ac86eb8/t-sim7000g.

[26] SIMCom Wireless Solutions, *SIM7000 Series — Hardware Design*, 1st ed., 2021, [Online]. Available from the official SIMCom download page: https://en.simcom.com/product/SIM7000G.html.

[27] Programming Electronics Academy, “Esp32 deep sleep mode,” [Online]. Available: https://www.programmingelectronics.com/esp32-deep-sleep-mode/.

[28] Google, “Welcome to firebase for flutter! | firebase documentation,” 2026, [Online]. Available: https://firebase.google.com/docs/flutter/setup.

[29] S. Bhatnagar, “Designing an intuitive mobile dashboard user interface (ui): 6 best practices,” Dec. 2024, [Online]. Available: https://www.pixelbrainy.com/blog/mobile-dashboard-user-interface/.

[30] PlatformIO, “PlatformIO core documentation,” [Online]. Available: https://docs.platformio.org/.

[31] Google, “Geolocation API overview,” [Online]. Available: https://developers.google.com/maps/documentation/geolocation/overview.

[32] “ESP32 deep sleep current: What the datasheet says vs. what you will actually measure,” [Online]. Available: https://hubble.com/community/guides/esp32-deep-sleep-current-what-the-datasheet-says-vs-what-you-ll-actually-measure/.

[33] Lucidar, “ESP32 real current consumption in deep sleep mode,” [Online]. Available: https://lucidar.me/en/esp32/power-consumption-of-esp32/.

[34] Botletics, “SIM7000 current consumption — SIM7000-LTE-Shield wiki,” [Online]. Available: https://github.com/botletics/SIM7000-LTE-Shield/wiki/Current-Consumption.

[35] Google, “Privacy and security in Firebase,” 2026, [Online]. Available: https://firebase.google.com/support/privacy. Consultado em 6 de setembro de 2026.

[36] ——, “Firebase security rules for cloud firestore,” [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started.

A

Proposta de Ligações com Componentes Discretos

A Figura A.1 apresenta uma proposta conceptual de ligações com componentes discretos, ilustrando a interligação dos módulos e a organização da alimentação. A comparação desta abordagem com o módulo integrado escolhido encontra-se na Secção 3.2.2.

A proposta representa um Arduino UNO R3, um recetor u-blox NEO-6M e um modem SIM800L V2, juntamente com uma célula de iões de lítio, um módulo de carregamento TP4056 e um conversor MT3608. São distinguidas as ligações de alimentação, a massa comum, as ligações série aos módulos e as ligações opcionais. No desenho, o Arduino é alimentado por USB e a alimentação do modem é representada separadamente, a partir da bateria.

Esta proposta não corresponde à montagem do protótipo SafeTrack, que utiliza a placa LilyGO T-SIM7000G. Uma eventual concretização da alternativa exigiria verificar a compatibilidade elétrica das variantes concretas dos módulos, os níveis lógicos das interfaces e o dimensionamento da alimentação antes de efetuar as ligações.

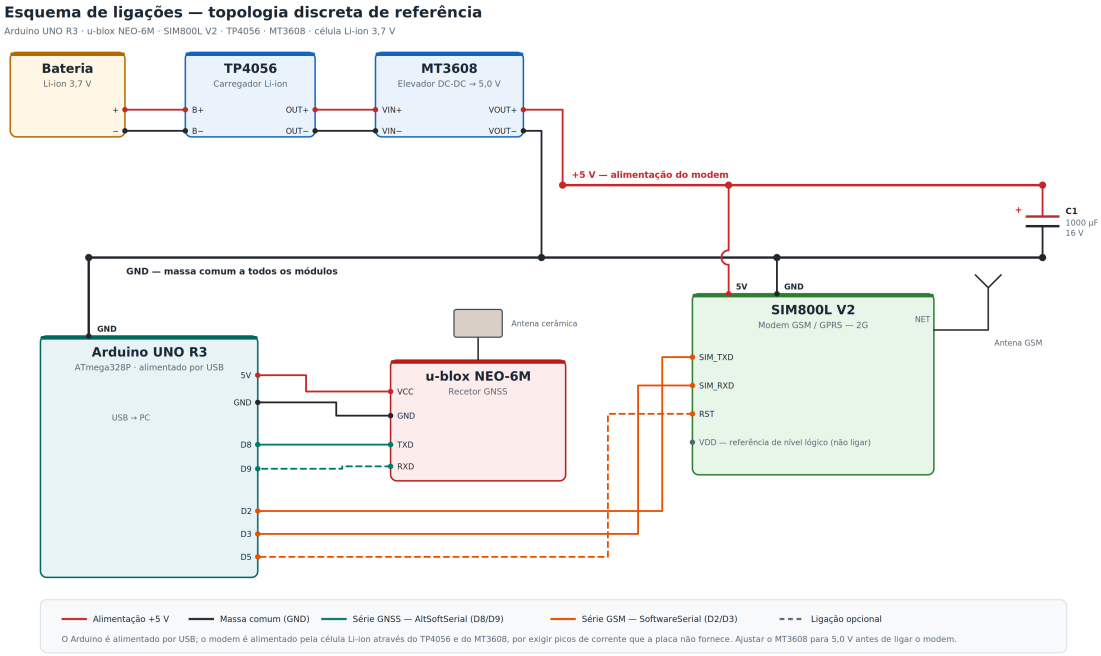


Figura A.1. – Proposta conceptual de ligações para uma topologia discreta de referência.

Manual do Utilizador

B

B.1 Âmbito

Este manual destina-se ao cuidador, ou seja, à pessoa que acompanha o portador do dispositivo através da aplicação móvel. Não pressupõe conhecimentos técnicos. A instalação e a configuração inicial do sistema estão descritas no Anexo C.

O sistema é composto por dois elementos que o cuidador utiliza: um dispositivo portátil, transportado pela pessoa monitorizada, e uma aplicação móvel instalada no telemóvel do cuidador. O dispositivo não tem interface: não tem ecrã, nem botões de utilização corrente, e a pessoa que o transporta não precisa de interagir com ele.

B.2 Antes de começar

Para utilizar o sistema são necessários:

- um telemóvel com iOS 15 ou superior;
- ligação à Internet no telemóvel;
- o dispositivo SafeTrack, com bateria carregada, cartão SIM ativo inserido e as duas antenas ligadas;
- o identificador do dispositivo e o respetivo código de emparelhamento de seis dígitos, fornecidos com o equipamento.

B.3 Criar conta e iniciar sessão

Ao abrir a aplicação pela primeira vez é apresentado o ecrã de autenticação.

1. Para criar uma conta, selecione *Registar*, introduza o endereço de correio eletrónico e defina uma palavra-passe. A aplicação indica os critérios mínimos exigidos e recusa o registo se não forem cumpridos.

2. Para entrar numa conta já existente, introduza as credenciais no ecrã de *Login*.

3. Se não se recordar da palavra-passe, selecione *Recuperar palavra-passe*. Receberá no seu endereço de correio eletrónico uma mensagem com a ligação de reposição.

Por motivos de segurança, quando as credenciais estão erradas a aplicação não indica qual dos campos falhou.

A sessão fica guardada no telemóvel: nas utilizações seguintes a aplicação abre diretamente no mapa.

B.4 Emparelhar o dispositivo

O emparelhamento associa um dispositivo à sua conta e só precisa de ser feito uma vez. Enquanto não existir um dispositivo associado, a aplicação apresenta o ecrã de emparelhamento em vez do mapa.

1. Introduza o **identificador do dispositivo**.

2. Introduza o **código de emparelhamento** de seis dígitos.

3. Confirme.

O pedido é validado no servidor. O emparelhamento é recusado se o identificador não existir, se o código não coincidir, ou se o dispositivo já estiver associado a outra conta —

neste último caso o proprietário anterior mantém-se, e o dispositivo tem de ser libertado primeiro por quem o detém.

Para transferir um dispositivo para outra conta, o proprietário atual deve desfazer o emparelhamento no ecrã de definições do dispositivo. O equipamento fica então disponível para ser reclamado de novo.

B.5 Ecrã do mapa

O mapa é o ecrã principal da aplicação e apresenta:

- o **marcador** com a última posição conhecida do dispositivo;
- as **zonas seguras** ativas, desenhadas sobre o mapa;
- um **cartão de estado** com o nível de bateria e a indicação de o portador se encontrar dentro ou fora de zona.

A posição é atualizada automaticamente, sem necessidade de recarregar o ecrã. O intervalo de repouso configurado é de 20 segundos (`INTERVAL_S`), mas não corresponde necessariamente ao intervalo entre posições. No ensaio de campo observaram-se ciclos de 83 segundos em condições de aquisição degradada; uma interrupção de ligação pode prolongar ainda mais a ausência de novas posições.

Marcador cinzento e aviso de ausência de sinal

Se não for recebida qualquer posição durante mais de cinco minutos, o marcador passa a cinzento e é apresentado um aviso de ausência de sinal. A última posição conhecida mantém-se visível. As causas mais frequentes são a passagem por um local sem cobertura de rede móvel, um túnel, um elevador ou um edifício com paredes espessas.

Este aviso é apenas informativo. Se a ausência de comunicação se prolongar, o sistema envia um alerta próprio, descrito na Secção B.7.

Posições obtidas por Wi-Fi

Em ambiente interior o dispositivo não consegue receber sinal de satélite. Nesses casos determina uma posição aproximada a partir das redes Wi-Fi visíveis à sua volta — sem se ligar a nenhuma delas. Estas posições são apresentadas com a respetiva margem de erro, que é tipicamente superior à das posições obtidas por satélite. O dispositivo regressa automaticamente ao posicionamento por satélite assim que voltar a haver cobertura.

B.6 Zonas seguras

Uma zona segura é uma área dentro da qual o portador é considerado em segurança. O sistema permite definir várias zonas por dispositivo — por exemplo, a residência, a escola e a casa de um familiar. O portador é considerado seguro enquanto se encontrar dentro de, pelo menos, uma zona ativa; o alerta é gerado apenas quando se encontra fora de todas.

Criar uma zona

No separador de zonas, selecione a criação de uma nova zona e escolha o tipo:

Zona circular Toque no mapa para definir o centro e ajuste o raio no controlo deslizante. O círculo é pré-visualizado à medida que ajusta.

Zona poligonal Toque sucessivamente no mapa para definir os vértices. São necessários pelo menos três.

Pode localizar um endereço através da pesquisa, evitando ter de navegar manualmente no mapa até ao local pretendido.

Gerir zonas

A biblioteca de zonas permite editar, desativar, reativar e eliminar cada zona. Uma zona desativada é ignorada na avaliação, mas fica guardada e pode ser reativada mais tarde — é a forma indicada de suspender temporariamente uma zona sem perder a sua configuração.

B.7 Alertas

O sistema gera três tipos de alerta.

Saída de zona Emitido quando o portador é detetado fora de todas as zonas ativas. O alerta inclui a posição e a distância ao perímetro.

Bateria fraca Emitido quando a carga do dispositivo desce a 20% ou menos. Não volta a ser emitido enquanto a carga não subir acima de 30%, o que evita notificações repetidas quando o valor oscila.

Ausência prolongada de comunicação Emitido quando o dispositivo deixa de comunicar durante mais de 15 minutos. Inclui a última posição conhecida, que é a informação mais útil quando um localizador deixa de responder.

Onde chegam os alertas

Com a aplicação aberta, o alerta é apresentado imediatamente no ecrã. Com a aplicação em segundo plano ou fechada, chega como notificação. Todos os alertas ficam registados no separador de alertas, mesmo que a notificação não seja vista no momento.

Consultar e resolver

O separador de alertas apresenta a lista, do mais recente para o mais antigo. Ao abrir um alerta são apresentados a posição, a distância ao perímetro e o histórico recente de posições.

Um alerta de saída de zona é resolvido automaticamente quando o portador regressa a uma zona segura. Os restantes podem ser marcados como resolvidos manualmente. O regresso a uma zona segura não fecha alertas de outro tipo: um alerta de bateria fraca em aberto continua a assinalar um problema real.

Uma leitura isolada não gera alerta

O sistema só gera um alerta de saída de zona após duas leituras consecutivas fora de todas as zonas. Esta espera é intencional: evita que uma única leitura imprecisa junto à fronteira desencadeie um falso alarme. Do mesmo modo, quando a margem de erro de uma posição ainda toca a zona mais próxima, a leitura é tratada como incerta e não contribui para o alerta.

B.8 Histórico

O separador de histórico apresenta o percurso do dispositivo de duas formas complementares: um trilho desenhado sobre o mapa e uma lista cronológica das posições registadas. Existem filtros temporais e um painel com estatísticas do período selecionado.

B.9 Estado do dispositivo

O ecrã de definições do dispositivo permite consultar o estado de comunicação e de bateria, atribuir um nome ao dispositivo e desfazer o emparelhamento.

O nível de bateria apresentado é uma estimativa obtida a partir da tensão da célula. Deve ser lido como indicação relativa e não como medida exata da carga restante — em particular, a descida entre 100% e 80% não corresponde ao mesmo tempo de utilização que a descida entre 30% e 10%.

B.10 Cuidados com o dispositivo

- Carregue o dispositivo diariamente. A autonomia depende das condições de utilização e é significativamente inferior à de um telemóvel.

- Não retire as antenas. O funcionamento correto depende das duas: uma para a rede móvel e outra para a receção de satélite.

- Mantenha o cartão MicroSD inserido. É nele que ficam guardadas as posições recolhidas em zonas sem cobertura, que são enviadas assim que a ligação for restabelecida.

- Verifique periodicamente o saldo ou o plano do cartão SIM. Sem dados móveis o dispositivo continua a registar posições localmente, mas o cuidador deixa de as receber.

B.11 Resolução de problemas

Tabela B.1. – Resolução de problemas correntes.

Sintoma	Causa provável	O que fazer
Marcador cinzento e aviso de ausência de sinal	Sem cobertura de rede móvel no local, ou dispositivo desligado	Aguardar alguns minutos; verificar a carga do dispositivo
Posição imprecisa ou desatualizada em casa	Posicionamento por Wi-Fi, usado quando não há sinal de satélite	Comportamento normal em interior; a precisão melhora no exterior
Não recebo notificações	Autorização de notificações recusada no telemóvel	Ativar as notificações da aplicação nas definições do telemóvel
Emparelhamento recusado	Identificador ou código incorretos, ou dispositivo já associado	Confirmar os dados; se necessário, libertar o dispositivo na conta anterior
Alerta de bateria fraca repetido	Carga a oscilar em torno do limiar	Carregar o dispositivo até acima de 30%
Alerta de ausência de comunicação	Dispositivo sem bateria, sem cobertura ou avariado	Consultar a última posição conhecida no alerta e verificar o equipamento

B.12 Privacidade

O sistema recolhe e conserva o histórico de posições da pessoa monitorizada. Esses dados residem na infraestrutura de nuvem do próprio cuidador, alojada em território da União Europeia, e não são partilhados com terceiros nem acessíveis a outros utilizadores da aplicação.

O cuidador é o responsável pelo tratamento destes dados e pode eliminá-los na totalidade a qualquer momento. Recomenda-se que a utilização do sistema seja precedida do consentimento informado da pessoa monitorizada, sempre que esta o possa prestar, ou da intervenção do seu representante legal quando não o possa.

C Manual Técnico

C.1 Âmbito

Este manual destina-se a quem tenha de instalar, configurar, compilar, implantar ou manter o sistema. Descreve o procedimento completo desde um computador sem nada instalado até um dispositivo a reportar posição para a aplicação do cuidador. A utilização corrente da aplicação está descrita no Anexo B.

C.2 Estrutura do repositório

O projeto está organizado num monorepo, com uma pasta de topo por camada e a configuração do Firebase na raiz.

`firmware/` Firmware C++ para a placa LilyGO T-SIM7000G, gerido por PlatformIO.

`functions/` Cloud Functions em Node.js: ingestão, geofencing, saúde do dispositivo, notificação e geolocalização.

`app/` Aplicação Flutter do cuidador.

`simulator/` Simulador de dispositivo em Node.js.

`docs/` Documentação de apoio.

`design/` Sistema de design e maquetas dos ecrãs.

Raiz `firestore.rules`, `firestore.indexes.json`, `firebase.json`.

A opção pelo monorepo é deliberada: a semântica de decisão dentro ou fora de uma zona existe simultaneamente no backend (`functions/geo.js`) e na aplicação (`app/lib/services/geo_utils.dart`). Manter os dois no mesmo repositório garante que qualquer alteração ao algoritmo é feita e revista num único *commit*.

C.3 Pré-requisitos e versões

As versões indicadas são as efetivamente utilizadas no desenvolvimento e validação do sistema.

Tabela C.1. – Versões do ambiente de desenvolvimento.

Componente	Versão
PlatformIO Core	6.1.19
Plataforma `espressif32`	7.0.1
Framework Arduino para ESP32	3.20017 (Arduino-ESP32 2.0.17, ESP-IDF v4.4.7)
`toolchain-xtensa-esp32`	8.4.0+2021r2-patch5
TinyGSM	0.12.0
ArduinoJson	7.4.3
StreamDebugger	1.0.1
Node.js	22
`firebase-functions`	5.1.1
`firebase-admin`	12.6.0
Firebase CLI	15.25.1
Flutter	3.44.7 (canal *stable*)
Dart SDK	3.12.2
Xcode	26.6 (build 17F113)
Visual Studio Code	1.136.1

C.4 Configuração do projeto Firebase

1. Criar um projeto na consola do Firebase.

2. Ativar os serviços: *Authentication* (método de correio eletrónico e palavra-passe), *Cloud Firestore*, *Cloud Functions*, *Cloud Messaging* e *Cloud Scheduler*. Este último é obrigatório: sem ele a função agendada de verificação de saúde não é implantada.

3. Definir a região `europe-west1` para o Firestore e para as funções. A região é relevante por dois motivos: reduz a latência de ingestão e mantém os dados em território da União Europeia.

4. Na Google Cloud Console do mesmo projeto, ativar a *Geolocation API* e a *Geocoding API*.

Chaves da Google: três chaves distintas

O sistema utiliza três chaves da Google, e não devem ser reutilizadas entre si:

- **Maps SDK**, usada pela aplicação para desenhar o mapa, restringida ao *bundle* iOS da aplicação;
- **Geolocation API**, usada pelo backend para resolver listas de redes Wi-Fi em coordenadas; a chamada parte de um servidor, pelo que esta chave não pode ter restrição por aplicação;
- **Geocoding API**, usada pela aplicação para a pesquisa de endereços no editor de zonas.

Reutilizar a chave do Maps para a chamada de servidor obrigaria a remover a restrição por aplicação, expondo-a a uso indevido.

C.5 Segredos e configuração

Nenhum segredo se encontra sob controlo de versões. O repositório contém três ficheiros de exemplo, versionados, que devem ser copiados e preenchidos:

```
cp firmware/src/config.example.h          firmware/src/config.h
cp functions/.env.example                 functions/.env
cp app/lib/config/secrets.example.dart    app/lib/config/secrets.dart
```

firmware/src/config.h

Tabela C.2. – Parâmetros de configuração do firmware.

Parâmetro	Significado	Valor
DEVICE_ID	Identificador; tem de existir em devices/{id}	—
DEVICE_KEY	Chave partilhada; igual à do functions/.env	—
SERVER_HOST	europe-west1-<projeto>.cloudfunctions.net	—
SERVER_PATH	Caminho da função de ingestão	/ingestTelemetry
SERVER_PORT	Porta HTTPS	443
GPRS_APN	APN do operador (Things Mobile)	TM
GPRS_USER, GPRS_PASS	Credenciais do APN	vazias
INTERVAL_S	Deep sleep entre ciclos	20 s
GPS_TIMEOUT_S	Orçamento total de aquisição	90 s
GPS_FAST_TIMEOUT_S	Fase rápida, com rádio ligado	20 s
GPS_INDOOR_TIMEOUT_S	Orçamento reduzido em interior	15 s
INDOOR_APOS_N_FALHAS	Ciclos sem fix que caracterizam interior	2
WIFI_FALLBACK	Ativa o varrimento Wi-Fi	1
WIFI_MAX_APS	Máximo de pontos de acesso enviados	12

Colocar GPS_FAST_TIMEOUT_S a zero repõe o comportamento anterior, em que a aquisição decorria sempre com o rádio celular desligado.

functions/.env

DEVICE_KEY Chave partilhada validada no cabeçalho X-Device-Key. Tem de coincidir com a do firmware e com a do simulador. Gerar com openssl rand -hex 32.

TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID Credenciais do canal de notificação alternativo. O *token* obtém-se junto do @BotFather; o identificador de conversa exige que se envie previamente uma mensagem ao *bot*, uma vez que um *bot* não pode iniciar conversas. O campo telegramChatId no documento do utilizador, quando definido, tem prioridade sobre este valor global.

`GEOLOCATION_API_KEY` Chave da Geolocation API, restringida a essa API e a nenhuma outra.

`app/lib/config/secrets.dart`

Contém a chave da Geocoding API, usada na pesquisa de endereços. O fluxo de integração contínua copia o ficheiro de exemplo antes de compilar, o que é suficiente para a análise estática e para os testes unitários, uma vez que nenhum teste efetua chamadas reais a serviços externos.

C.6 Implantação do backend

```
cd functions && npm install
cd ..
firebase deploy --only functions
firebase deploy --only firestore:rules,firestore:indexes
```

Comportamento a ter presente

Alterações efetuadas apenas ao ficheiro de variáveis de ambiente não são detetadas pela CLI, que compara o resumo criptográfico do código. A propagação de uma nova chave exige, por isso, uma alteração ao código ou uma implantação forçada.

C.7 Provisionar um dispositivo

Antes de o firmware poder enviar telemetria, o dispositivo tem de existir no Firestore. Criar o documento `devices/{deviceId}` exclusivamente através do Admin SDK, com, no mínimo, o código de emparelhamento de seis dígitos e sem proprietário atribuído. É esse código que o cuidador introduz na aplicação; as regras de segurança só aceitam a reivindicação de um dispositivo sem proprietário se o código enviado coincidir com o guardado, e o cliente reenvia o código mas nunca o lê.

C.8 Compilar e gravar o firmware

O projeto define três ambientes de compilação a partir da mesma base de código:

`esp32dev` Ambiente de produção, com o ciclo normal.

`diag` Substitui o ciclo por uma bateria de testes de transporte HTTP e HTTPS isolados.

`gnss` Executa um ciclo de alimentação limpo do recetor GNSS e imprime a trama `CGNSINF` em bruto, distinguindo satélites visíveis de satélites utilizados.

```
cd firmware
pio run -t upload            # ambiente de producao
pio device monitor           # 115200 baud

pio run -e diag -t upload    # diagnostico de transporte
pio run -e gnss -t upload    # diagnostico do receptor GNSS
```

Se o comando `pio` não estiver disponível no terminal, encontra-se em `~/.platformio/penv/bin/` quando o PlatformIO foi instalado através da extensão do Visual Studio Code.

Portas série

A configuração fixa as portas através do padrão `/dev/cu.*usb*`. Sem esta fixação, a deteção automática em macOS seleciona a primeira porta série enumerada pelo sistema, que é a porta de entrada Bluetooth; o erro resultante, `Failed to connect to ESP32: No serial data received`, é facilmente confundido com uma avaria de hardware. O padrão cobre os dois circuitos de conversão USB para série usados nas revisões da placa, o CP210x e o CH9102F.

Opções de compilação

`TINY_GSM_MODEM_SIM7000SSL` delega o estabelecimento da ligação segura na pilha do próprio modem; `TINY_GSM_RX_BUFFER=1024` evita o truncamento das respostas às interrogações de estado do GNSS e às operações HTTP.

C.9 Simulador de dispositivo

O simulador injeta telemetria sintética no mesmo *endpoint* do dispositivo real, o que permite exercitar o backend e a aplicação sem hardware.

```
cd simulator && npm install
node simulate.js --escape        # afasta-se ate sair da zona
node simulate.js --inside        # permanece sempre dentro
node simulate.js --return        # sai e regressa
node simulate.js --indoor        # sem GPS: envia BSSIDs de WiFi
node simulate.js --inside --once # envia um unico ponto
```

A variável `DEVICE_KEY` tem de coincidir com a do backend.

C.10 Testes

```
cd functions
npm test                 # 43 testes de logica pura, sem emulador
npm run test:integration # exige o emulador do Firestore

cd ../app
flutter analyze lib
flutter test
```

Os testes de lógica pura estão organizados em três ficheiros — `geo.test.js`, `health.test.js` e `indoor.test.js` — e não necessitam de emulador, credenciais

ou rede. A aplicação dispõe de quatro conjuntos de testes, incidindo sobre o serviço de dispositivo, os utilitários geométricos, o modelo de zona e um componente de interface.

A integração contínua está definida em `.github/workflows/ci.yml` e executa, em cada *push* e *pull request*, os testes do backend, a análise estática e os testes unitários da aplicação. Os testes de integração ficam deliberadamente fora deste circuito por exigirem a inicialização do emulador.

C.11 Modelo de dados

`users/{uid}` Perfil do cuidador, incluindo o *token* FCM e, opcionalmente, o identificador de conversa do Telegram.

`devices/{deviceId}` Estado do dispositivo: proprietário, código de emparelhamento, último contacto, bateria, contador de leituras consecutivas fora de zona e sinalizadores de alerta ativo.

`devices/{deviceId}/telemetry/{pointId}` Histórico de posições, com origem e precisão associada.

`devices/{deviceId}/geofences/{zoneId}` Zonas seguras: nome, tipo, geometria, estado de ativação e cor.

`alerts/{alertId}` Alertas gerados, com tipo, dispositivo, posição, zona de referência e estado de resolução.

A subcoleção `geofences` substituiu um documento único `geofence/current`, mantido por retrocompatibilidade. A migração é executada no máximo uma vez por dispositivo e regista um sinalizador para não voltar a correr.

C.12 Regras de segurança

As regras em `firestore.rules` obedecem a quatro princípios: o cuidador acede exclusivamente aos seus próprios dados; a telemetria e os alertas são de escrita reservada às Cloud Functions, que operam através do Admin SDK; e o dispositivo nunca comunica diretamente com o Firestore, pelo que não necessita de regras próprias. Autentica-se perante a função de ingestão através do cabeçalho `X-Device-Key`.

O quarto princípio veda ao cliente a criação de documentos de dispositivo, através de `allow create: if false`. O provisionamento é efetuado exclusivamente pelo Admin SDK, conforme o Anexo C.7, e o emparelhamento consiste na atualização de um documento existente. Esta regra foi corrigida e implantada em 6 de setembro de 2026, após TF-02.2 revelar que a criação anteriormente permitida dispensava o código de emparelhamento. TF-02.6 confirmou que o emparelhamento legítimo continuava a funcionar após a correção.

C.13 Limitações conhecidas

Validação de certificado A ligação do dispositivo à nuvem usa TLS 1.2 sem validação do certificado do servidor. A causa é um defeito da firmware do módulo SIM7000G, confirmado com três âncoras de confiança distintas e com o relógio do modem sincronizado. O canal é cifrado, mas o servidor não é autenticado.

Chave partilhada única A chave de autenticação do dispositivo é comum, sem rotação, sem assinatura do corpo do pedido e sem proteção contra repetição.

Notificação nativa em iOS Exige uma chave APNs, disponível apenas com conta paga do Apple Developer Program. O canal FCM está integralmente implementado; até essa chave existir, a entrega é assegurada pelo canal alternativo via Telegram.

Autenticação por conta Google Implementada, mas não operacional na configuração atual, por o cliente OAuth estar associado ao identificador de *bundle* anterior.

C.14 Diagnóstico de problemas correntes

Tabela C.3. – Diagnóstico técnico.

Sintoma	Verificação
`Failed to connect to ESP32`	Porta série errada; confirmar que o padrão `/dev/cu.*usb*` encontra a placa e não a porta Bluetooth
Resposta 401 na ingestão	`DEVICE_KEY` diferente entre firmware e `functions/.env`; recordar que alterar apenas o `.env` não é detetado pela CLI
Resposta 422 na ingestão	*Payload* correto, mas a Google não resolveu a lista de redes Wi-Fi em coordenadas
Sem fix GNSS no exterior	Antena GPS desligada ou trocada com a celular; usar o ambiente `gnss` para ver satélites visíveis e utilizados
Ciclos muito mais longos que `INTERVAL_S`	Registo celular a ser restabelecido em cada ciclo; confirmar que o modem e a rede são mantidos ativos entre ciclos
Função agendada não implanta	Cloud Scheduler API não ativada no projeto